How Do We Compare Biological Sequences?

Dynamic Programming and Divide-and Conquer Algorithms

Phillip Compeau and Pavel Pevzner.

Bioinformatics Algorithms: An Active Learning Approach

©2018 by Compeau and Pevzner. All rights reserved.

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

How Do We Compare Biological Sequences

- **The Alignment Game and the Longest Common Subsequence**
- The Manhattan Tourist Problem
- Dynamic Programming and Backtracking Pointers
- From Manhattan to the Alignment Graph
- From Global to Local Alignment
- Penalizing Insertions and Deletions in Sequence Alignment
- Space-Efficient Sequence Alignment
- Multiple Sequence Alignment

Copyright 2018 Compeau and Pevzner.

Sequence Alignment

- The Hamming distance, which counts mismatches in two strings, rigidly assumes that we align the *i*-th symbol of one sequence against the *i*-th symbol of the other.

- However, since biological sequences are subject to insertions and deletions, it is often the case that the *i*-th symbol of one sequence corresponds to a symbol at a completely different position in the other sequence.

- The goal is to find the most appropriate correspondence of symbols.

Example

- ATGCATGC and TGCATGCA have no matching positions, and so their Hamming distance is equal to 8:
- **ATGCATGC**
  TGCATGCA
- These strings have seven matching positions if we align them differently:
- **ATGCATGC-**
  - TGCATGCA
- Strings ATGCTTA and TGCATTAA have more subtle similarities:
- **ATGC -TTA-**
  - TGCATTAA

Good alignment: Matches as many symbols as possible.

How to find matched symbols?

- Maximizing the number of matched symbols in two strings: You have two choices.
  - You can remove the first symbol from each sequence,
    - you earn a point if the symbols match
  - You can remove the first symbol from either of the two sequences
    - you earn no points but may set yourself up to earn more points in later moves.

- Your goal is to maximize the number of points.

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.
5

The Alignment Game

```
A T G T T A T A
A T C G T C C
```

Alignment Game (maximizing the number of points):

- Remove the 1st symbol from each sequence
  - 1 point if the symbols match, 0 points if they don't match
- Remove the 1st symbol from one of the sequences
  - 0 points

Copyright 2018 Compeau and Pevzner.

The Alignment Game

```
 A T G T T A T A
 A T C G T C C
+1
```


Copyright 2018 Compeau and Pevzner.

The Alignment Game

```
 A T G T T A T A
 A T C G T C C
+1+1
```


Copyright 2018 Compeau and Pevzner.

The Alignment Game

```
A T - G T T A T A
A T C G T C C
+1+1
```


Copyright 2018 Compeau and Pevzner.

The Alignment Game

```
 A T - G T T A T A
 A T C G T C C
+1+1  +1
```


Copyright 2018 Compeau and Pevzner.

The Alignment Game

```
 A T - G T T A T A
 A T C G T C C
+1+1  +1+1
```


Copyright 2018 Compeau and Pevzner.

The Alignment Game

```
 A T - G T T A T A
 A T C G T - C C
+1+1  +1+1
```


Copyright 2018 Compeau and Pevzner.

The Alignment Game

```
 A T - G T T A T A
 A T C G T - C C
+1+1  +1+1
```


Copyright 2018 Compeau and Pevzner.

The Alignment Game

```
 A T - G T T A T A
 A T C G T - C - C
+1+1  +1+1
```


Copyright 2018 Compeau and Pevzner.

The Alignment Game

```
 A T - G T T A T A
 A T C G T - C - C
+1+1  +1+1       =4
```


Copyright 2018 Compeau and Pevzner.

Sequence Alignment: Definition

- Define an alignment of sequences *v* and *w* as a two-row matrix such that
  - the first row contains the symbols of *v* (in order)
  - the second row contains the symbols of *w* (in order)
  - space symbols (called gap symbols and shown as dashes) may be interspersed throughout both strings
    - two space symbols are not aligned against each other.
- The alignment of ATGTTATA and ATCGTCC:

  A T - G T T A T A
  A T C G T - C - C

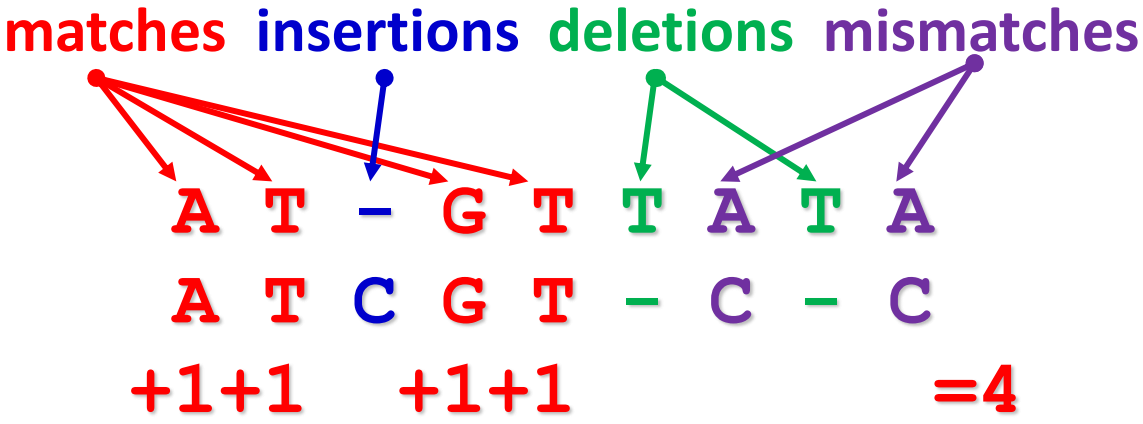
- An alignment presents one possible scenario by which *v* could have evolved into *w*.
- The alignment above has four matches, two mismatches, one insertion, and two deletions.

Sequence Alignment: Definition – cnt.

- Matches: Columns containing the same letter in both rows
- Mismatches: Columns containing different letters which represent single-nucleotide substitutions.
- Indels: Columns containing a space symbol:
  - Insertion: Column containing a space symbol in the top row of the alignment
    - implies the insertion of a symbol when transforming *v* into *w*;
  - Deletion: Column containing a space symbol in the bottom row of the alignment
    - indicates the deletion of a symbol when transforming *v* into *w*.

- The matches in an alignment of two strings define a **common subsequence** of the two strings, or a sequence of symbols appearing in the same order (although not necessarily consecutively) in both strings.

What Is the Sequence Alignment?



Alignment of two sequences is a two-row matrix:

1st row: symbols of the 1st sequence (in order) interspersed by "-"
2nd row: symbols of the 2nd sequence (in order) interspersed by "-"

Copyright 2018 Compeau and Pevzner.

Longest Common Subsequence

```
A T - G T T A T A
A T C G T - C - C
```

Matches in alignment of two sequences (**ATGT**) form their **Common Subsequence**

Longest Common Subsequence Problem: Find a longest common subsequence of two strings.

- **Input:** Two strings.
- **Output:** A longest common subsequence of these strings.

Copyright 2018 Compeau and Pevzner.

Longest Common Subsequence

- **ATGT** is a common subsequence of **ATGT**TATA and **AT**C**GT**CC.

- An alignment of two strings maximizing the number of matches therefore corresponds to a **longest common subsequence (LCS)** of these strings.

- Note that two strings may have more than one longest common subsequence.

- **Longest Common Subsequence Problem:** *Find a longest common subsequence of two strings.*
  Input: Two strings.
  Output: A longest common subsequence of these strings.

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.
20

Question

- Find all longest common subsequences of the strings ACTGCA and CATCGC

- CTGC
- ATGC
- ACGC

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.
21

Solution of the Longest Common Subsequence Problem

- There are many different ways to solve this problem
- We will start with dynamic programming
- To apply dynamic programming to the Longest Common Subsequence Problem, we will start with Manhattan Tourist Problem

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.
22

How Do We Compare Biological Sequences

- The Alignment Game and the Longest Common Subsequence
- **The Manhattan Tourist Problem**
- Dynamic Programming and Backtracking Pointers
- From Manhattan to the Alignment Graph
- From Global to Local Alignment
- Penalizing Insertions and Deletions in Sequence Alignment
- Space-Efficient Sequence Alignment
- Multiple Sequence Alignment

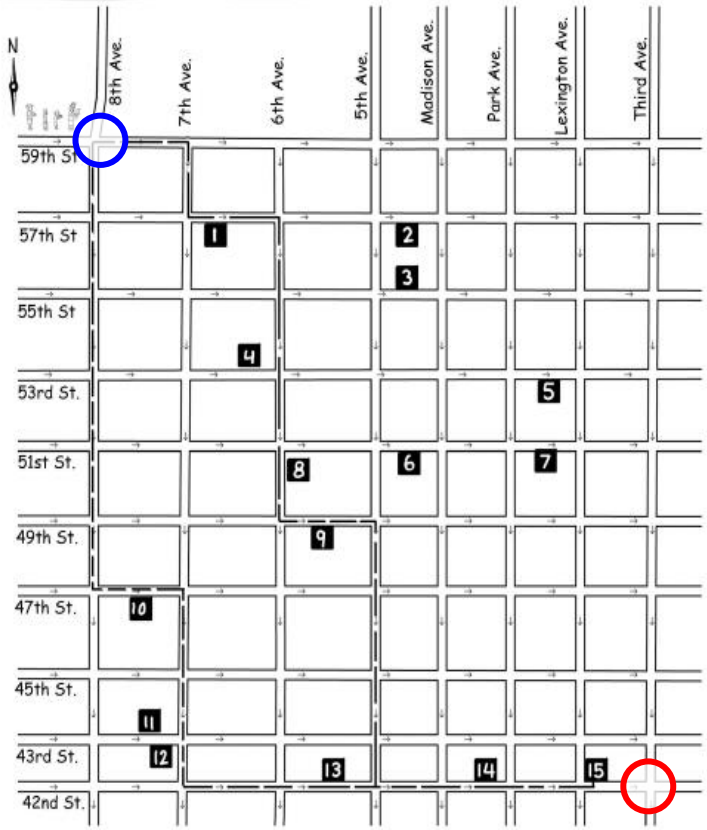
Copyright 2018 Compeau and Pevzner.

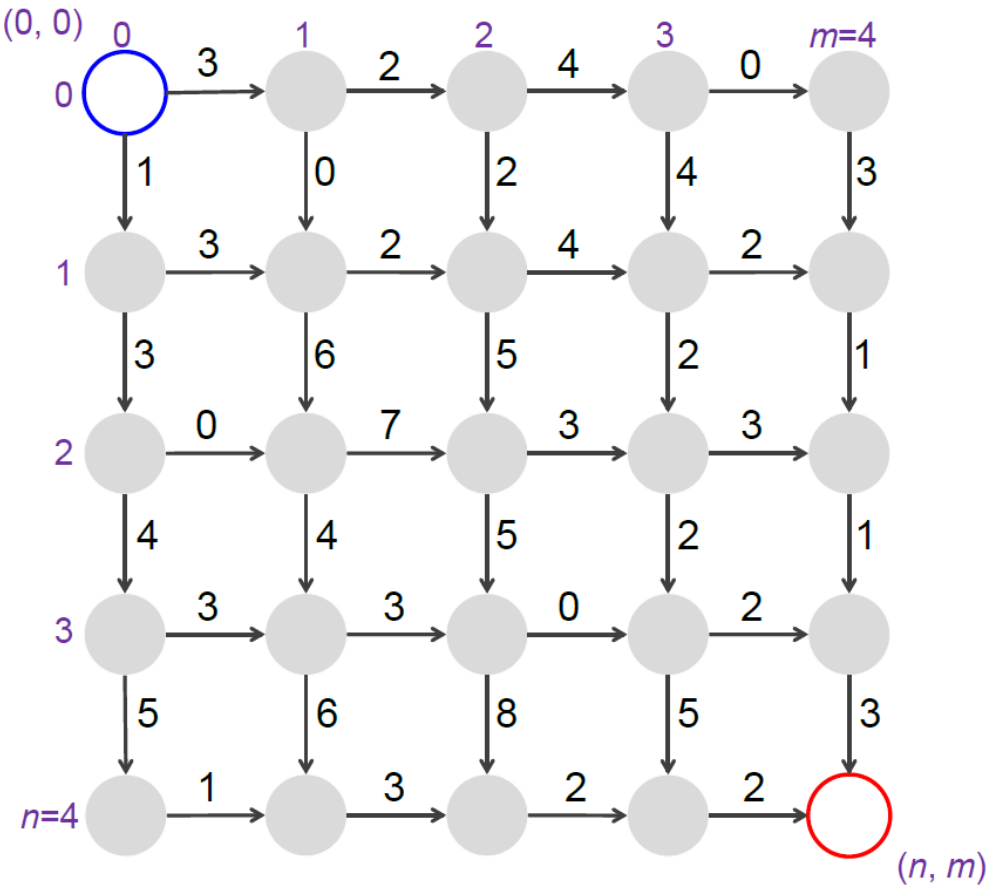
Manhattan Tourist Problem

- You are a tourist in Midtown Manhattan
- You want to see as many sights as possible on your way from the corner of 59th Street and 8th Avenue to the corner of 42nd Street and 3rd Avenue
- You are short on time, and at each intersection, you can only move south (↓) or east (→).
- You can choose from many different paths through the map, but no path will visit all the sights.
- The challenge of finding a legal path through the city that visits the most sights is called the **Manhattan Tourist Problem**.

From Manhattan to a Grid Graph







Copyright 2018 Compeau and Pevzner.

Problem Representation

- Represent the map of Manhattan as a directed graph *ManhattanGraph*
- model each intersection as a node
- represent each city block between two intersections as a directed edge indicating the legal direction of travel (↓ or →)
- Assign each directed edge a **weight** equal to the number of attractions along the corresponding block.
- The starting node is called the **source node** and the ending node is called the **sink node**.
- Adding the weights along a path from the source to the sink yields the number of attractions along that path
- To solve the Manhattan Tourist Problem, find a **maximum-weight path** connecting the source to the sink (also called a **longest path**) in *ManhattanGraph*.

Manhattan Tourist Problem

Manhattan Tourist Problem: Find a longest path in a rectangular city grid.

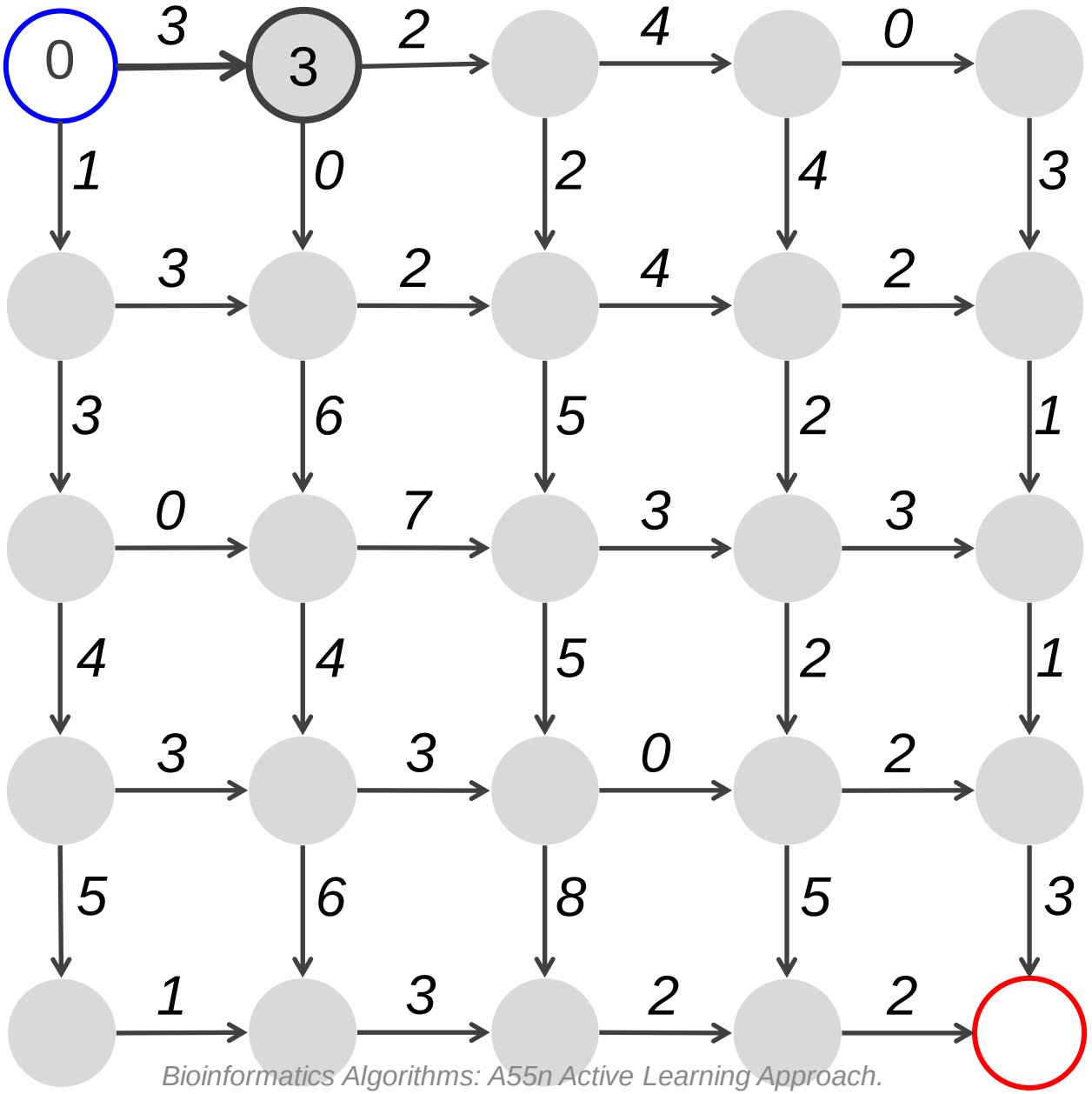
- **Input:** A weighted rectangular grid.
- **Output:** A longest path from the source to the sink in the grid.

Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Solution?

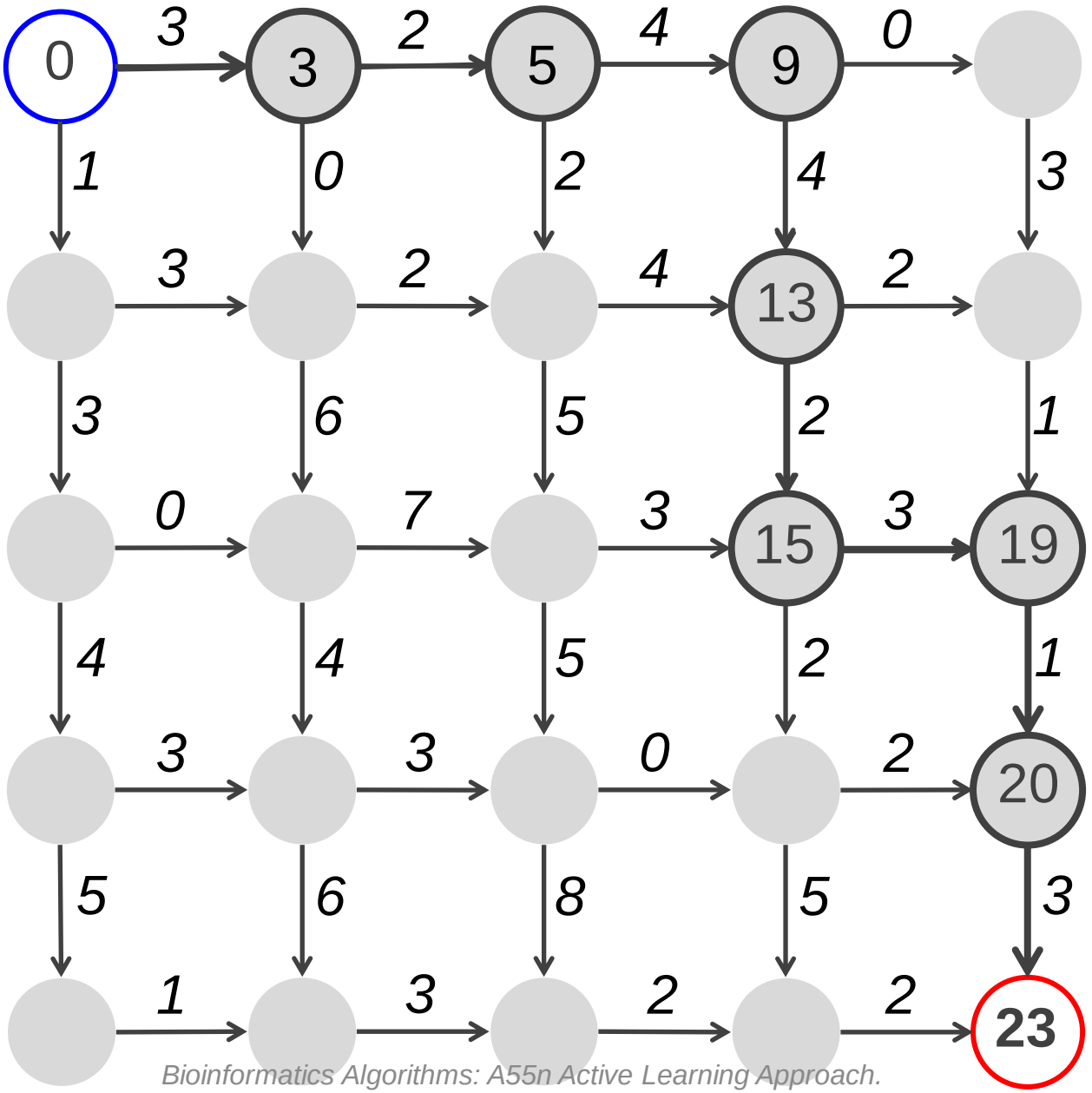
- Applying a brute force approach to the Manhattan Tourist Problem is impractical because the number of paths is huge.
- A sensible greedy approach would choose between the two possible directions at each node (→ or ↓) according to how many attractions you would see if you moved only one block south versus moving one block east.
- Unfortunately, this greedy strategy may miss the longest path in the long run.

Greedy algorithm?



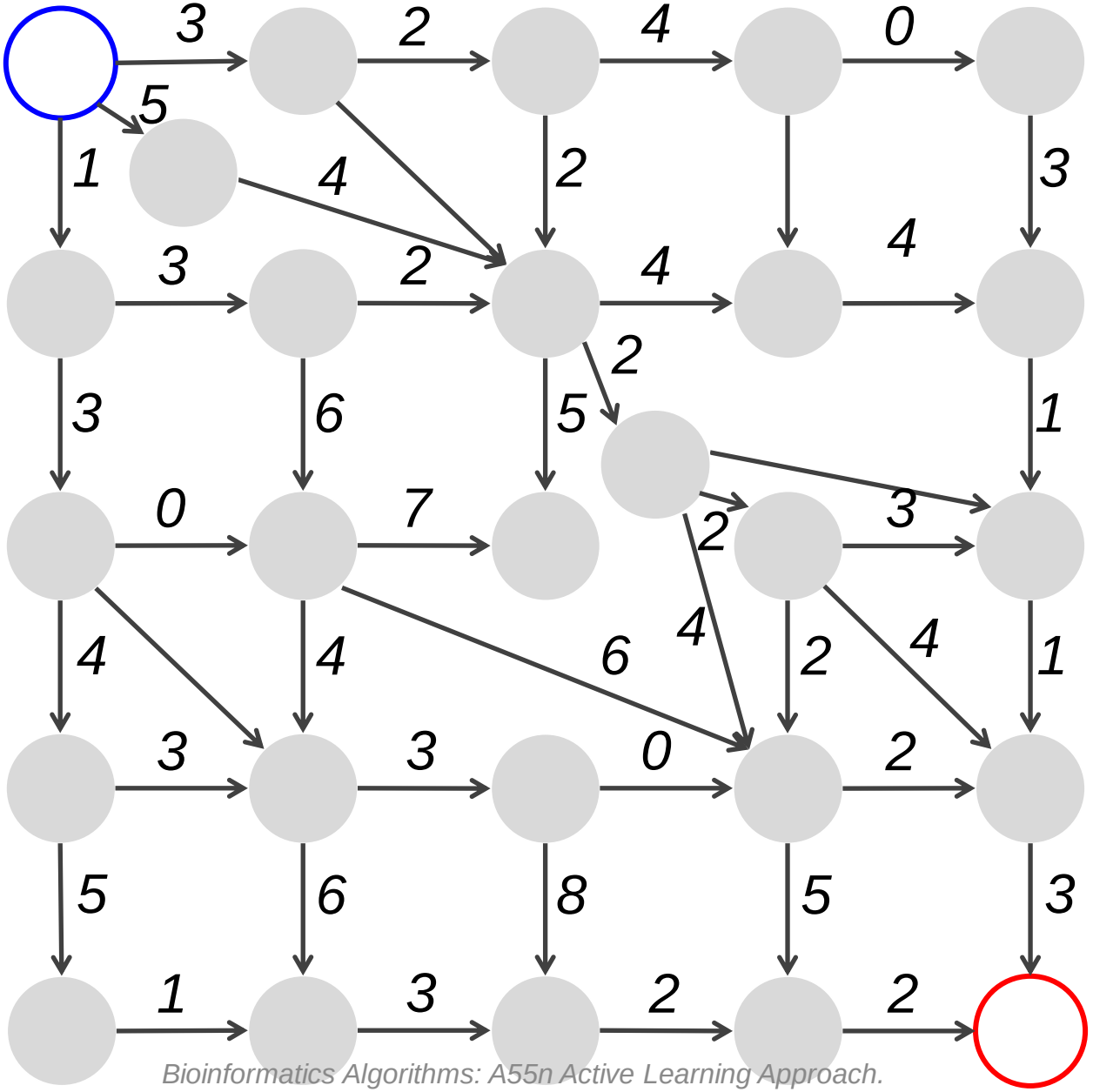
Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Greedy algorithm?



Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

From a regular to an irregular grid



Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Search for Longest Paths in a Directed Graph

Longest Path in a Directed Graph Problem: Find a longest path between two nodes in an edge-weighted directed graph.

- **Input:** An edge-weighted directed graph with source and sink nodes.
- **Output:** A longest path from source to sink in the directed graph.

Copyright 2018 Compeau and Pevzner.

Do You See a Connection between the Manhattan Tourist and the Alignment Game?

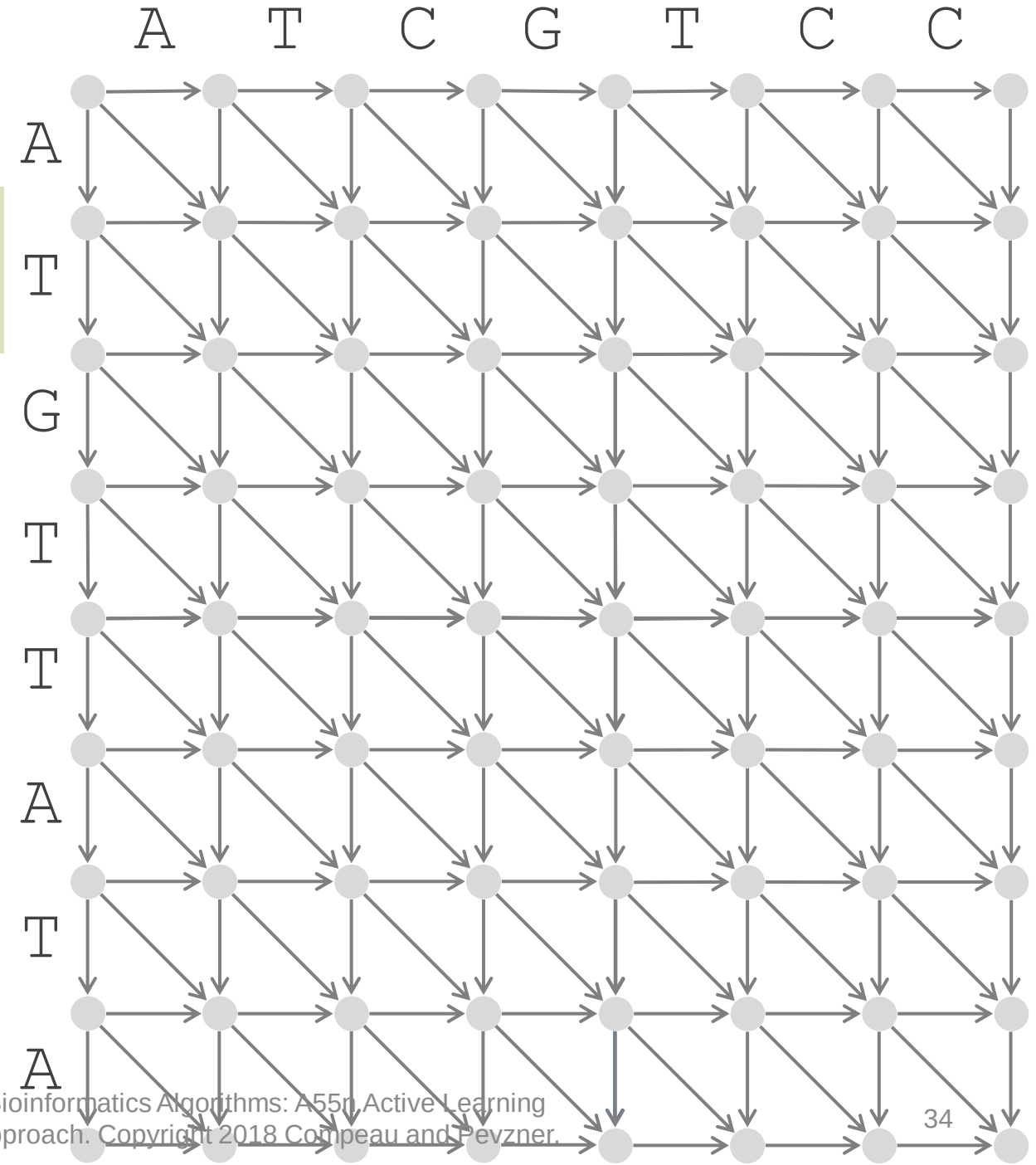
A T - G T T A T A
A T C G T - C - C
↘ ↘ → ↘ ↘ ↓ ↘ ↓ ↘

Copyright 2018 Compeau and Pevzner.

?

alignment → path

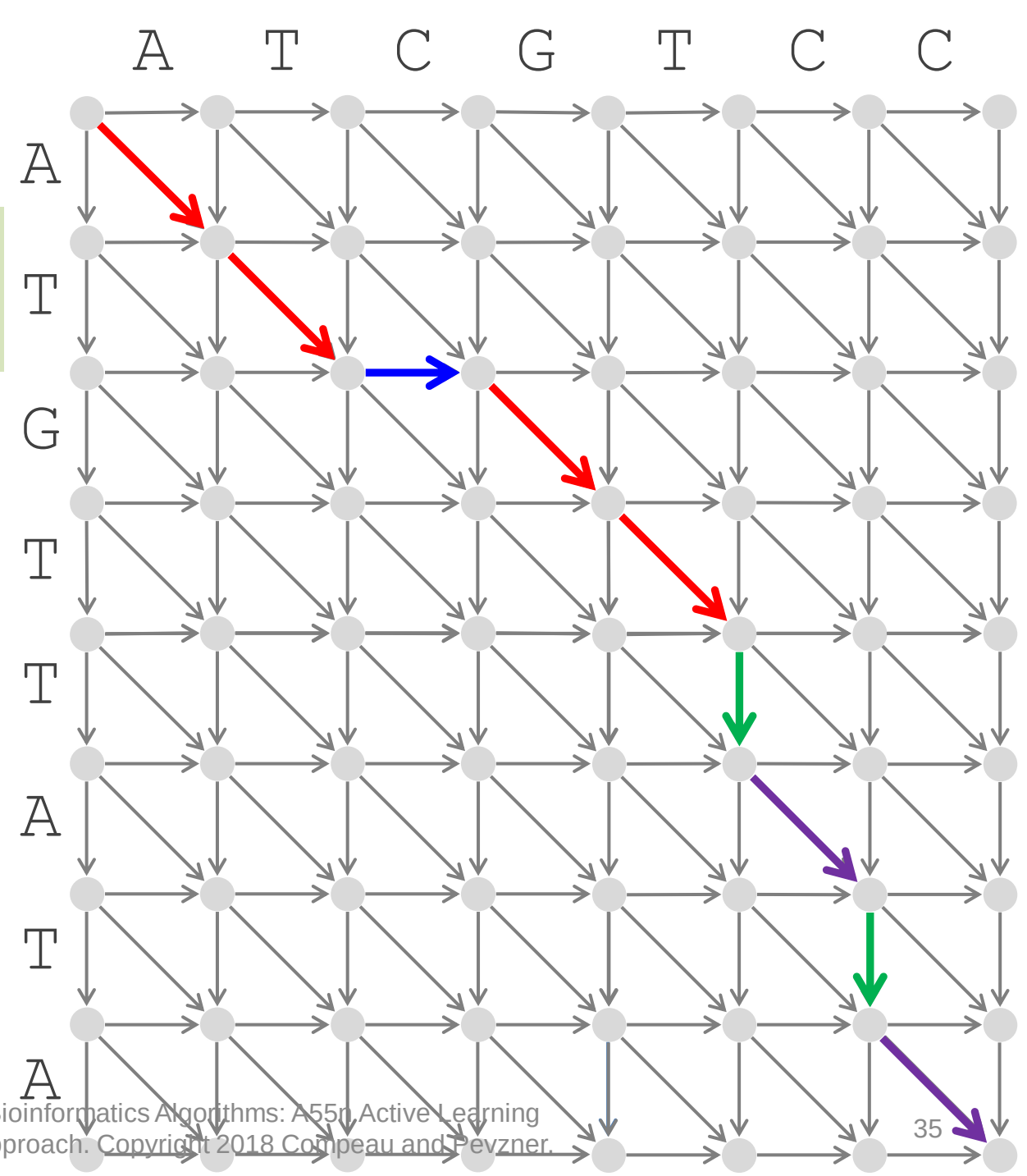
A	T	-	G	T	T	A	T	A
A	T	C	G	T	-	C	-	C
↘	↘	→	↘	↘	↓	↘	↓	↘



?

alignment → path

A	T	-	G	T	T	A	T	A
A	T	C	G	T	-	C	-	C
↘	↘	→	↘	↘	↓	↘	↓	↘



?

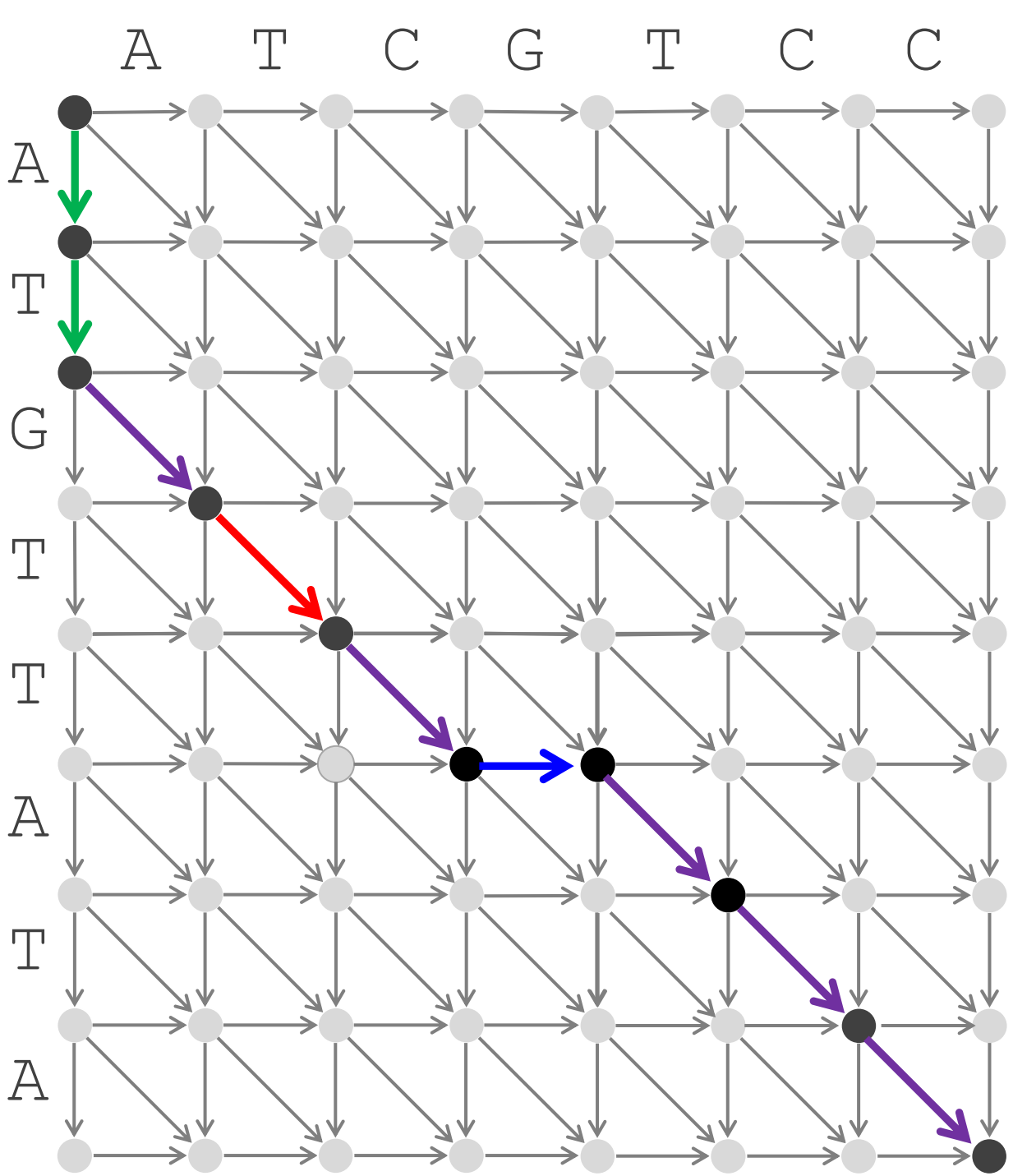
path ⟶ alignment

A	T	G	T	T	-	A	T	A
-	-	A	T	C	G	T	C	C
↓	↓	↘	↘	↘	→	↘	↘	↘

highest-scoring alignment

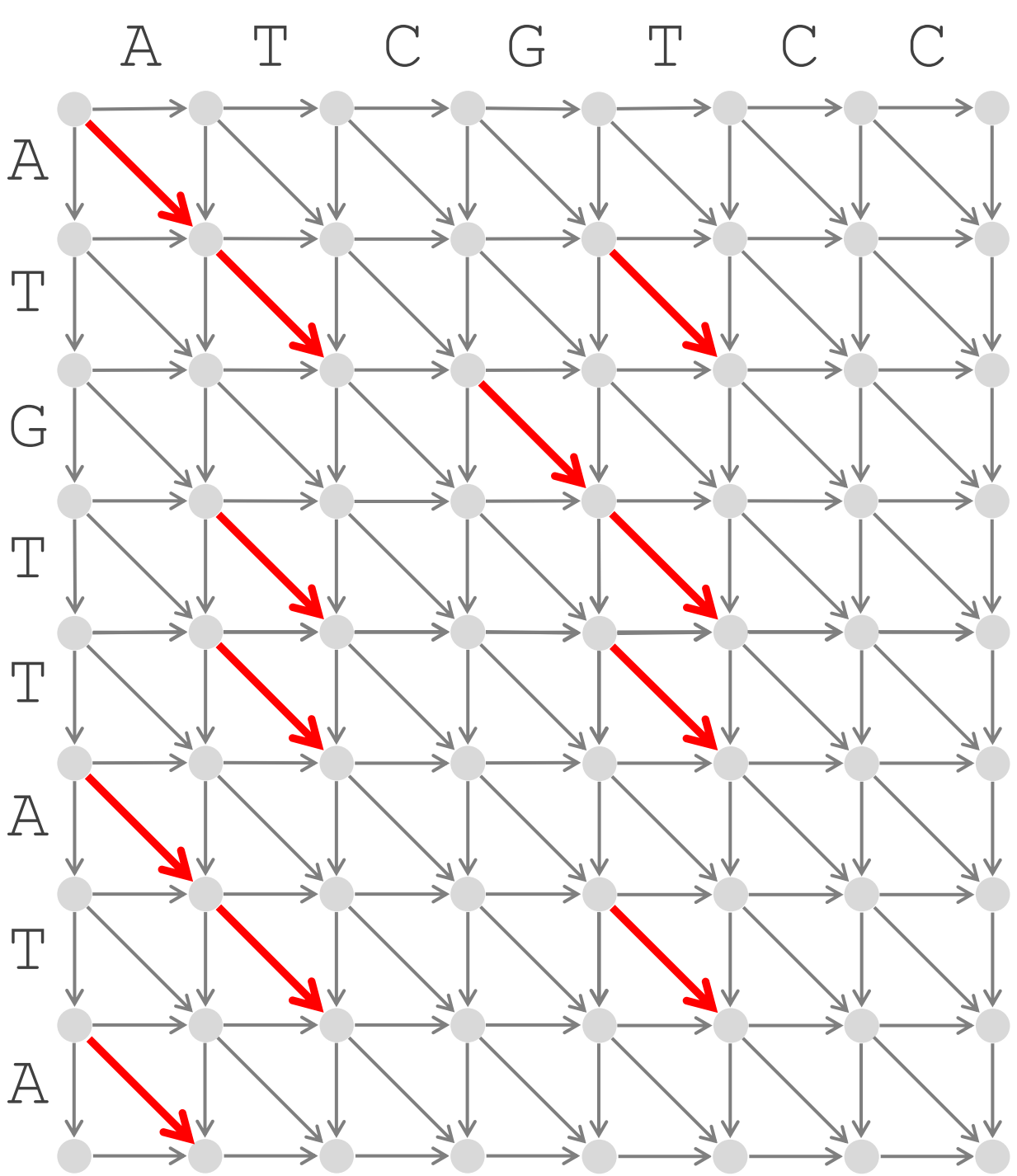
=

longest path in a properly built Manhattan



How to built a Manhattan for the Alignment Game and the Longest Common Subsequence Problem?

Diagonal red edges correspond to matching symbols and have scores 1

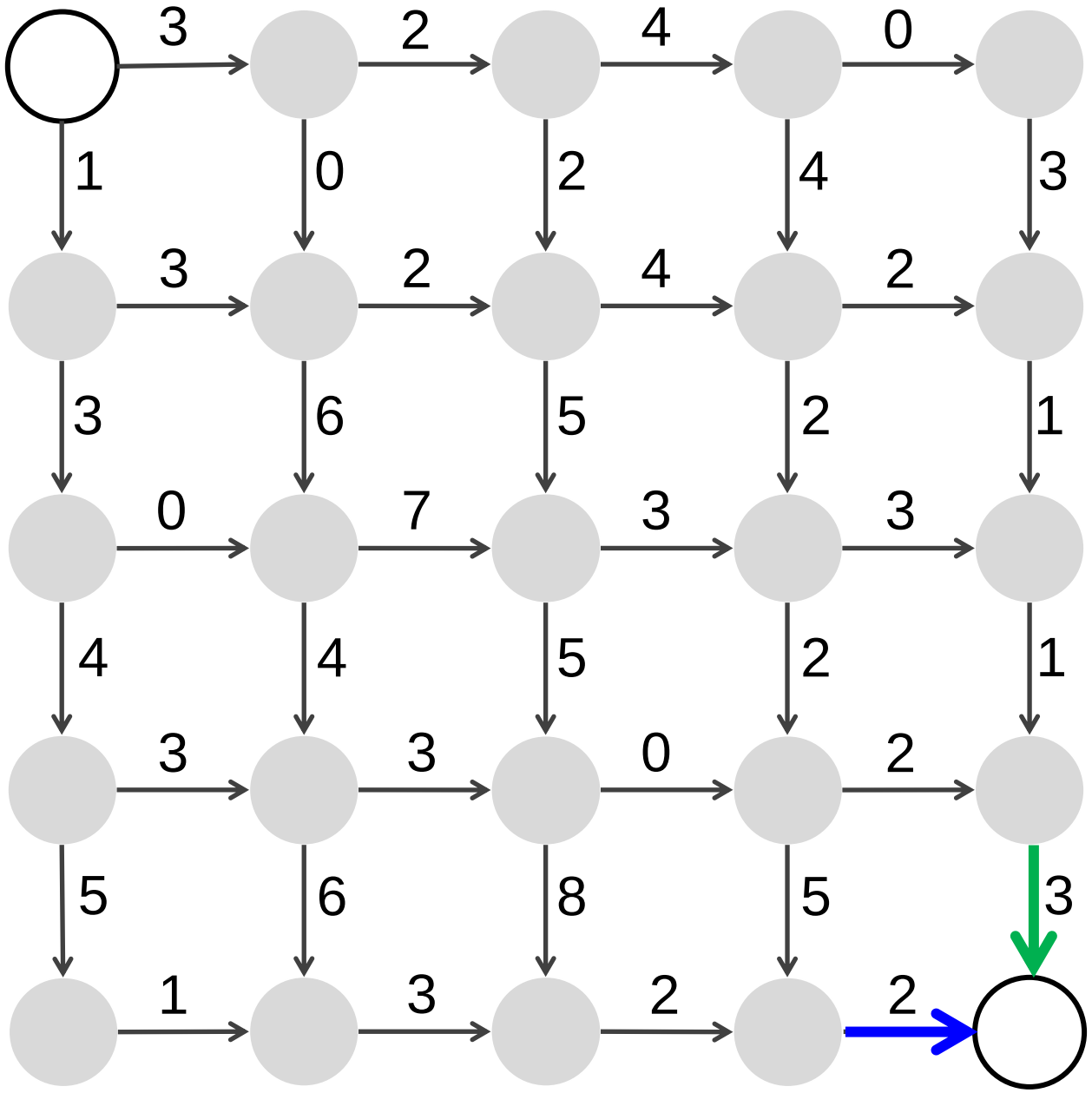


How Do We Compare Biological Sequences

- The Alignment Game and the Longest Common Subsequence
- The Manhattan Tourist Problem
- **Dynamic Programming and Backtracking Pointers**
- From Manhattan to the Alignment Graph
- From Global to Local Alignment
- Penalizing Insertions and Deletions in Sequence Alignment
- Space-Efficient Sequence Alignment
- Multiple Sequence Alignment

Copyright 2018 Compeau and Pevzner.

There are only 2 ways to arrive to the sink:
by moving **South ↓**
or by moving **East →**



South or East?

South or East?

SouthOrEast(*n,m*)

if *n*=0 and *m*=0

return 0

if *n*>0 and *m*>0

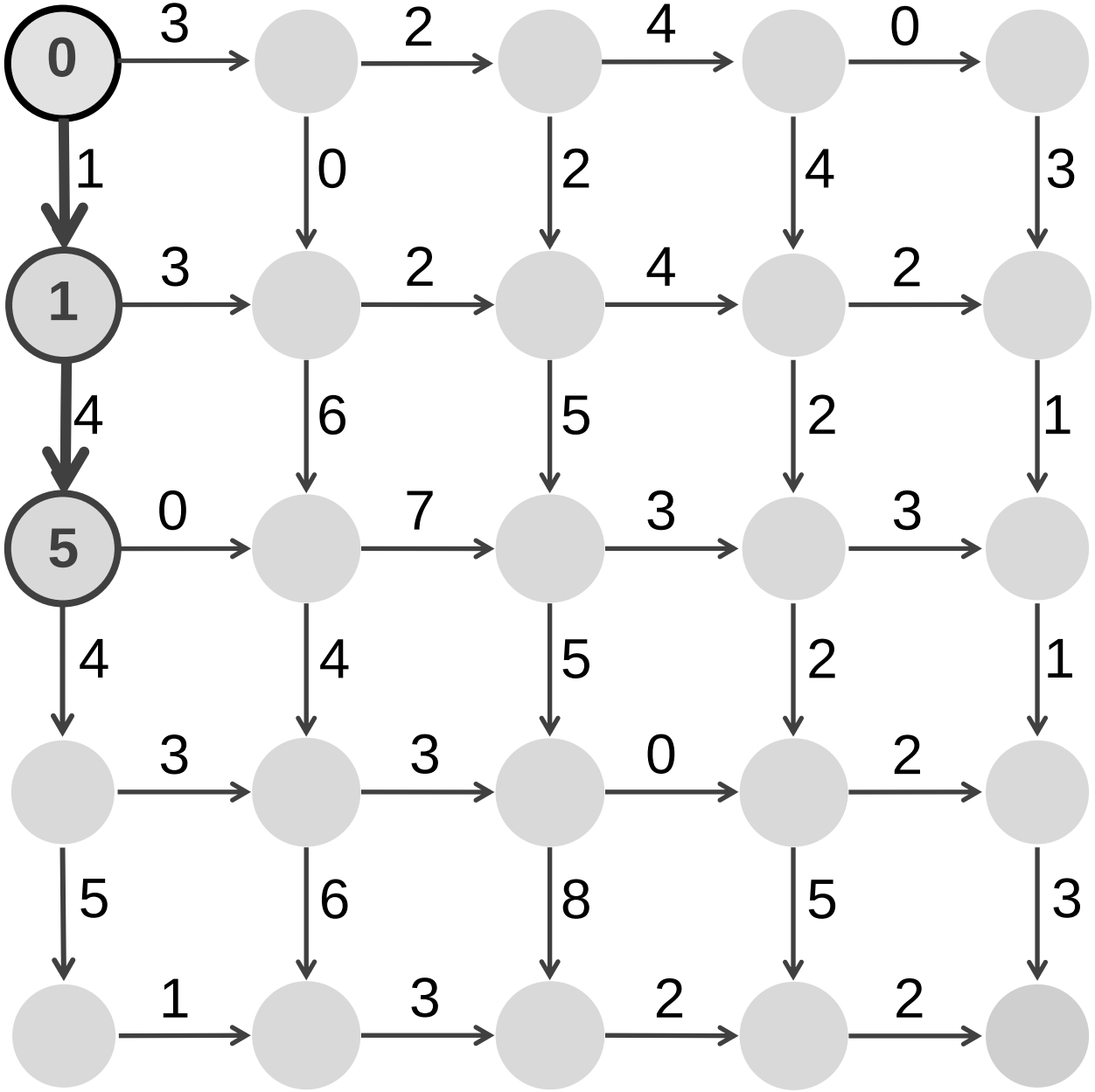
x ← **SouthOrEast**(*n-1,m*)+weight of edge "↓"into (*n,m*)

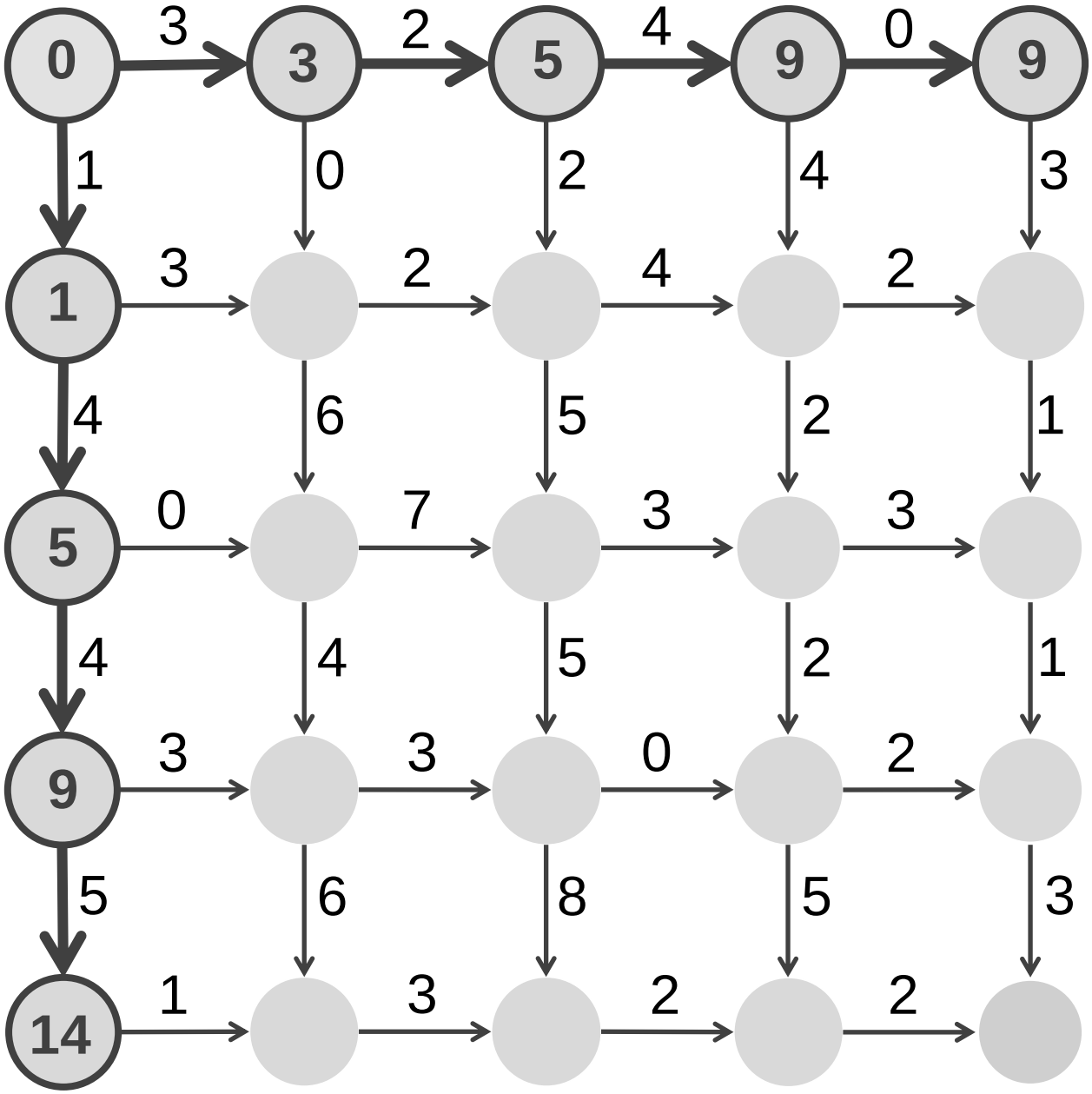
y ← **SouthOrEast**(*n,m-1*)+ weight of edge "→"into (*n,m*)

return max{*x,y*}

return -infinity

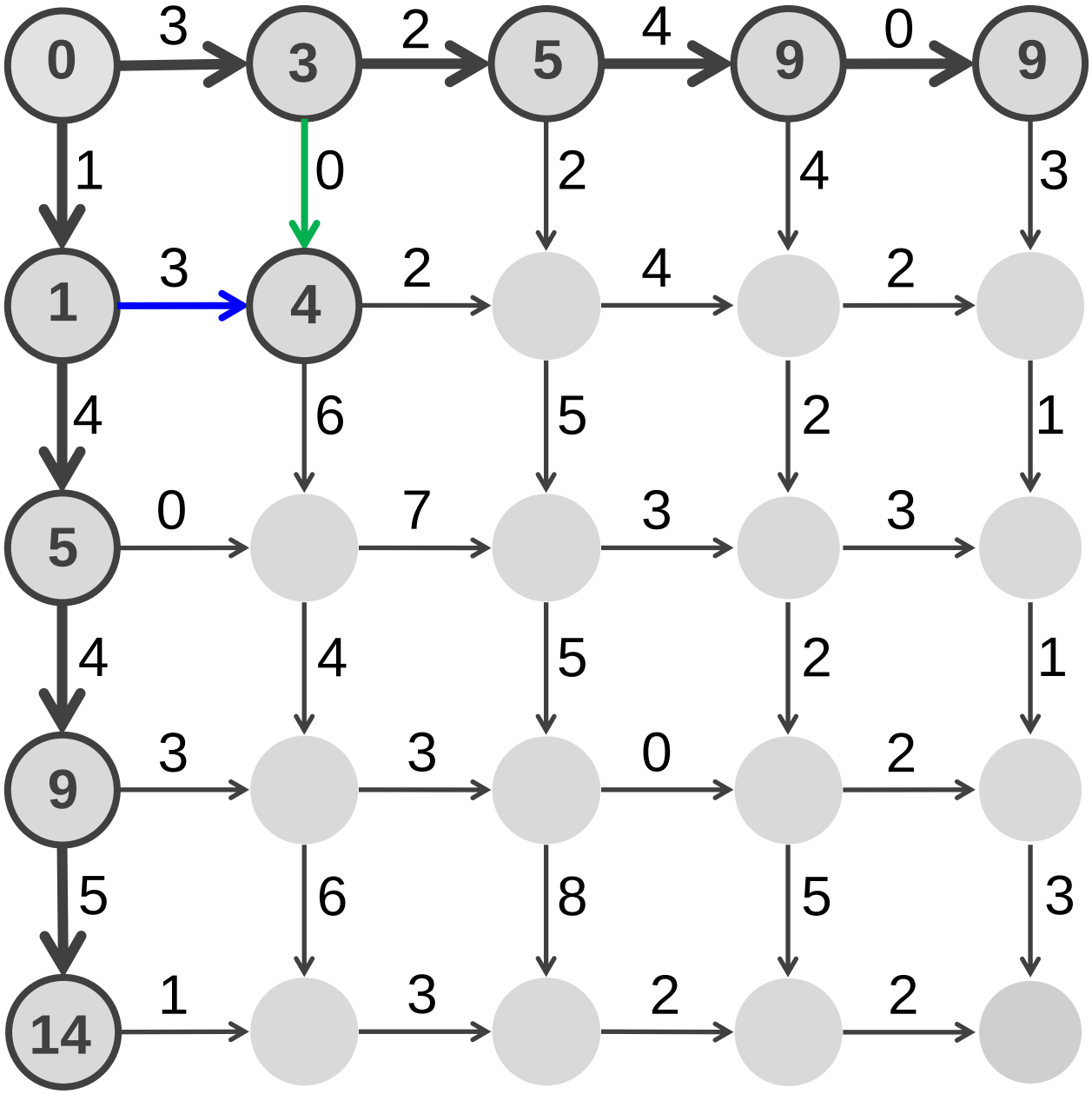
Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.


0
3
2
4
0
1
0
2
4
3
1
3
2
4
2
4
6
5
2
1
5
0
7
3
3
4
4
5
2
1
3
3
0
2
5
6
8
5
3
1
3
2
2

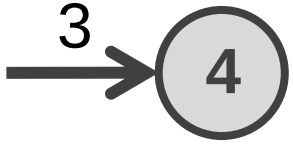

0
3
2
5
4
9
0
9
1
0
2
4
3
1
3
2
4
2
4
6
5
2
1
5
0
7
3
3
4
4
5
2
1
9
3
3
0
2
5
6
8
5
3
14
1
3
2
2

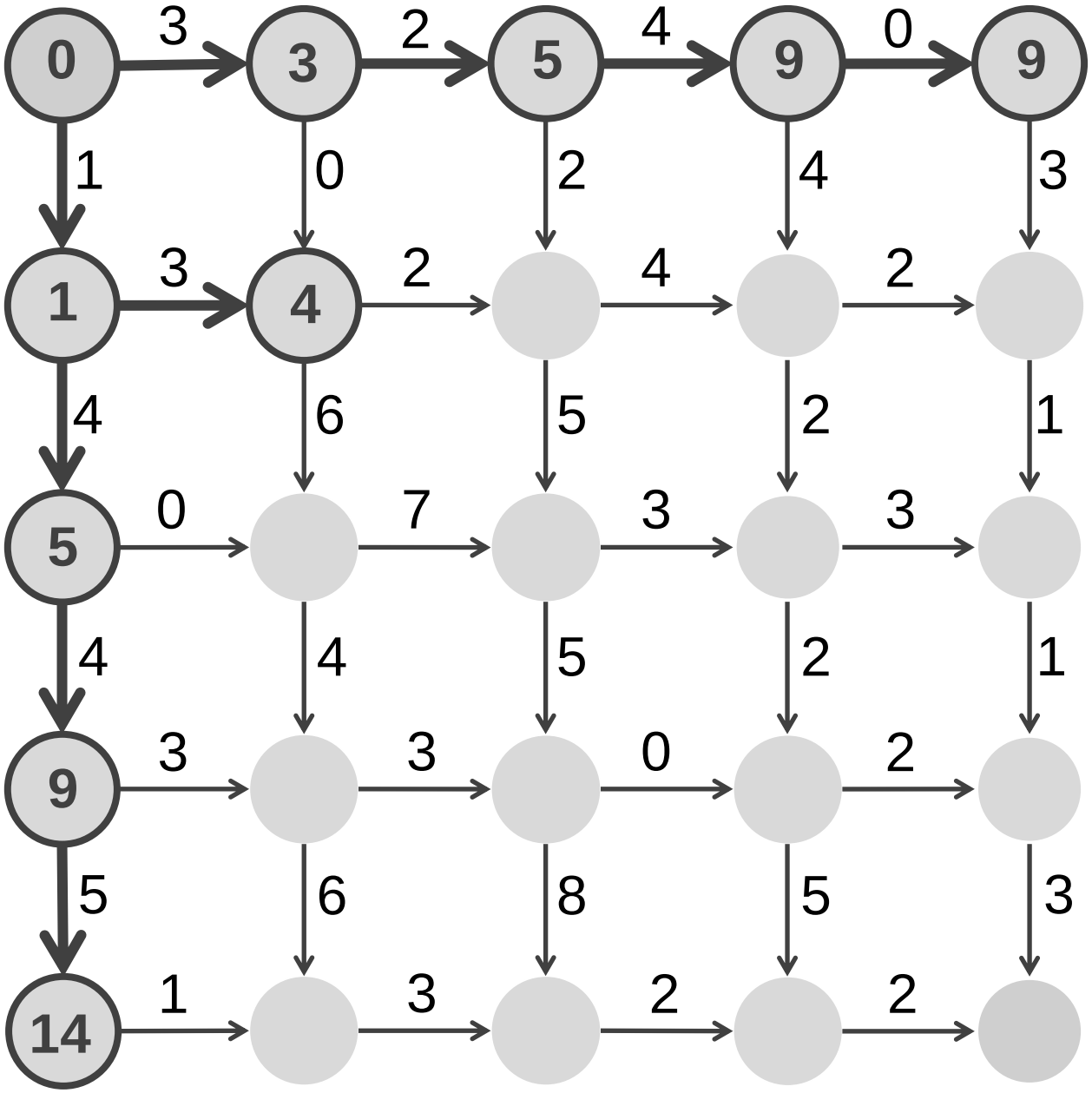
South
or
East?

1+3 > 3+0



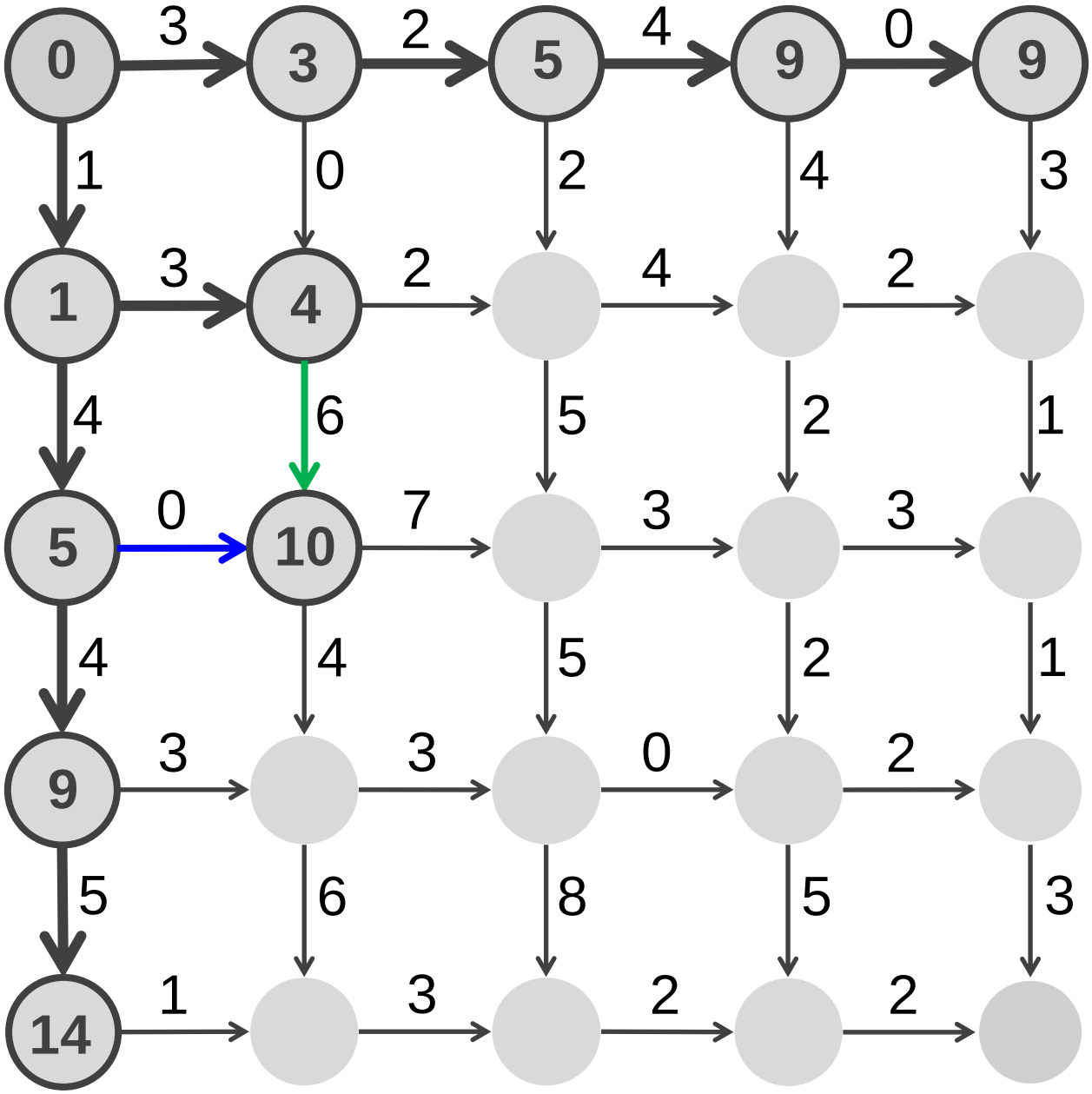
We arrived to (1,1) by the **bold** edge:



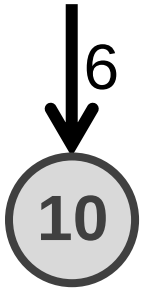


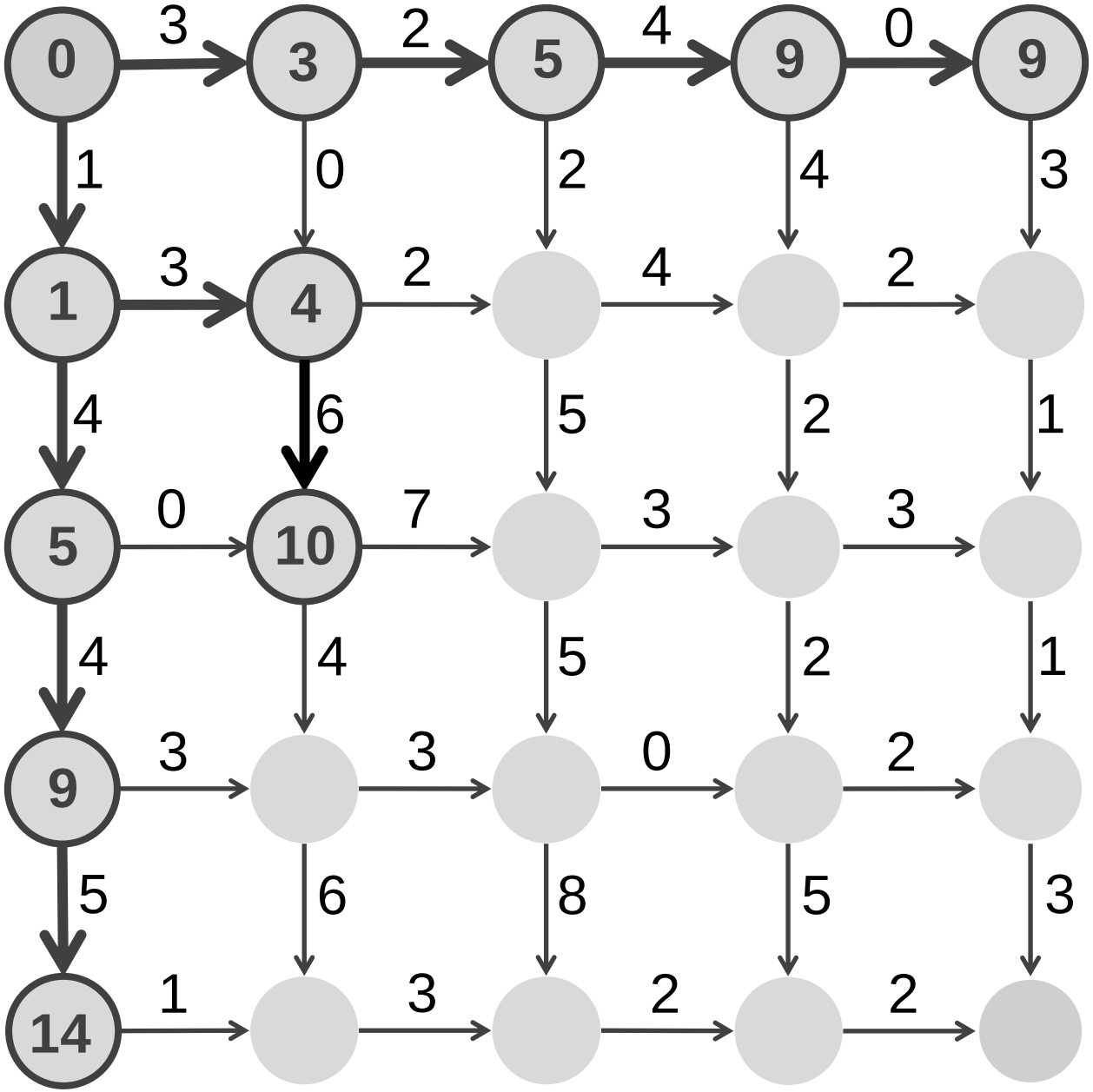
South
or
East?

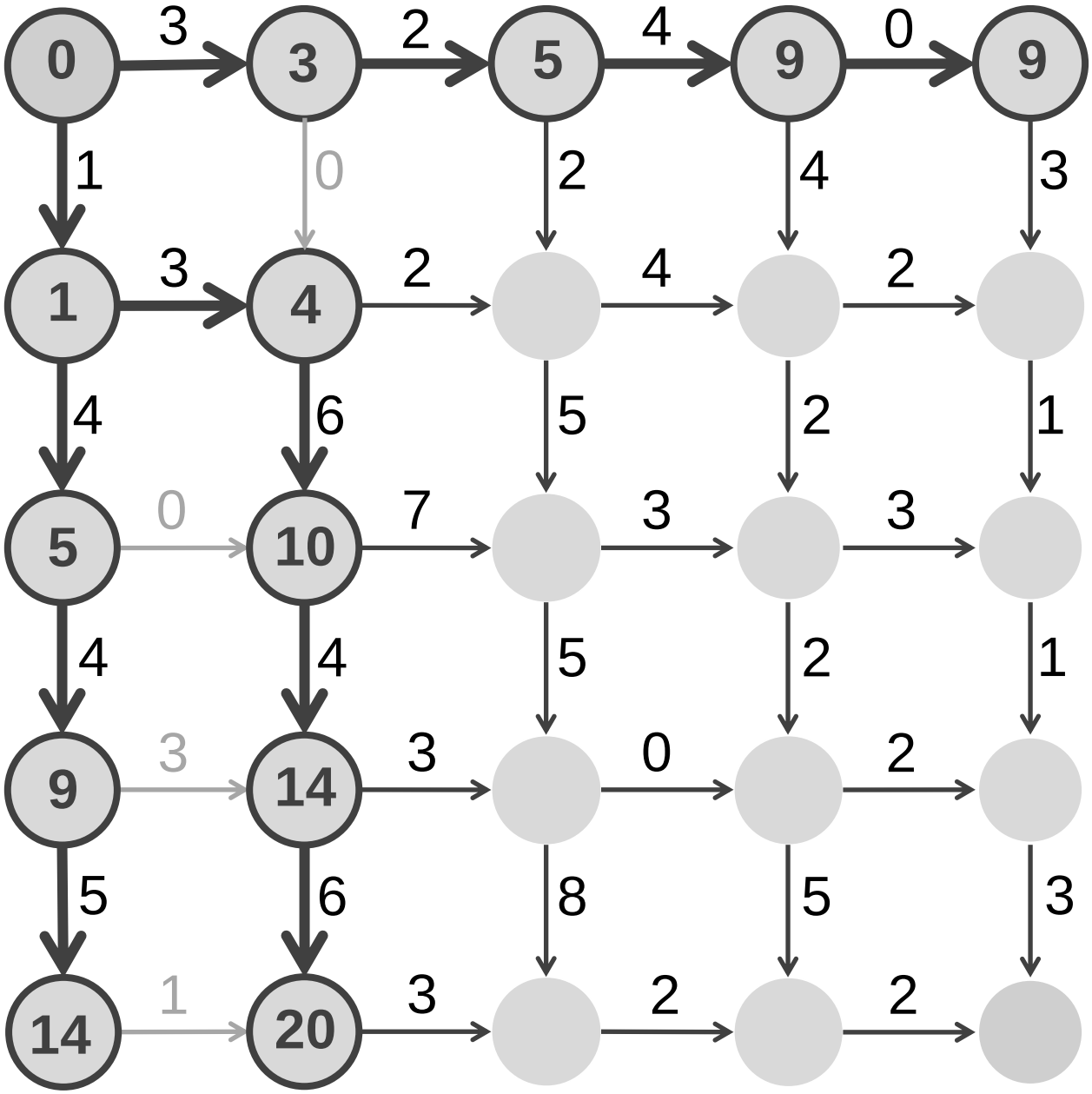
5+0 < 4+6



We arrived to (2,1) by the **bold** edge:

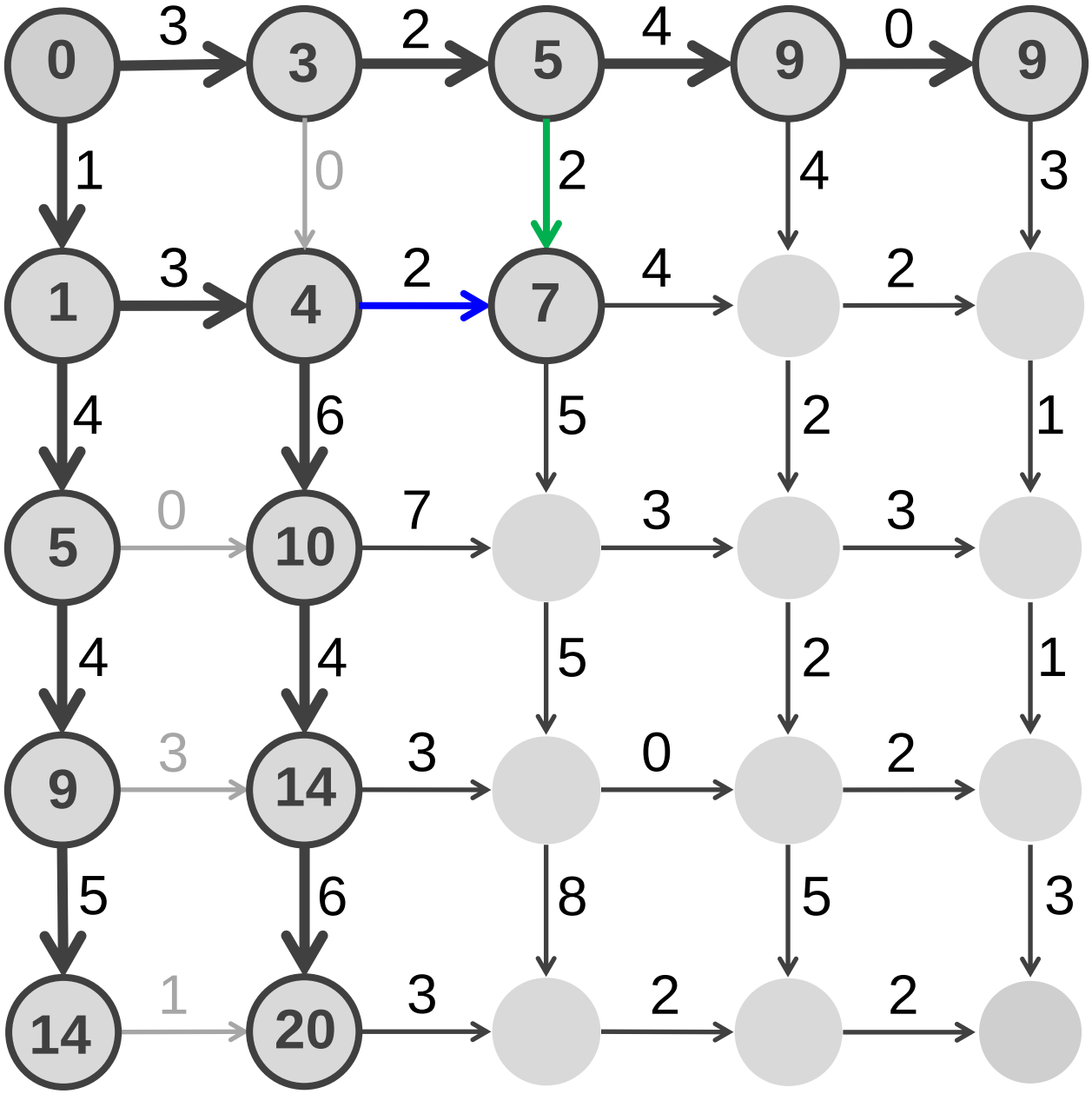





0
3
5
9
9
1
4
5
10
9
14
14
20
3
2
4
0
1
3
2
4
2
4
6
5
2
1
0
7
3
3
4
4
5
2
1
3
3
0
2
5
6
8
5
3
1
3
2
2

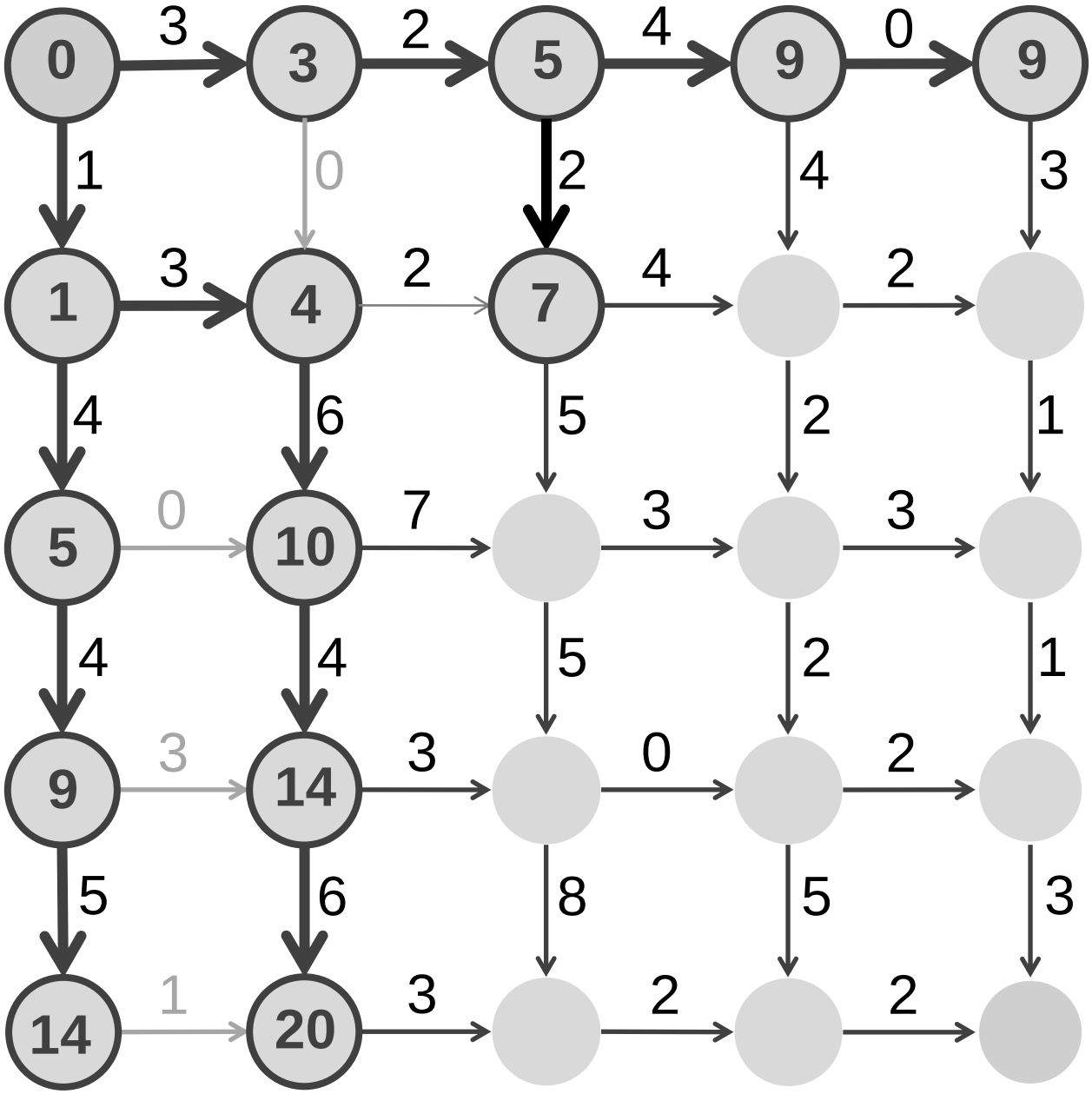
South
or
East?

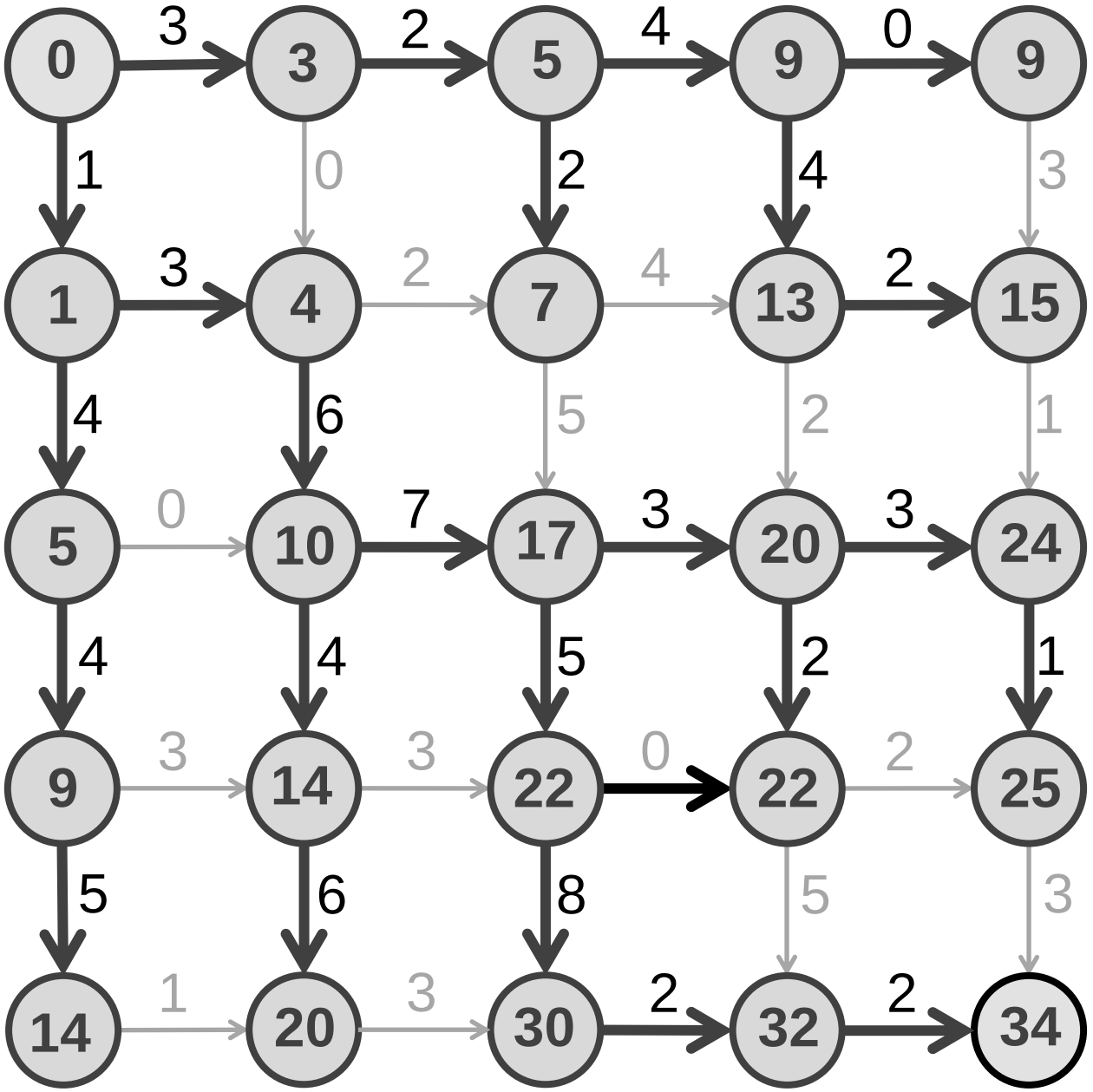
5+2 > 4+2



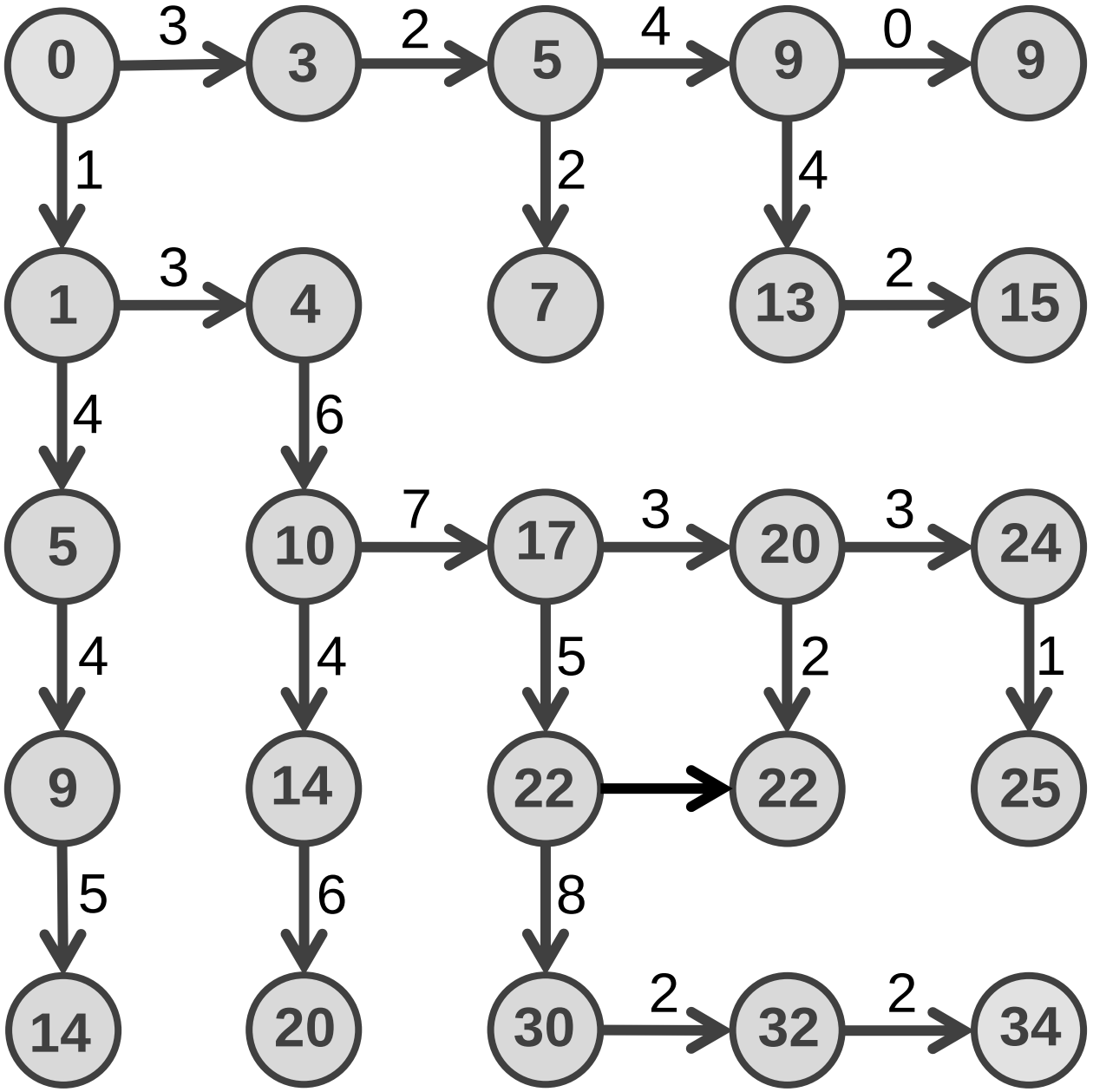
South
or
East?

5+2 > 4+2




0
3
5
9
9
1
3
2
4
0
2
4
3
1
4
7
13
15
3
2
4
2
4
6
5
2
1
5
10
17
20
24
0
7
3
3
4
4
5
2
1
9
14
22
22
25
3
3
0
2
5
6
8
5
3
14
20
30
32
34
1
3
2
2

Backtracking pointers: the best way to get to each node



Dynamic Programming Recurrence

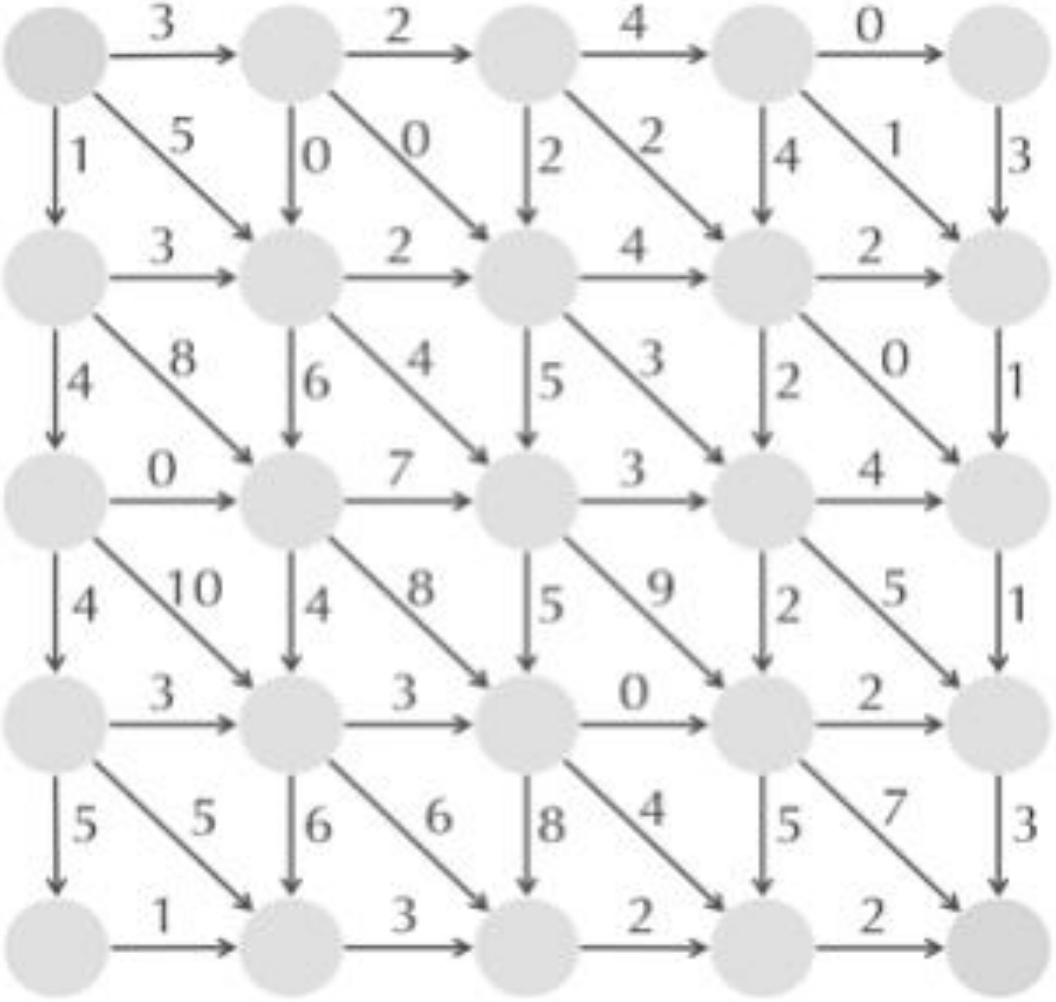
$s_{i,j}$: the length of a longest path from (0,0) to (i,j)

$$s_{i,j} = \max \begin{cases} s_{i-1,j} + \text{weight of edge “}\downarrow\text{”into } (i,j) \\ s_{i,j-1} + \text{weight of edge “}\rightarrow\text{”into } (i,j) \end{cases}$$

Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Example

- Find the longest path

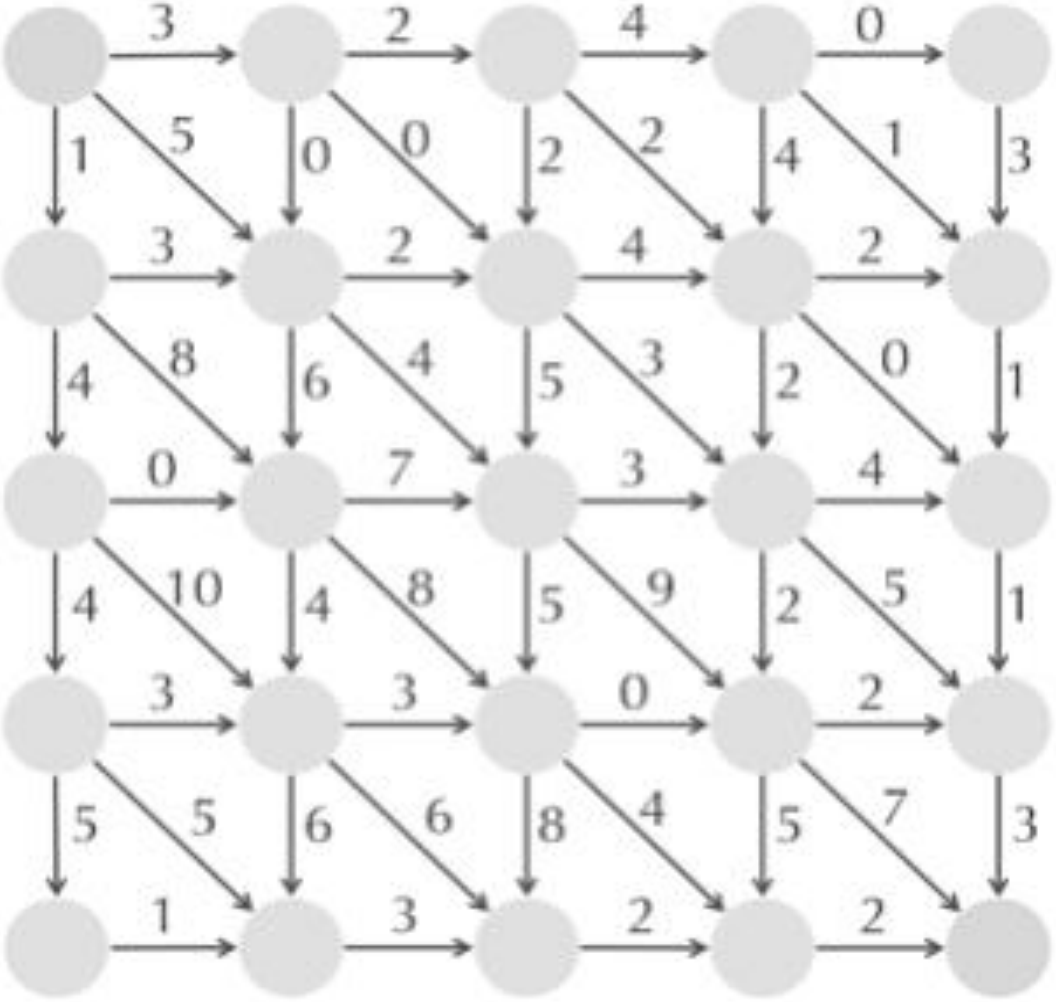


Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

53

Example

- Find the longest path: 35



Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

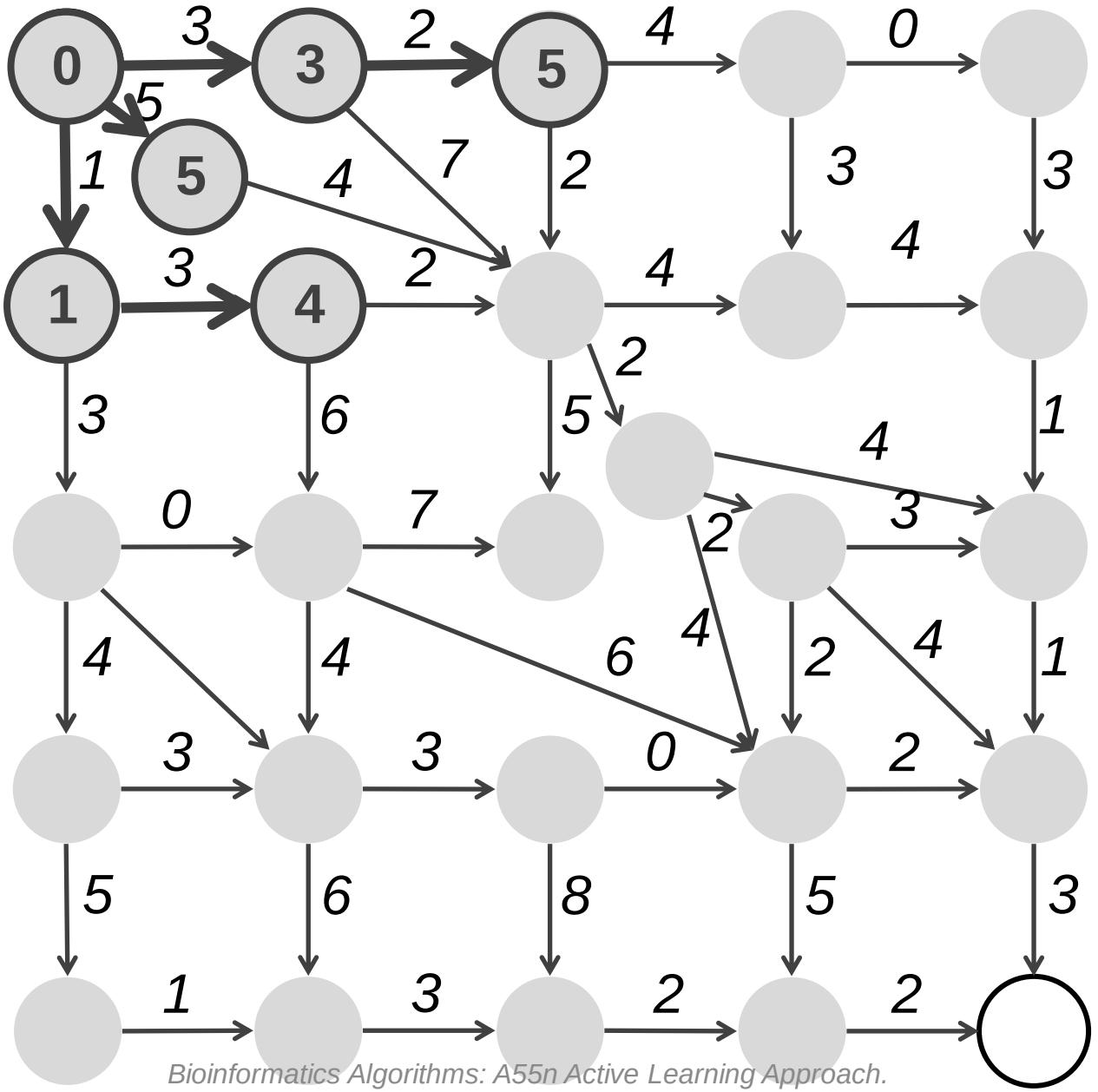
54

How Do We Compare Biological Sequences

- The Alignment Game and the Longest Common Subsequence
- The Manhattan Tourist Problem
- Dynamic Programming and Backtracking Pointers
- **From Manhattan to the Alignment Graph**
- From Global to Local Alignment
- Penalizing Insertions and Deletions in Sequence Alignment
- Space-Efficient Sequence Alignment
- Multiple Sequence Alignment

Copyright 2018 Compeau and Pevzner.

How does the recurrence change for this graph?



Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

$s_a = \max_{\text{all predecessors } b \text{ of node } a}\{s_b + \text{weight of edge from } b \text{ to } a\}$

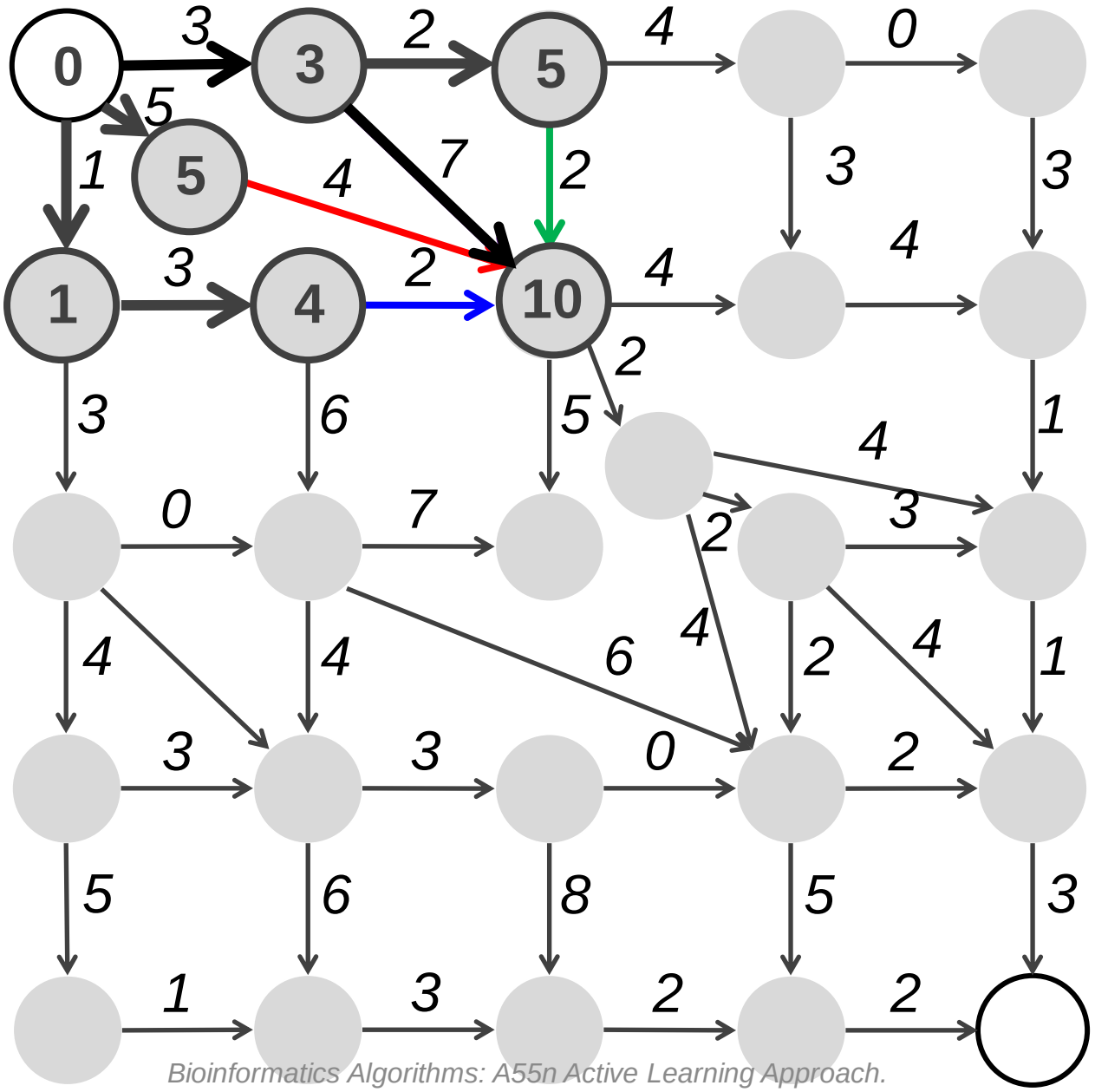
4 choices:

5 + 2

3 + 7

5 + 4

4 + 2



Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

$s_a = \max_{\text{all predecessors } b \text{ of node } a}\{s_b + \text{weight of edge from } b \text{ to } a\}$

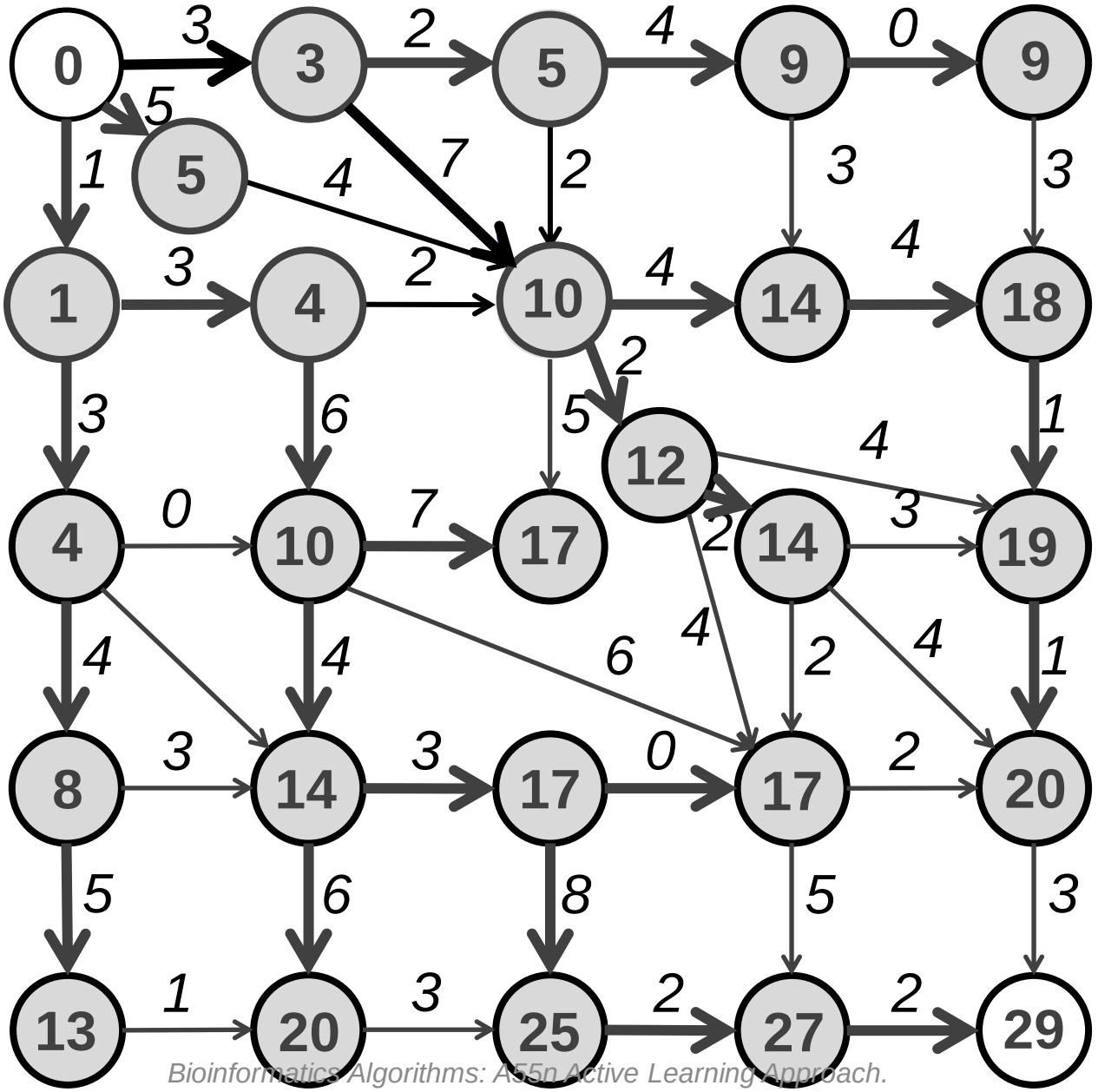
4 choices:

5 + 2

3 + 7

5 + 4

4 + 2

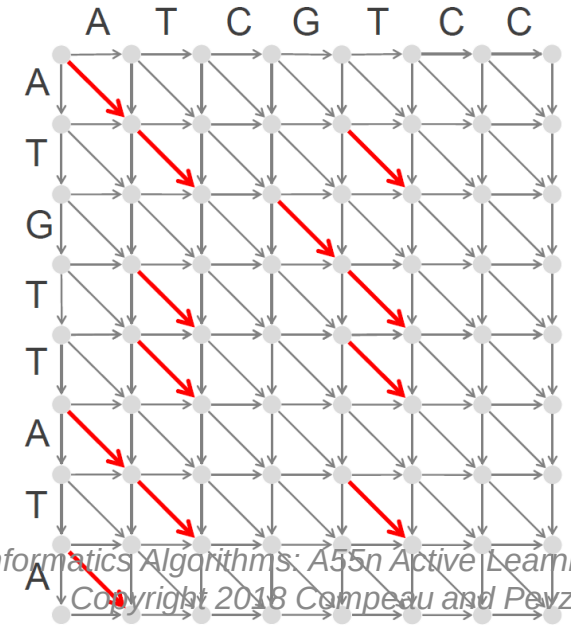


Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Dynamic Programming Recurrence for the Alignment Graph

$s_{i,j}$: the length of a longest path from (0,0) to (i,j)

$$s_{i,j} = \max \begin{cases} s_{i-1,j} + \text{weight of edge "↓" into } (i,j) \\ s_{i,j-1} + \text{weight of edge "→" into } (i,j) \\ s_{i-1,j-1} + \text{weight of edge "↘" into } (i,j) \end{cases}$$



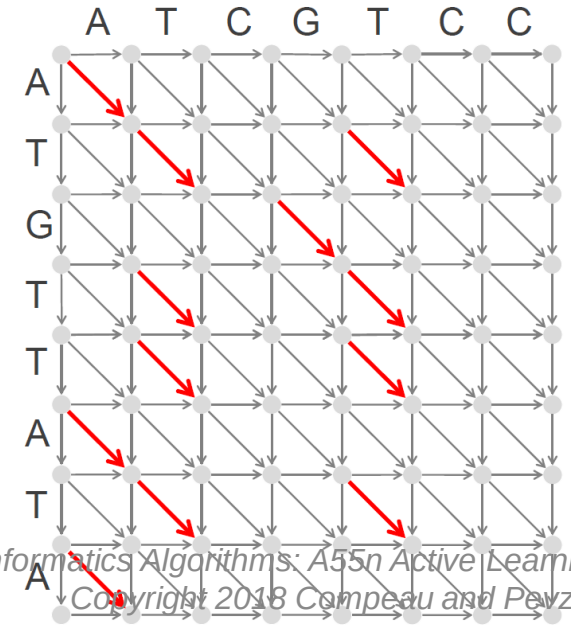
red edges ↘ – weight 1
other edges – weight 0

Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Dynamic Programming Recurrence for the Longest Common Subsequence Problem

$s_{i,j}$: the length of a longest path from (0,0) to (i,j)

$$s_{i,j} = \max \begin{cases} s_{i-1,j} + 0 \\ s_{i,j-1} + 0 \\ s_{i-1,j-1} + 1, \text{ if } v_i = w_j \end{cases}$$

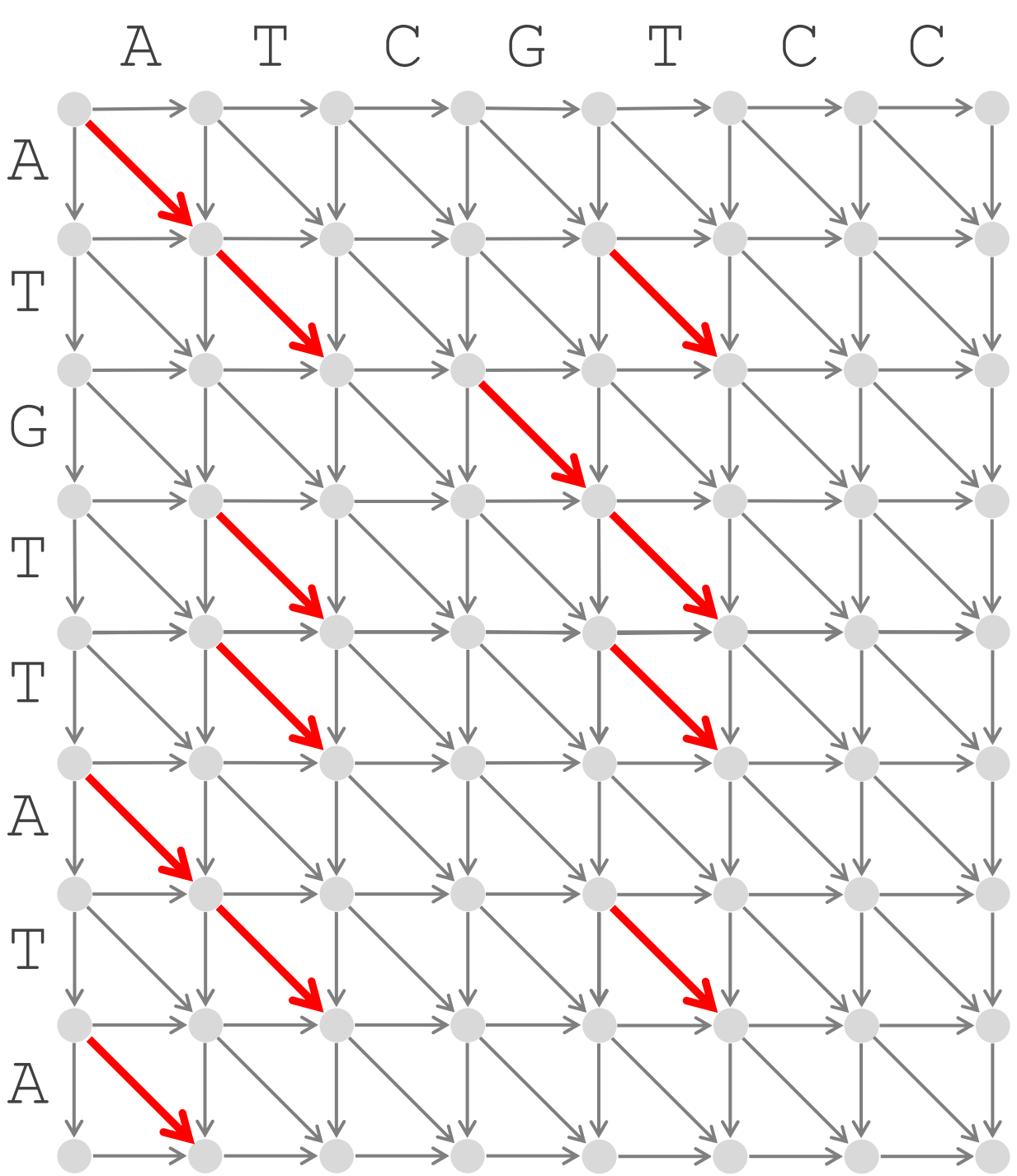


red edges ↘ – weight 1
other edges – weight 0

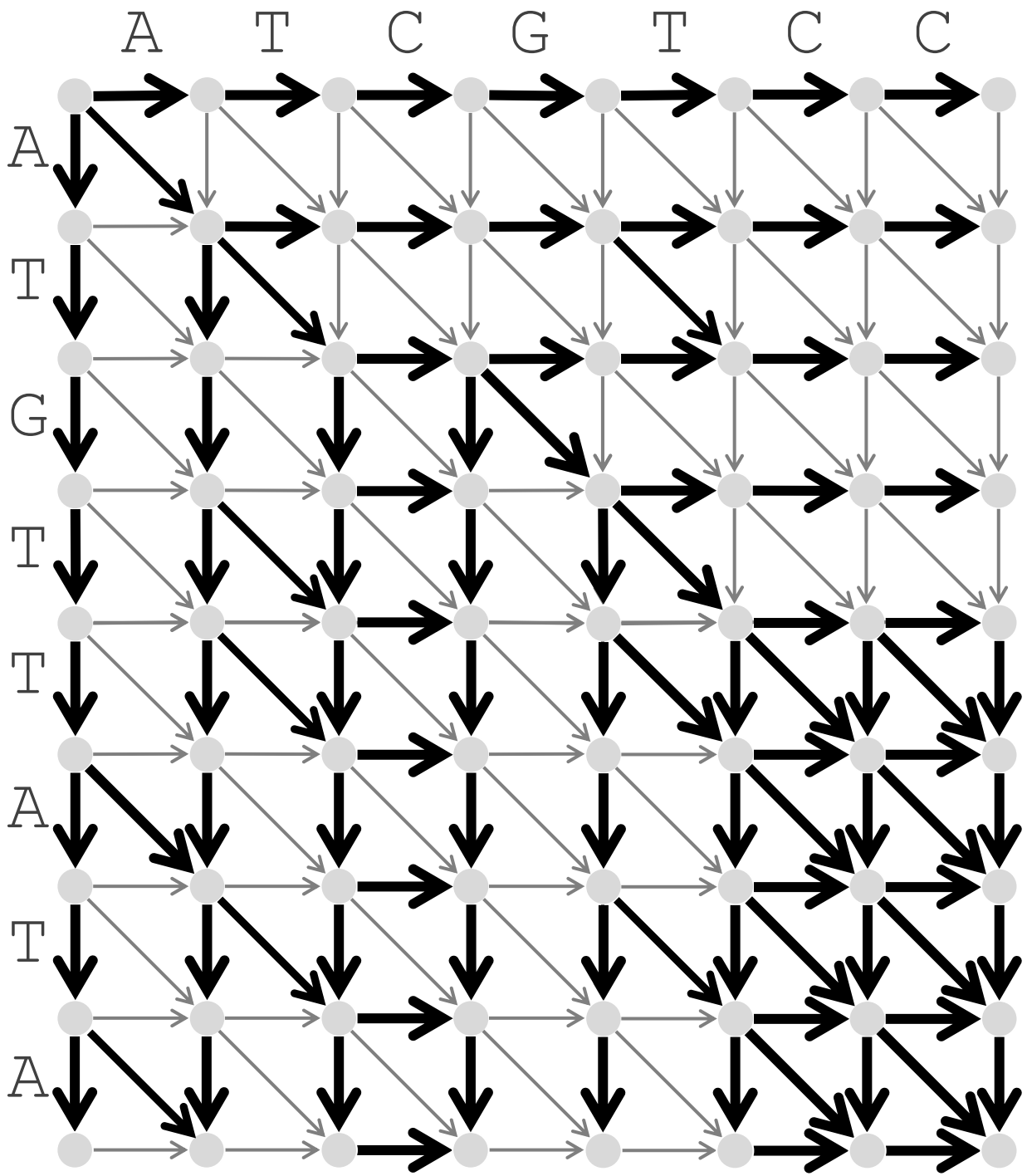
Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

backtracking pointers
for the Longest
Common Subsequence

red edges ↘ – weight 1
other edges – weight 0



backtracking pointers
for the Longest
Common Subsequence



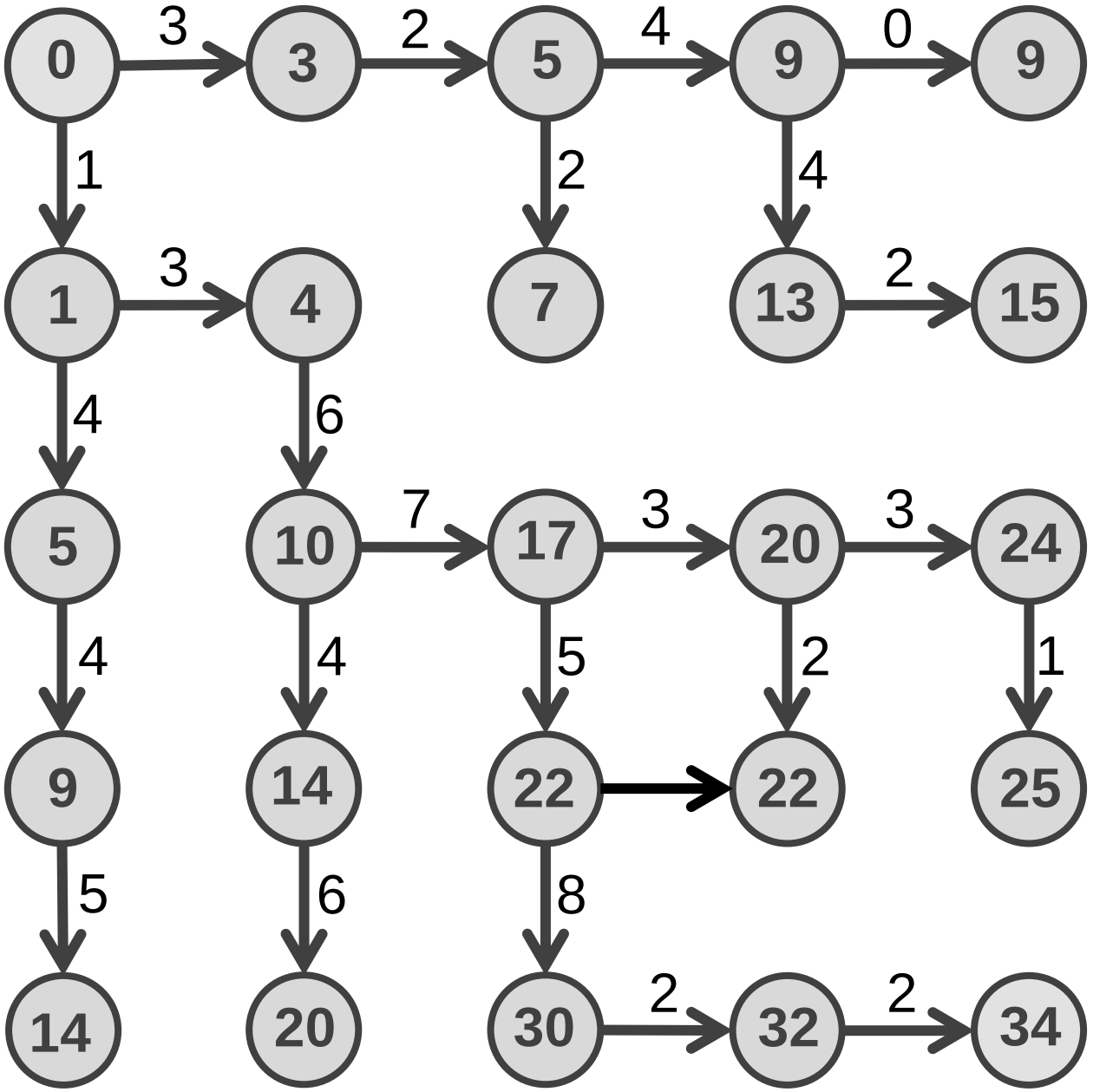
Computing Backtracking Pointers

$$s_{i,j} \leftarrow \max\{ \begin{matrix} s_{i,j-1}+0 \\ s_{i-1,j}+0 \\ s_{i-1,j-1}+1, \text{ if } v_i=w_j \end{matrix}$$

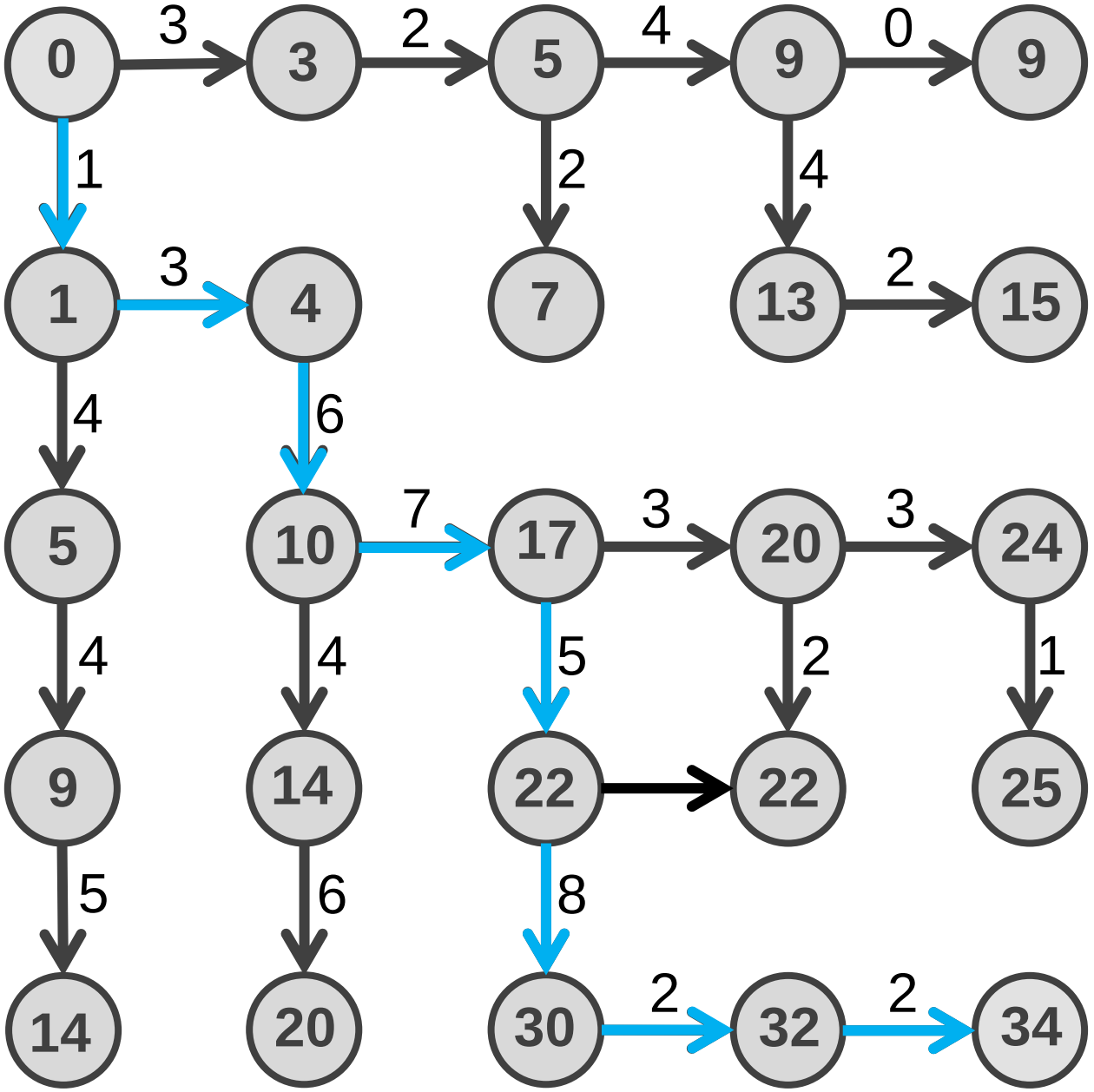
$$backtrack_{i,j} \leftarrow \{ \begin{matrix} \text{“}\rightarrow\text{”, if } s_{i,j}=s_{i,j-1} \\ \text{“}\downarrow\text{", if } s_{i,j}=s_{i-1,j} \\ \text{“}\searrow\text{”, if } s_{i,j}=s_{i-1,j-1}+1 \end{matrix}$$

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

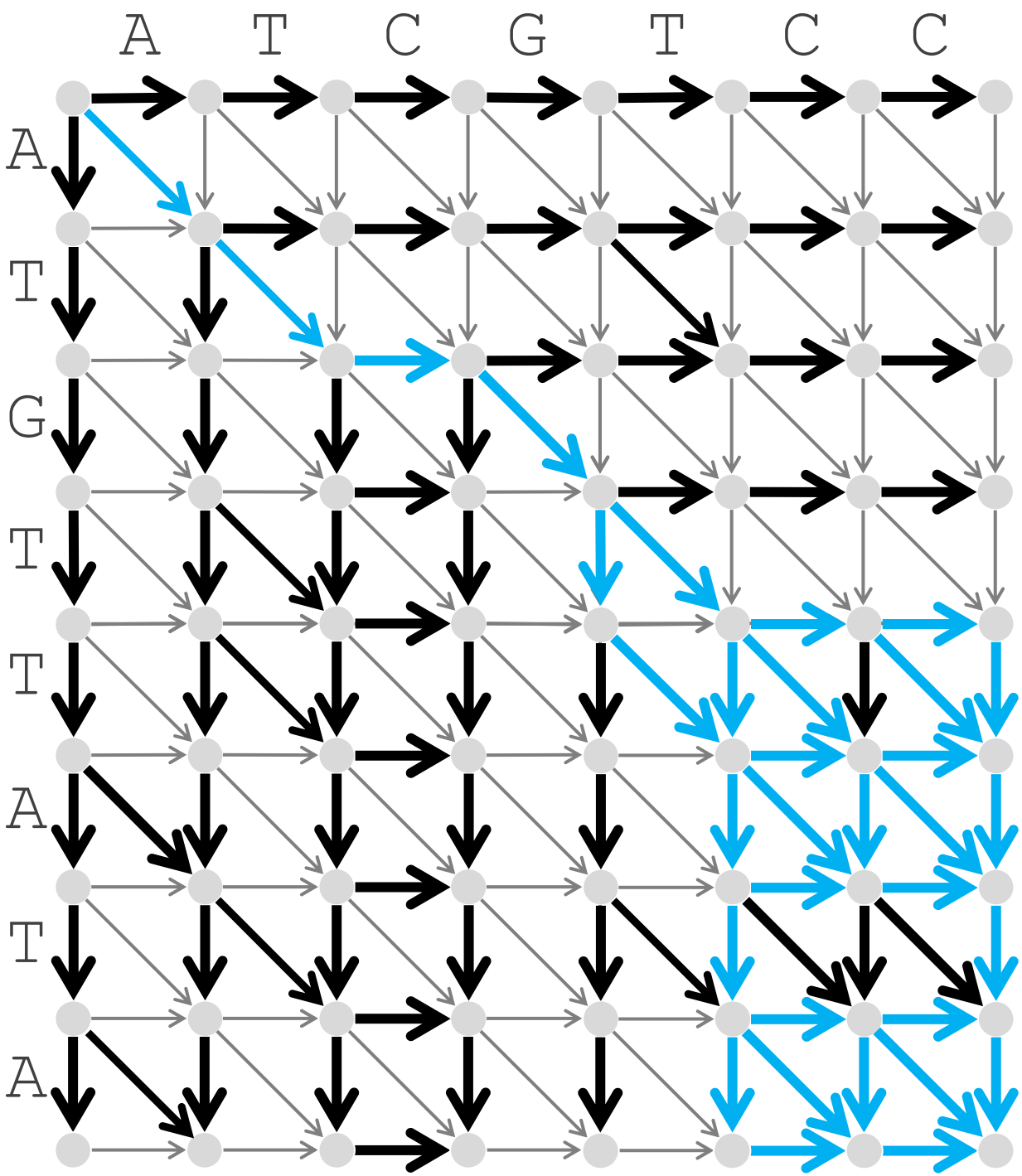
Why did we store the **backtracking pointers?**



What is the optimal alignment path?



backtracking pointers
for the Longest
Common Subsequence

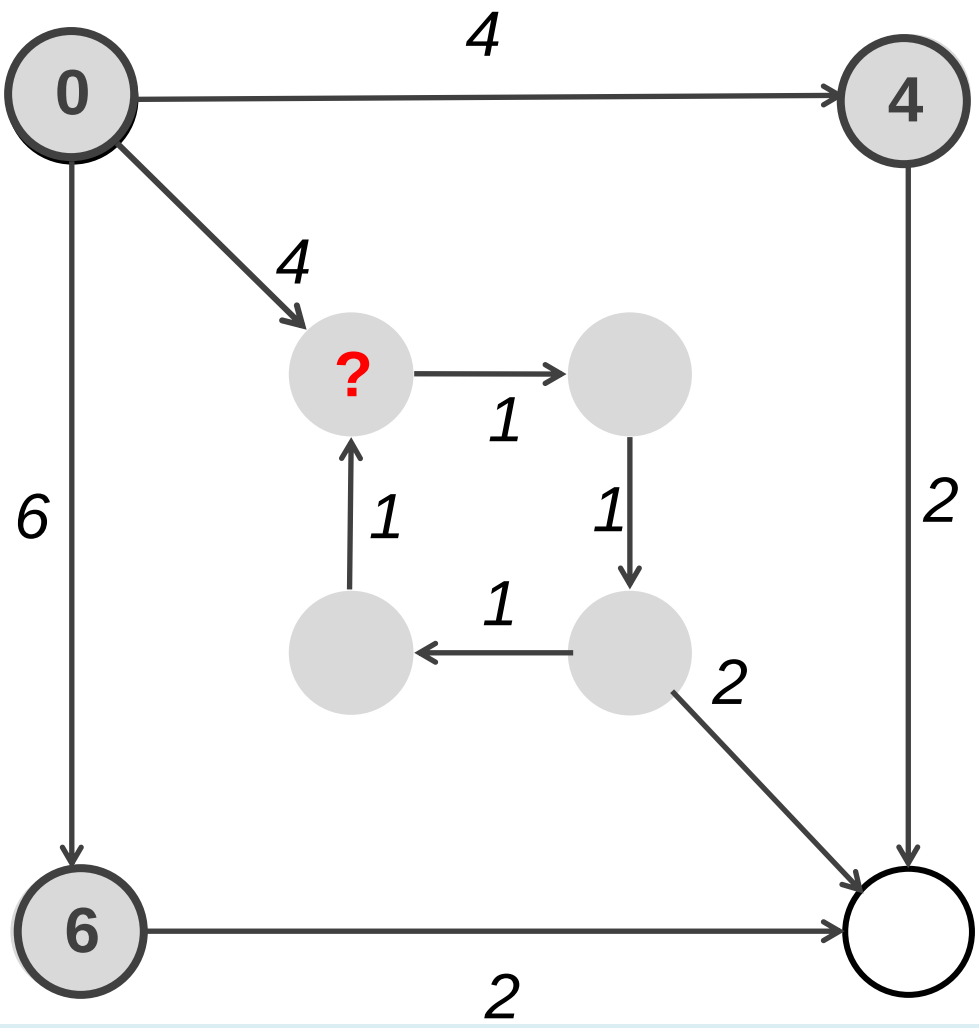


Using Backtracking Pointers to Compute LCS

```
OutputLCS (backtrack, v, i, j)
  if i = 0 or j = 0
    return
  if backtrack_{i,j} = "→"
      OutputLCS (backtrack, v, i, j-1)
    else if backtrack_{i,j} = "↓"
      OutputLCS (backtrack, v, i-1, j)
    else
      OutputLCS (backtrack, v, i-1, j-1)
      output v_i
```

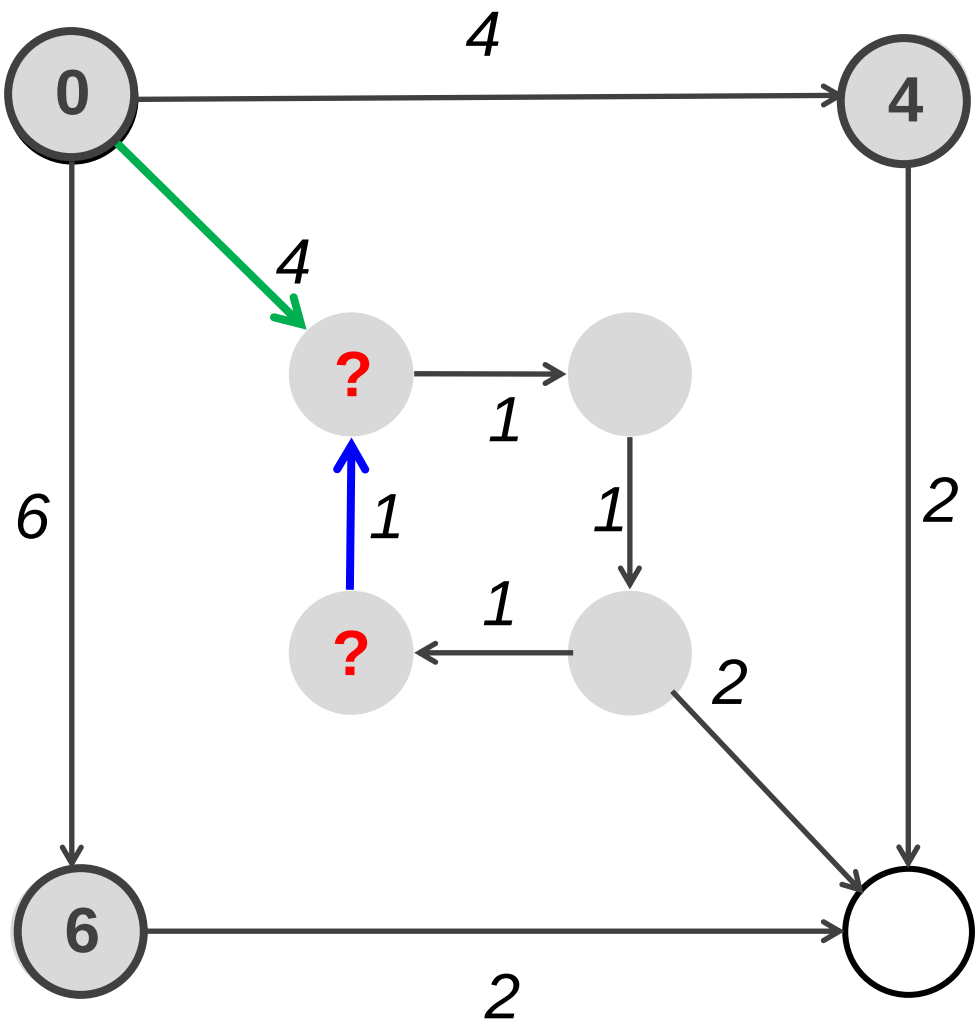
Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Computing Scores of ALL Predecessors

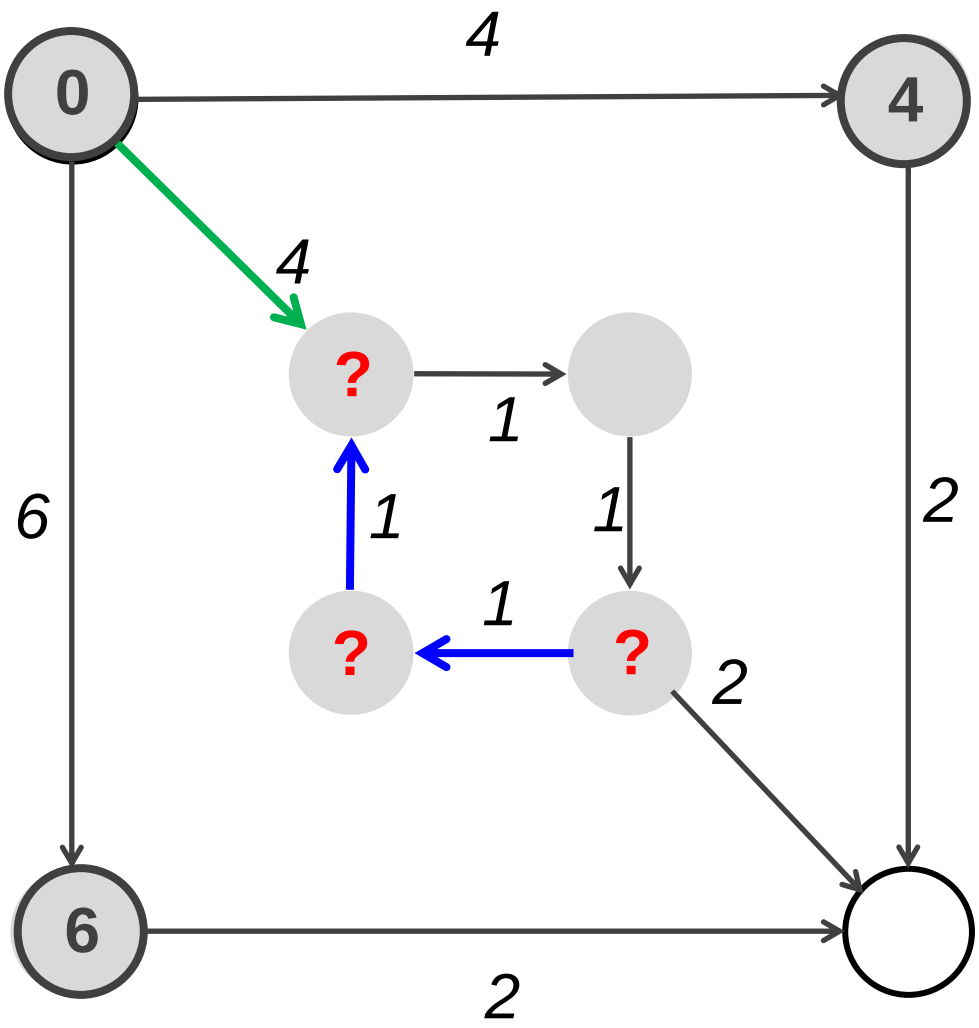


$s_a = \max_{\textbf{ALL}\ \text{predecessors}\ b\ \text{of node}\ a}\{s_b + \text{weight of edge from } b \text{ to } a\}$

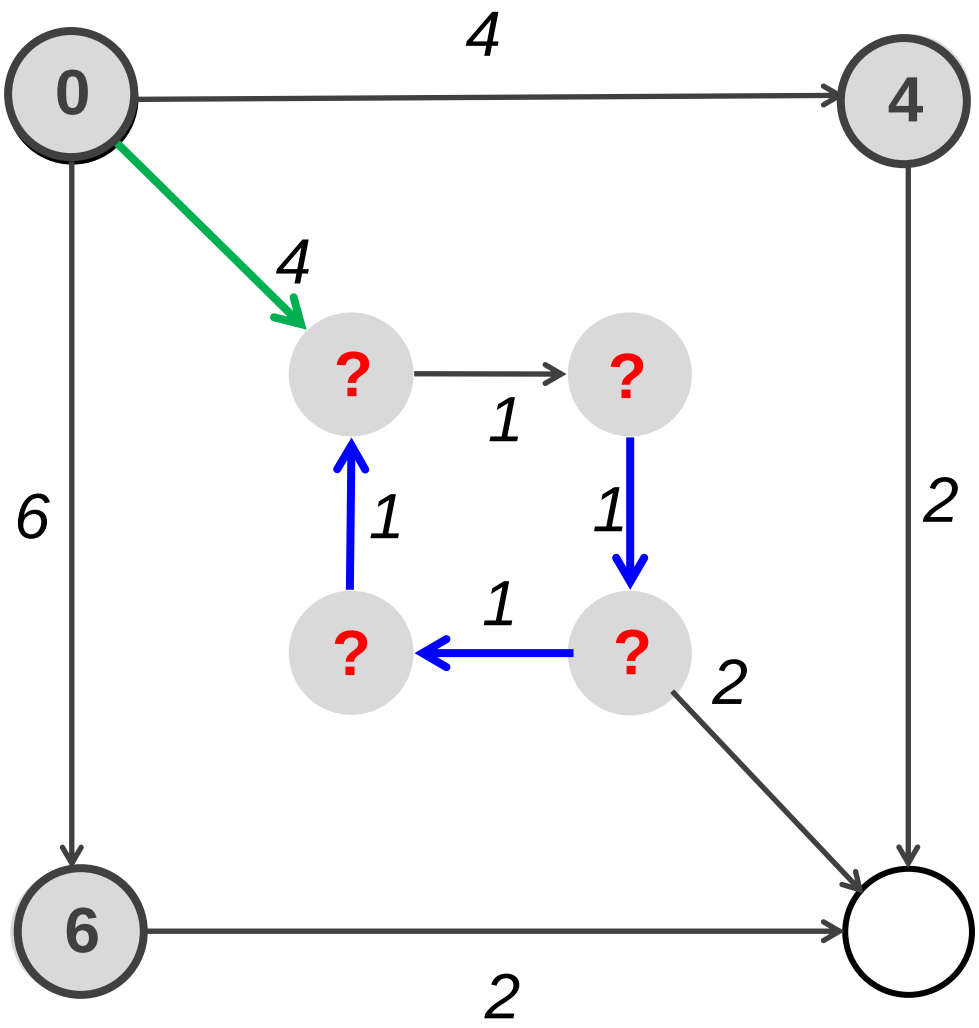
Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.



Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

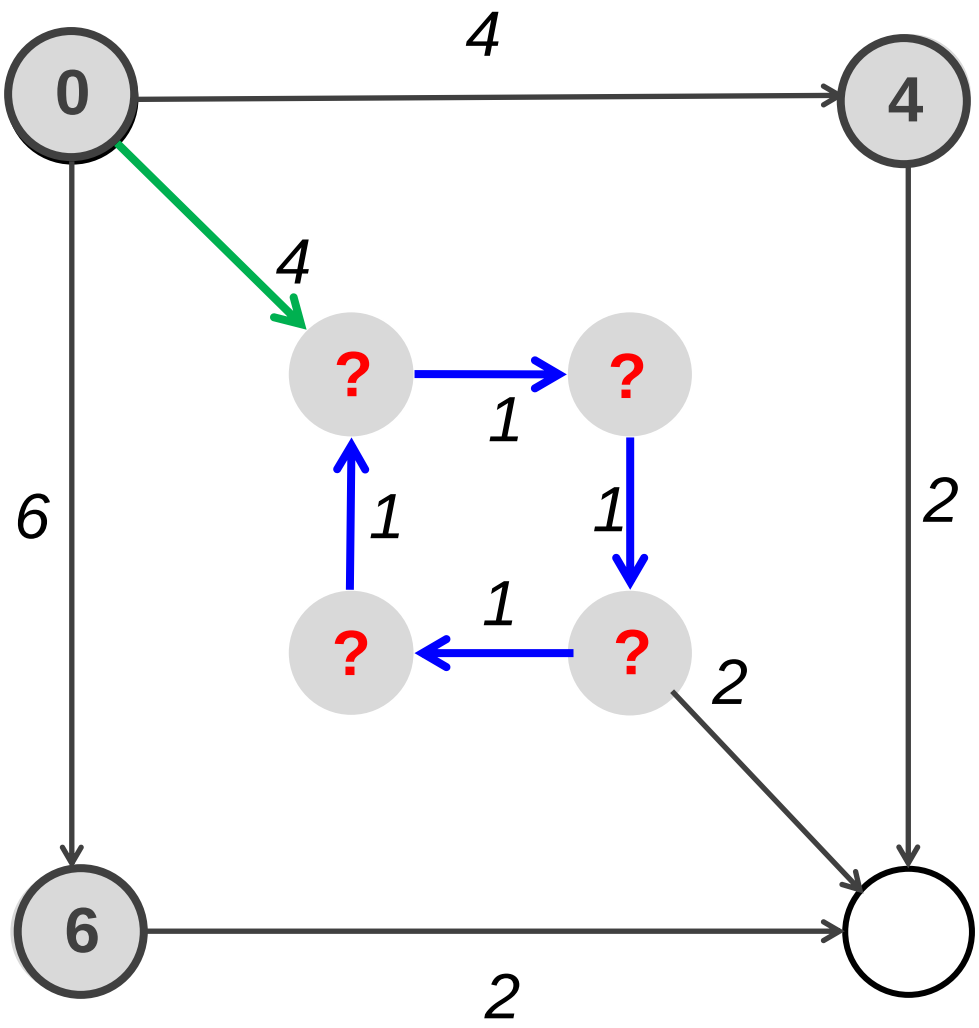


Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.



Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

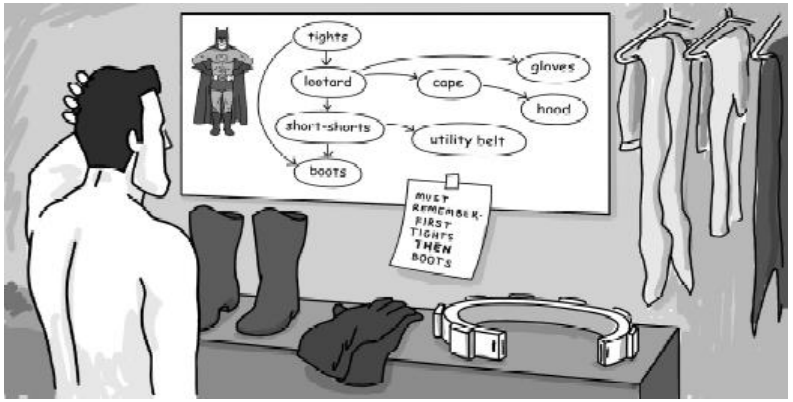
A Vicious Cycle



Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

In What Order Should We Explore Nodes of the Graph?

$s_a = \max_{\textbf{ALL}\ \text{predecessors}\ b\ \text{of node}\ a}\{s_b + \text{weight of edge from } b \text{ to } a\}$

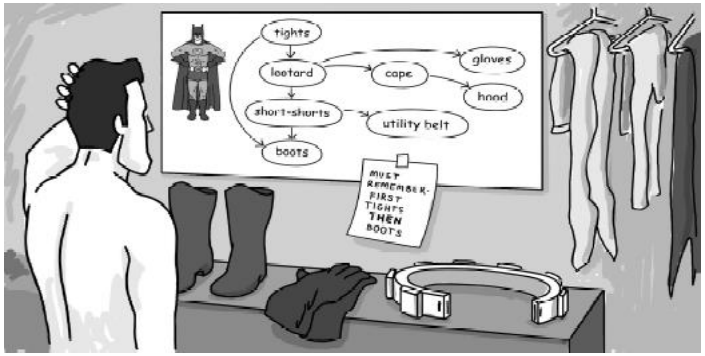


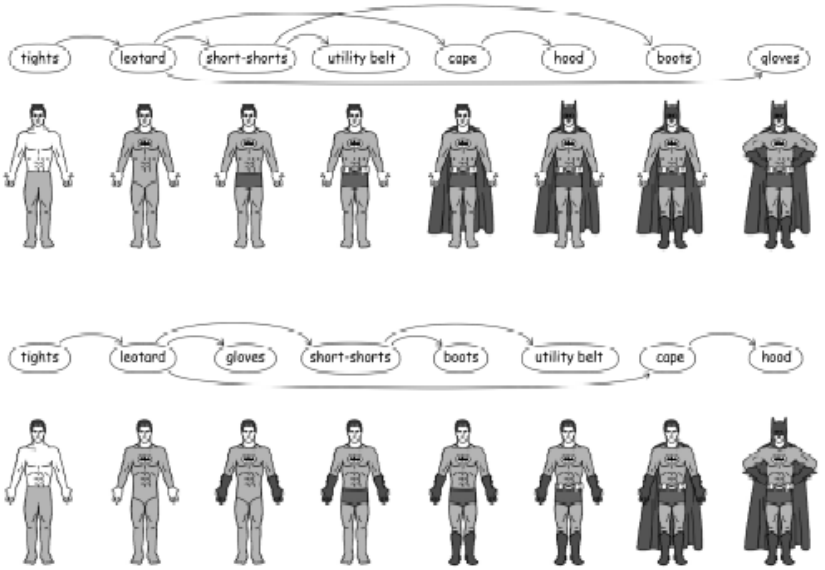
- By the time a node is analyzed, the scores of all its predecessors should already be computed.
- If the graph has a **directed cycle**, this condition is impossible to satisfy.
- **Directed Acyclic Graph** (**DAG**): a graph without directed cycles.

Copyright 2018 Compeau and Pevzner.

Topological Ordering

- **Topological Ordering**: Ordering of nodes of a DAG on a line such that all edges go from left to right.

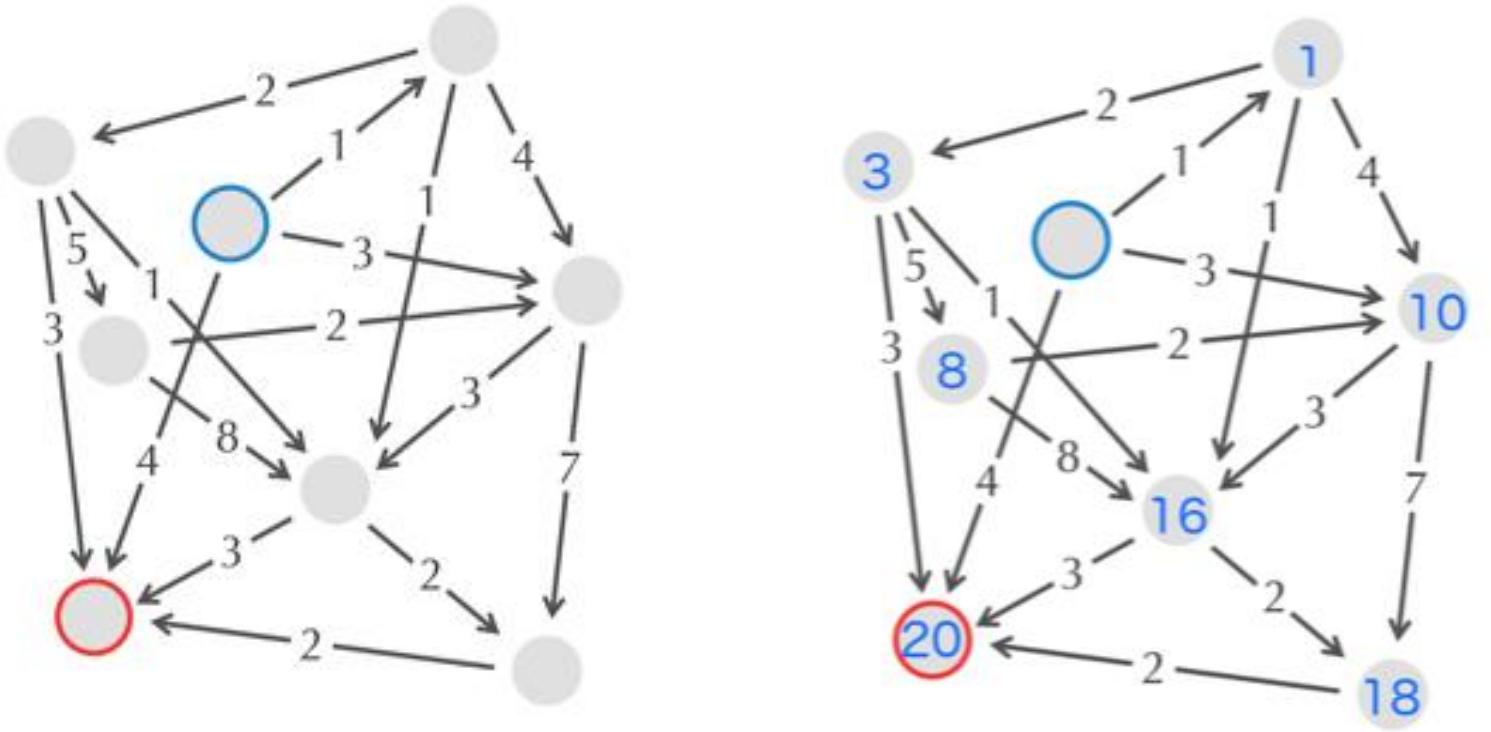




- **Theorem**: Every DAG has a topological ordering.

Copyright 2018 Compeau and Pevzner.

Q: What is the longest path from blue to red node?



LongestPath

LongestPath(*Graph, source, sink*)
  for each node *a* in *Graph*
    $s_a \leftarrow$ -infinity
  $s_{source} \leftarrow 0$
  topologically order *Graph*
  for each node *a* (from *source* to *sink* in topological order)
    $s_a \leftarrow \max_{\text{all predecessors } b \text{ of node } a}\{s_b + \text{weight of edge from } b \text{ to } a\}$
  return s_{sink}

Copyright 2018 Compeau and Pevzner.

How Do We Compare Biological Sequences

- The Alignment Game and the Longest Common Subsequence
- The Manhattan Tourist Problem
- Dynamic Programming and Backtracking Pointers
- From Manhattan to the Alignment Graph
- **From Global to Local Alignment**
- Penalizing Insertions and Deletions in Sequence Alignment
- Space-Efficient Sequence Alignment
- Multiple Sequence Alignment

Copyright 2018 Compeau and Pevzner.

Current (Primitive) Scoring

#matches

Copyright 2018 Compeau and Pevzner.

Mismatches and Indel Penalties

#matches – μ · #mismatches – σ · #indels

```
A T - G T T A T A
A T C G T - C - C
+1+1-2+1+1-2-3-2-3=-7
```

	A	C	G	T	–
A	+1	$-\mu$	$-\mu$	$-\mu$	$-\sigma$
C	$-\mu$	+1	$-\mu$	$-\mu$	$-\sigma$
G	$-\mu$	$-\mu$	+1	$-\mu$	$-\sigma$
T	$-\mu$	$-\mu$	$-\mu$	+1	$-\sigma$
–	$-\sigma$	$-\sigma$	$-\sigma$	$-\sigma$	

Scoring matrix

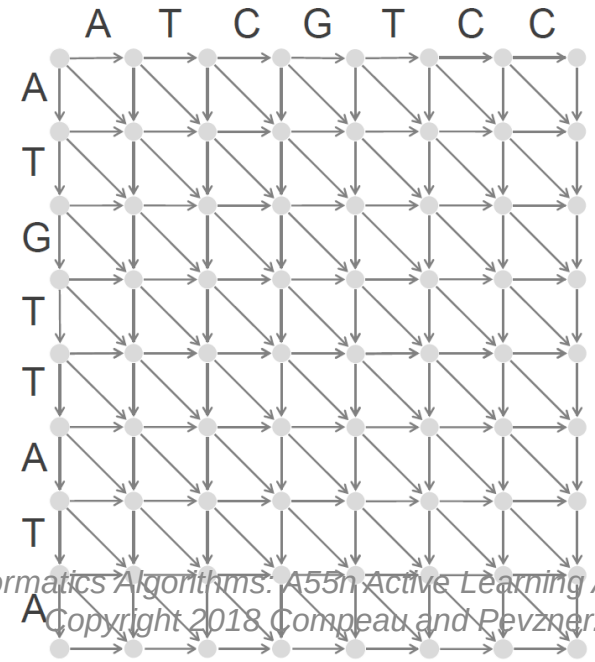
	A	C	G	T	–
A	+1	–3	–5	–1	–3
C	–4	+1	–3	–2	–3
G	–9	–7	+1	–1	–3
T	–3	–5	–8	+1	–4
–	–4	–2	–2	–1	

Even more general scoring matrix

Copyright 2018 Compeau and Pevzner.

Dynamic Programming Recurrence for the Alignment Graph

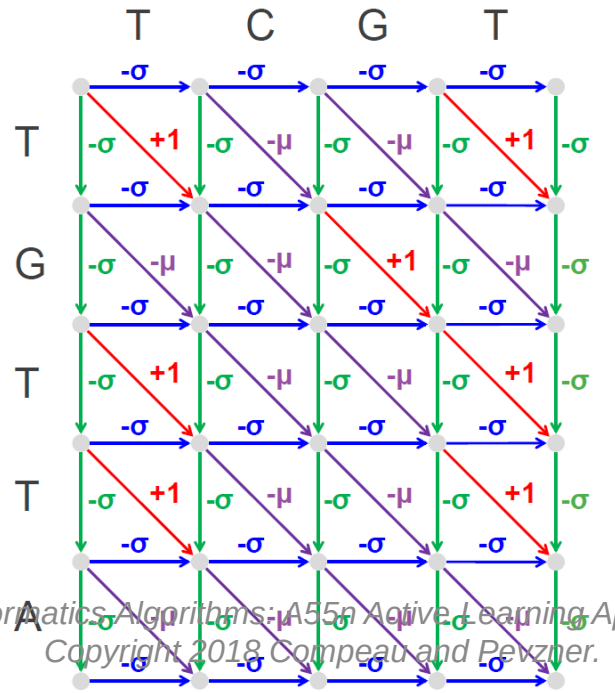
$$s_{i,j} = \max \begin{cases} s_{i-1,j} + \text{weight of edge “}\downarrow\text{” into } (i,j) \\ s_{i,j-1} + \text{weight of edge “}\rightarrow\text{” into } (i,j) \\ s_{i-1,j-1} + \text{weight of edge “}\searrow\text{” into } (i,j) \end{cases}$$



Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Dynamic Programming Recurrence for the Alignment Graph

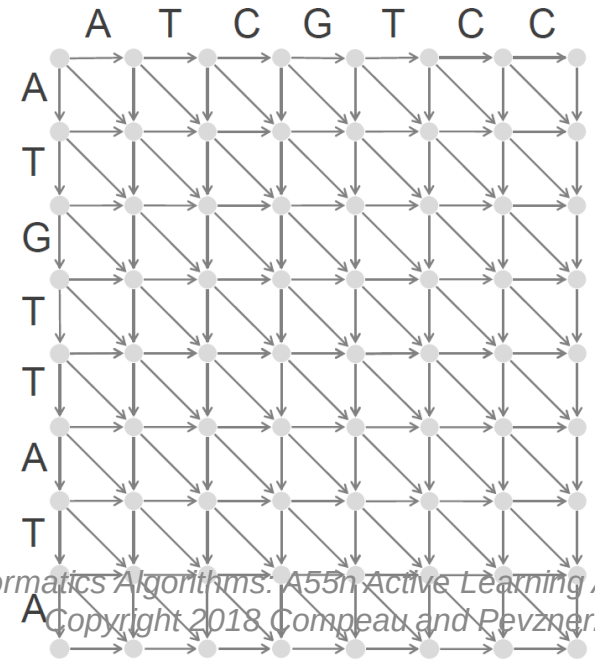
$$s_{i,j} = \max \begin{cases} s_{i-1,j} - \sigma \\ s_{i,j-1} - \sigma \\ s_{i-1,j-1} + 1, \text{ if } v_i = w_j \\ s_{i-1,j-1} - \mu, \text{ if } v_i \neq w_j \end{cases}$$



Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Dynamic Programming Recurrence for the Alignment Graph

$$s_{i,j} = \max \begin{cases} s_{i-1,j} + score(v_i, -) \\ s_{i,j-1} + score(-, w_j) \\ s_{i-1,j-1} + score(v_i, w_j) \end{cases}$$



Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Global Alignment

Global Alignment Problem: Find the highest-scoring alignment between two strings by using a scoring matrix.

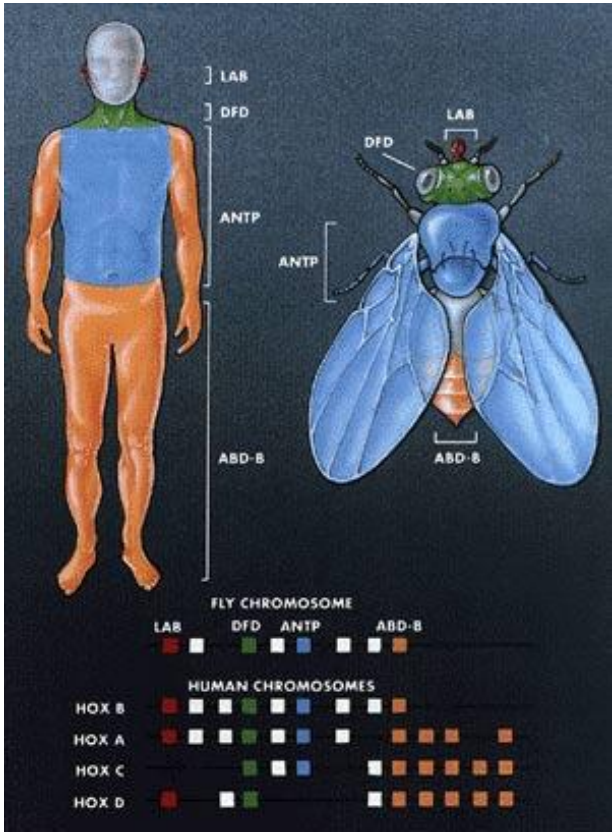
- **Input:** Strings *v* and *w* as well as a matrix ***score***.
- **Output:** An alignment of *v* and *w* whose alignment score (as defined by the scoring matrix ***score***) is maximal among all possible alignments of *v* and *w*.

Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Homeobox Genes

- Two genes in different species may be similar over short conserved regions and dissimilar over remaining regions.

- Homeobox genes have a short region called the **homeodomain** that is highly conserved among species.
  - A global alignment may not find the homeodomain because it would try to align the *entire* sequence.



Which Alignment is Better?

- Alignment 1: score = 22 (matches) - 20 (indels)=2.

```
GCC-C-AGT--TATGT-CAGGGGGCACG--A-GCATGCAGA-
GCCGCC-GTCGT-T-TTCAG----CA-GTTATG--T-CAGAT
```

- Alignment 2: score = 17 (matches) - 30 (indels)=-13.

```
---G----C-----C--CAGTTATGTCAGGGGGCACGAGCATGCAGA
GCCGCCGTCGTTTTCAGCAGTTATGTCAG-----A------T-----
```


Copyright 2018 Compeau and Pevzner.

Which Alignment is Better?

- Alignment 1: score = 22 (matches) - 20 (indels)=2.

```
GCC-C-AGT--TATGT-CAGGGGGCACG--A-GCATGCAGA-
GCCGCC-GTCGT-T-TTCAG----CA-GTTATG--T-CAGAT
```

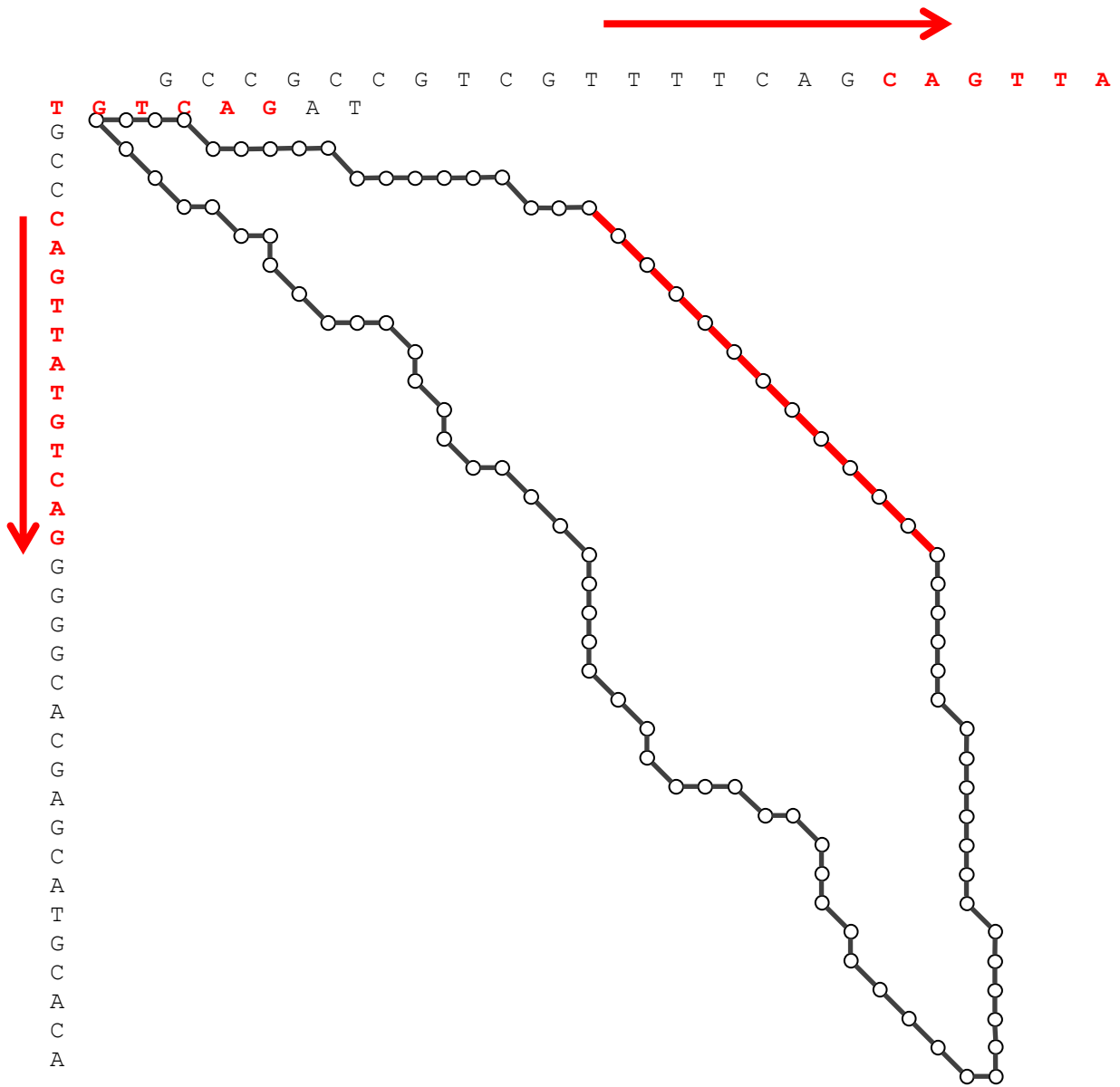
- Alignment 2: score = 17 (matches) - 30 (indels)=-13.

```
---G----C------C--CAGTTATGTCAGGGGGCACGAGCATGCAGA
GCCGCCGTCGTTTTCAGCAGTTATGTCAG-----A------T-----
```

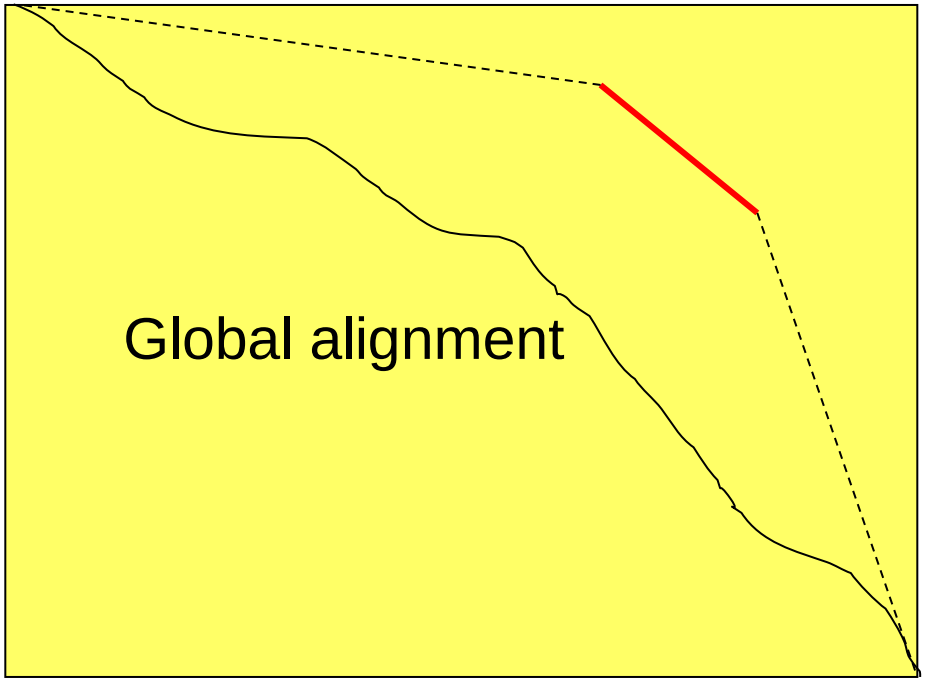
local alignment

Copyright 2018 Compeau and Pevzner.

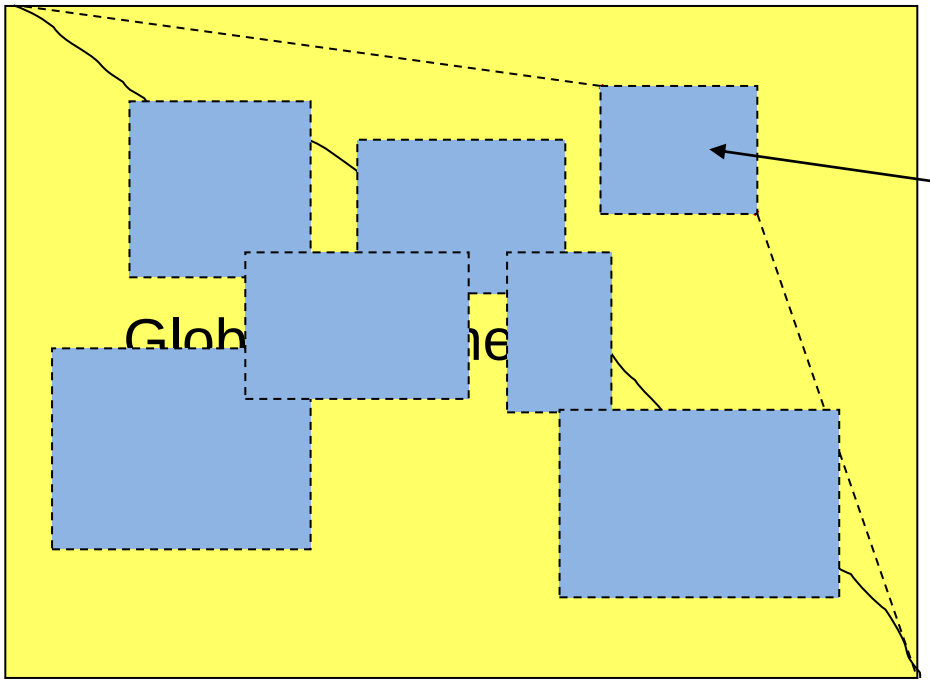

G C C G C C G T C G T T T T C A G C A G T T A
T G T C A G A T
T G C C C A G T T A T G T C A G G G G G G C A C G A G C A T G C A C A
---G----C-----C--CAGTTATGTCAGGGGGCACGAGCATGCAGA
GCCGCCGTCGTTTTCAGCAGTTATGTCAG-----A------T ----
Local alignment
GCC-C-AGT-TATGT-CAGGGGGCACG--A-GCATGCAGA-
GCCGCC-GTCGT-T-TTCAG----CA-GTTATG-T-CAGAT
Global alignment


G C C G C C G T C G T T T T C A G C A G T T A
T G T C A G A T
T
G
C
C
C A G T T A T G T C A G
G
G
G
G
C
A
C
G
A
G
C
A
T
G
C
A
C
A

Local Alignment



Local Alignment= Global Alignment in a Subrectangle



Compute a Global Alignment within each rectangle to get a Local Alignment

Local Alignment Problem

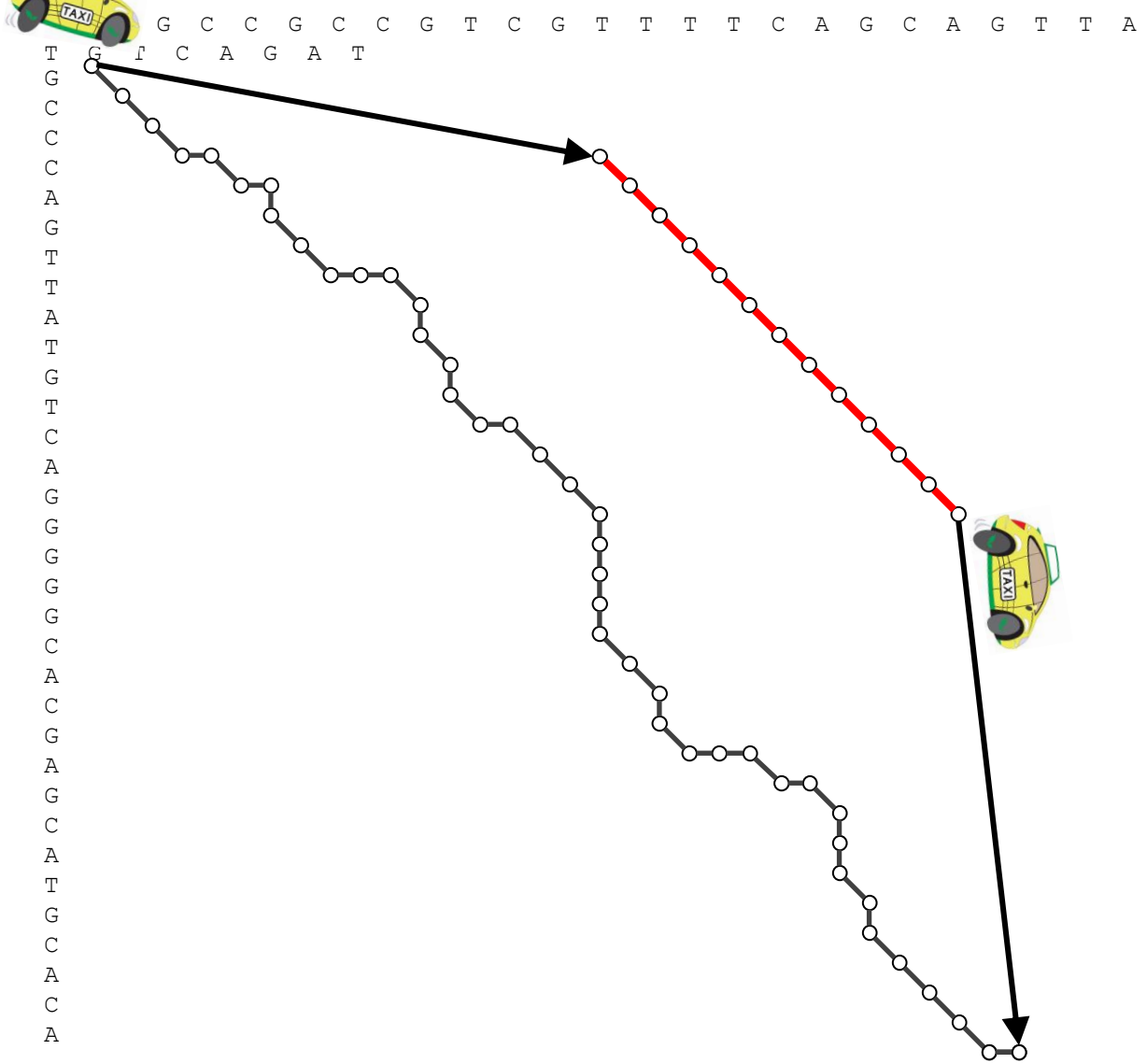
Local Alignment Problem: Find the highest-scoring local alignment between two strings.

- **Input:** Strings *v* and *w* as well as a matrix ***score***.
- **Output:** Substrings of *v* and *w* whose global alignment (as defined by the matrix ***score***), is maximal among all global alignments of all substrings of *v* and *w*.

Copyright 2018 Compeau and Pevzner.

Free Taxi Rides!





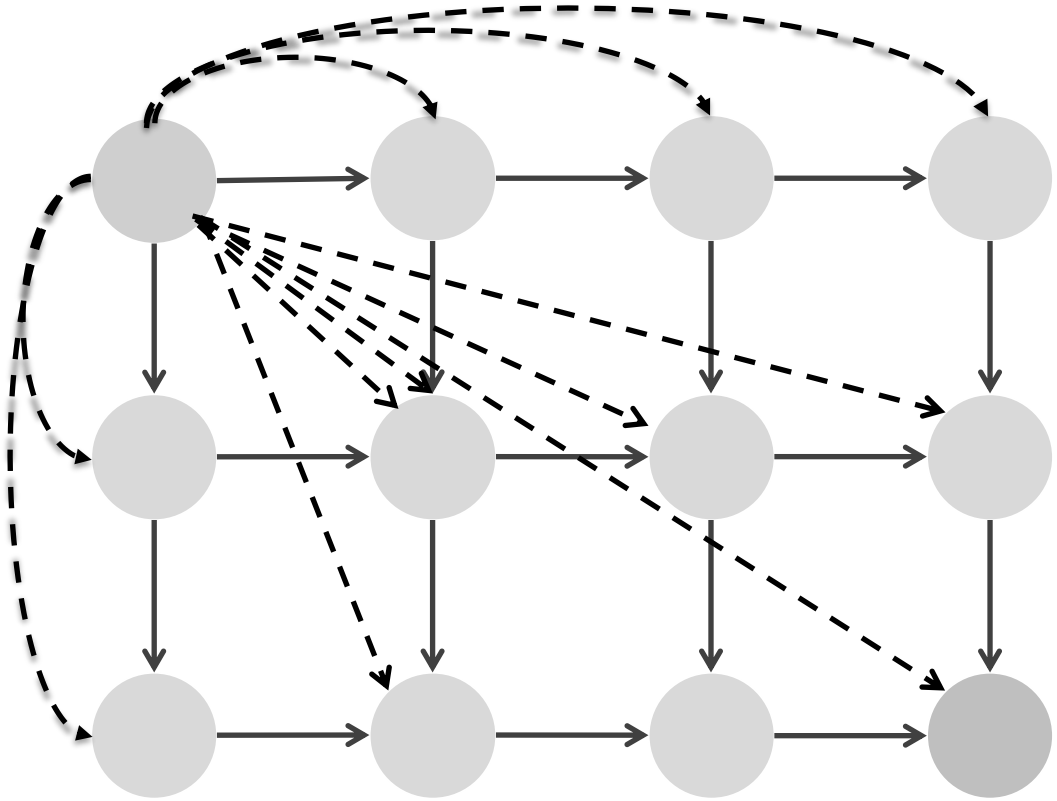
GCC–C–AGT–TATGT–CAGGGGGCACG––A–GCATGCACA–
GCCGCC–GTCGT–T–TTCAG––––CA–GTTATG–T–CAGAT

Global alignment

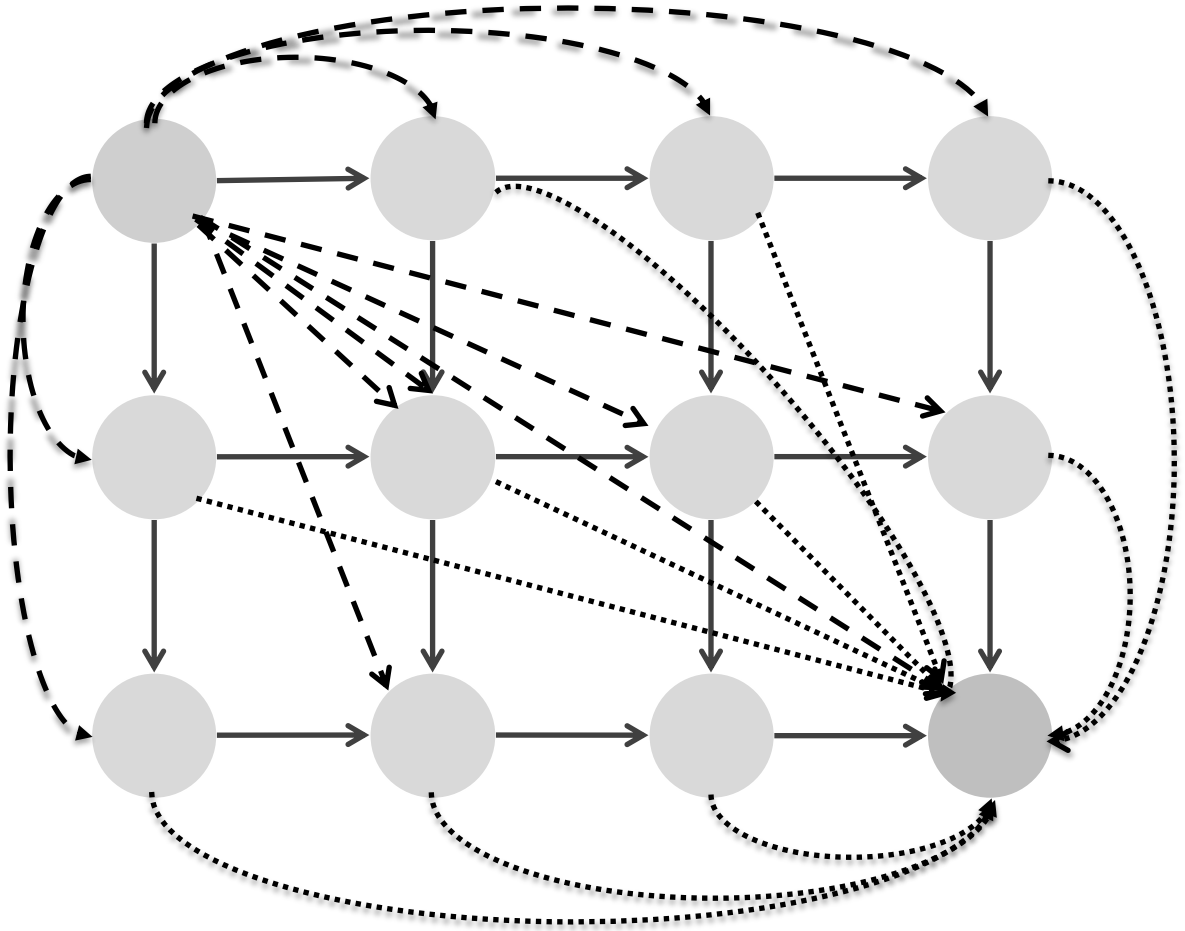
---G----C-----C--**CAGTTATGTCAG**GGGGCACGAGCATGCACA
GCCGCCGTCGTTTTCAG**CAGTTATGTCAG**-----A------T ----

Local alignment

What Do Free Taxi Rides Mean in the Terms of the Alignment Graph?



Building Manhattan for the Local Alignment Problem

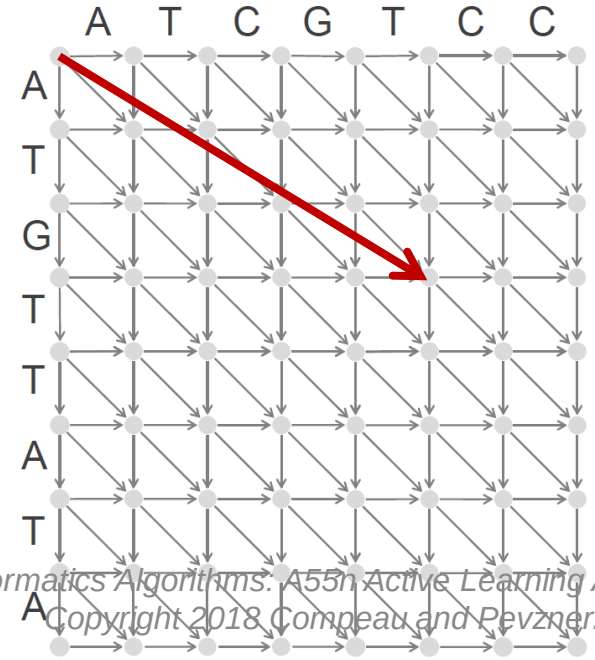


How many edges have we added?

Dynamic Programming for the Local Alignment

weight of edge from (0,0) to (i,j)

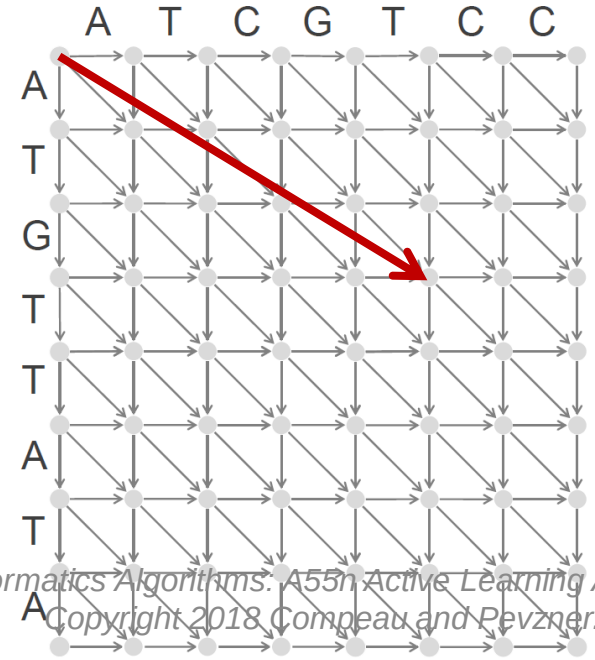
$$s_{i,j} = \max \begin{cases} s_{i-1,j} + \text{weight of edge "}\downarrow\text{" into } (i,j) \\ s_{i,j-1} + \text{weight of edge "}\rightarrow\text{" into } (i,j) \\ s_{i-1,j-1} + \text{weight of edge "}\searrow\text{" into } (i,j) \end{cases}$$



Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Dynamic Programming for the Local Alignment

$$s_{i,j} = \max \begin{cases} 0 \\ s_{i-1,j} + \text{weight of edge “}\downarrow\text{” into } (i,j) \\ s_{i,j-1} + \text{weight of edge “}\rightarrow\text{” into } (i,j) \\ s_{i-1,j-1} + \text{weight of edge “}\searrow\text{” into } (i,j) \end{cases}$$



Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

How Do We Compare Biological Sequences

- The Alignment Game and the Longest Common Subsequence
- The Change Problem
- Dynamic Programming and Backtracking Pointers
- From Manhattan to the Alignment Graph
- From Global to Local Alignment
- **Penalizing Insertions and Deletions in Sequence Alignment**
- Space-Efficient Sequence Alignment
- Multiple Sequence Alignment

Copyright 2018 Compeau and Pevzner.

Scoring Gaps

- We previously assigned a fixed penalty σ to each indel.
- However, this fixed penalty may be too severe for a series of 100 consecutive indels.
- A series of *k* indels often represents a single evolutionary event (**gap**) rather than *k* events:

two gaps (lower score)

```
GATCCAG
GA-C-AG
```

```
GATCCAG
GA--CAG
```

a single gap (higher score)

More Adequate Gap Penalties

Affine gap penalty for a gap of length *k*: $\sigma+\varepsilon\cdot(k-1)$

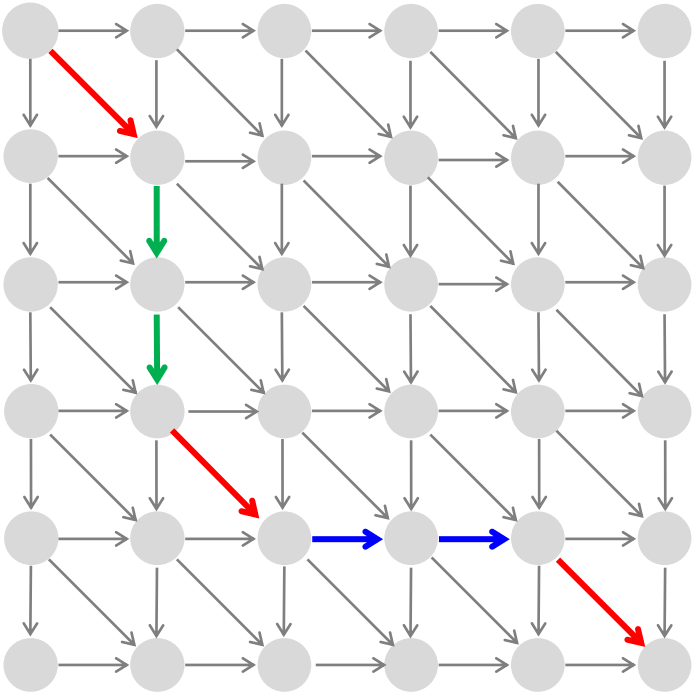
σ - the **gap opening penalty**

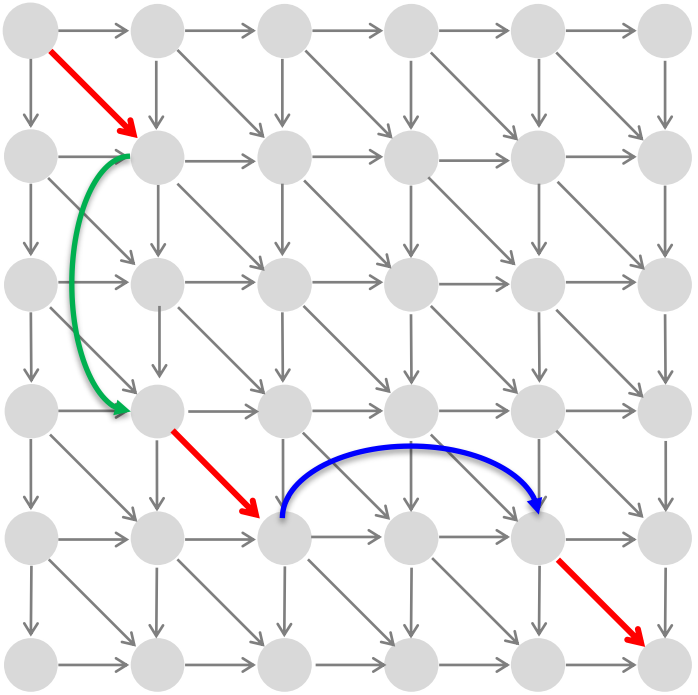
ε - the **gap extension penalty**

$\sigma > \varepsilon$, since starting a gap should be penalized more than extending it.

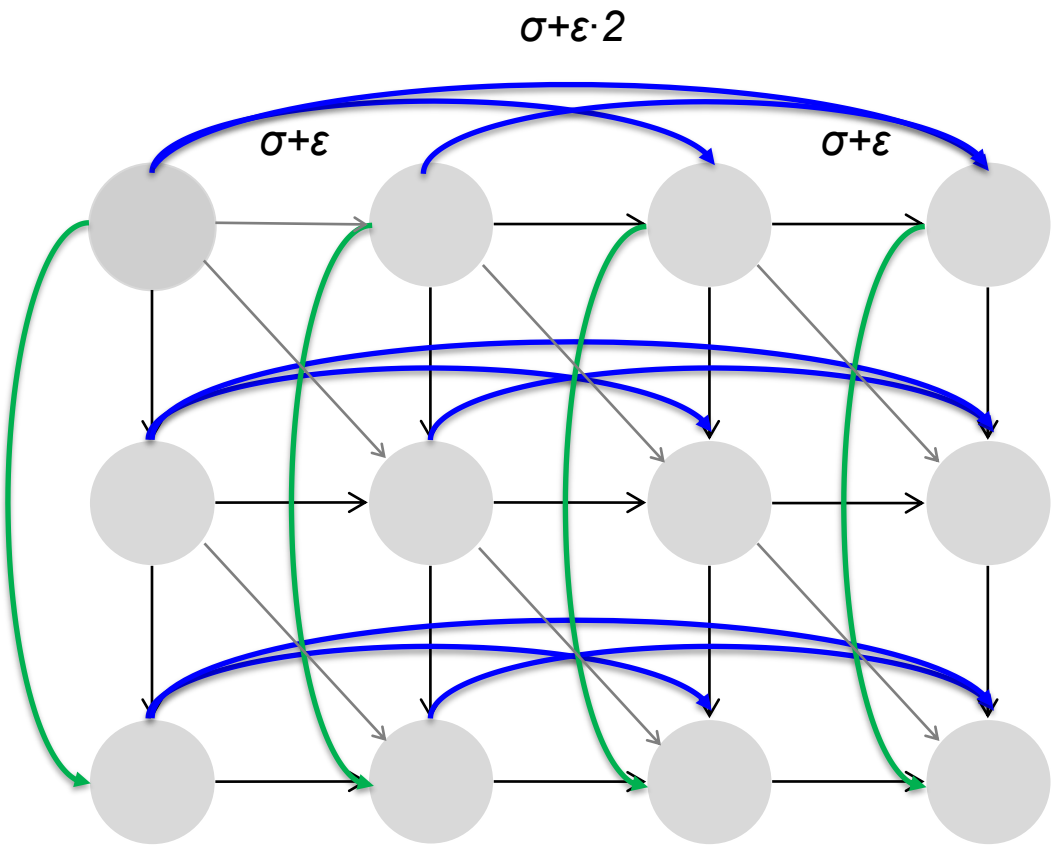
Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Modelling Affine Gap Penalties by Long Edges



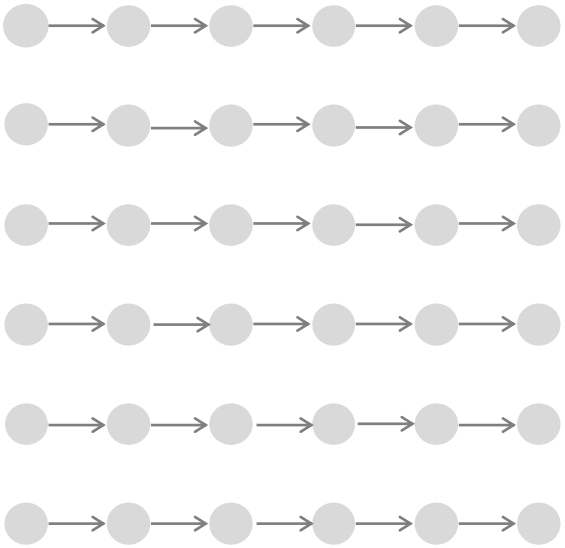


Building Manhattan with Affine Gap Penalties

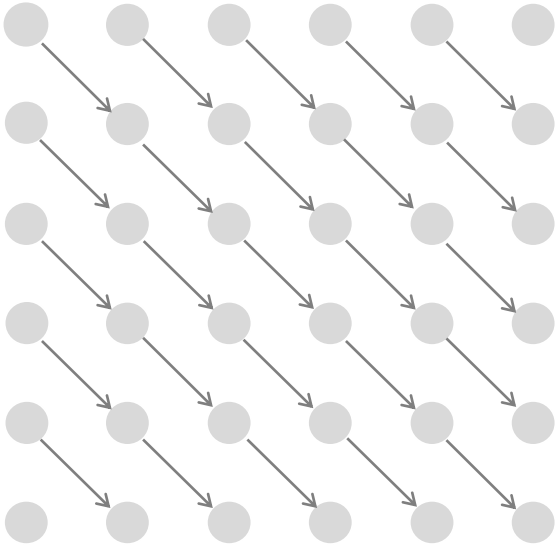


We have just added O(n^3) edges to the graph…

Building Manhattan on 3 levels

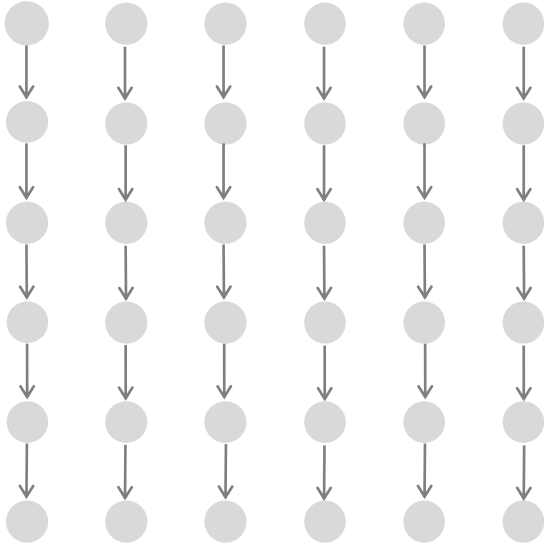


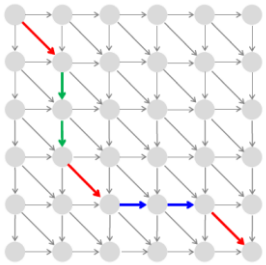
upper level
(deletions)



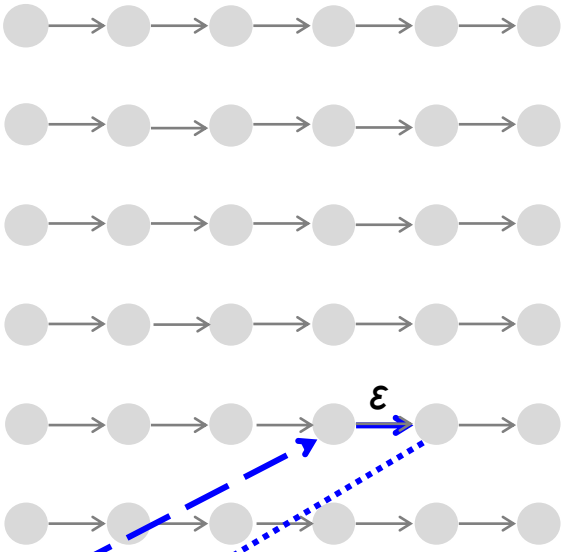
middle level
(matches/mismatches)

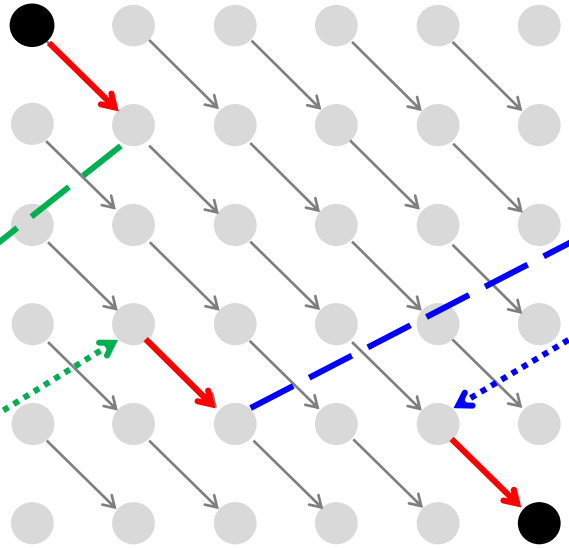
bottom level
(insertions)





How can we emulate this path in the 3-level Manhattan?





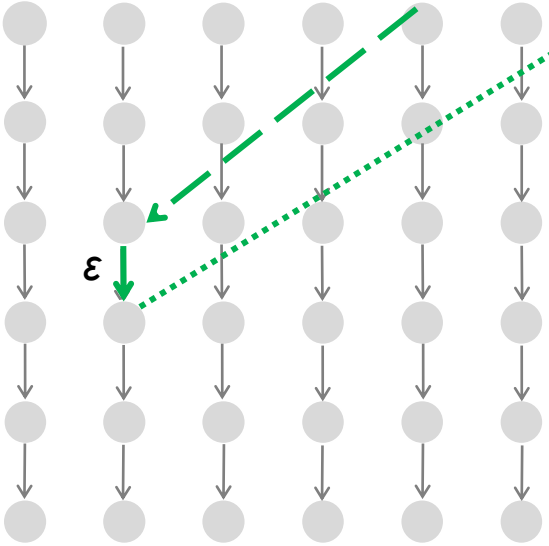
$lower_{i,j} = \max \{ lower_{i-1,j} - \varepsilon,\ middle_{i-1,j} - \sigma \}$

σ

ε

0

$upper_{i,j} = \max \{ upper_{i,j-1} - \varepsilon,\ middle_{i,j-1} - \sigma \}$



$lower_{i,j}$

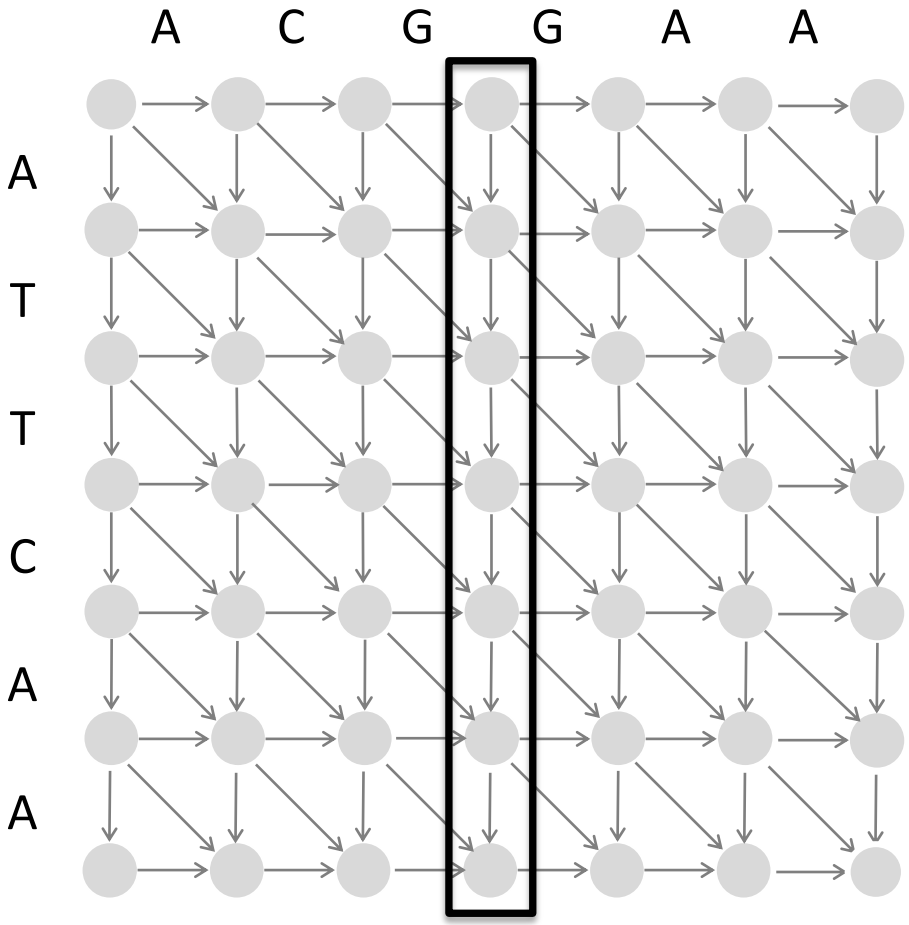
$middle_{i,j} = \max \{ middle_{i-1,j-1} + score(v_i, w_j),\ lower_{i,j},\ upper_{i,j} \}$

How Do We Compare Biological Sequences

- The Alignment Game and the Longest Common Subsequence
- The Manhattan Tourist Problem
- Dynamic Programming and Backtracking Pointers
- From Manhattan to an Arbitrary Directed Acyclic Graph
- From Global to Local Alignment
- Penalizing Insertions and Deletions in Sequence Alignment
- **Space-Efficient Sequence Alignment**
- Multiple Sequence Alignment

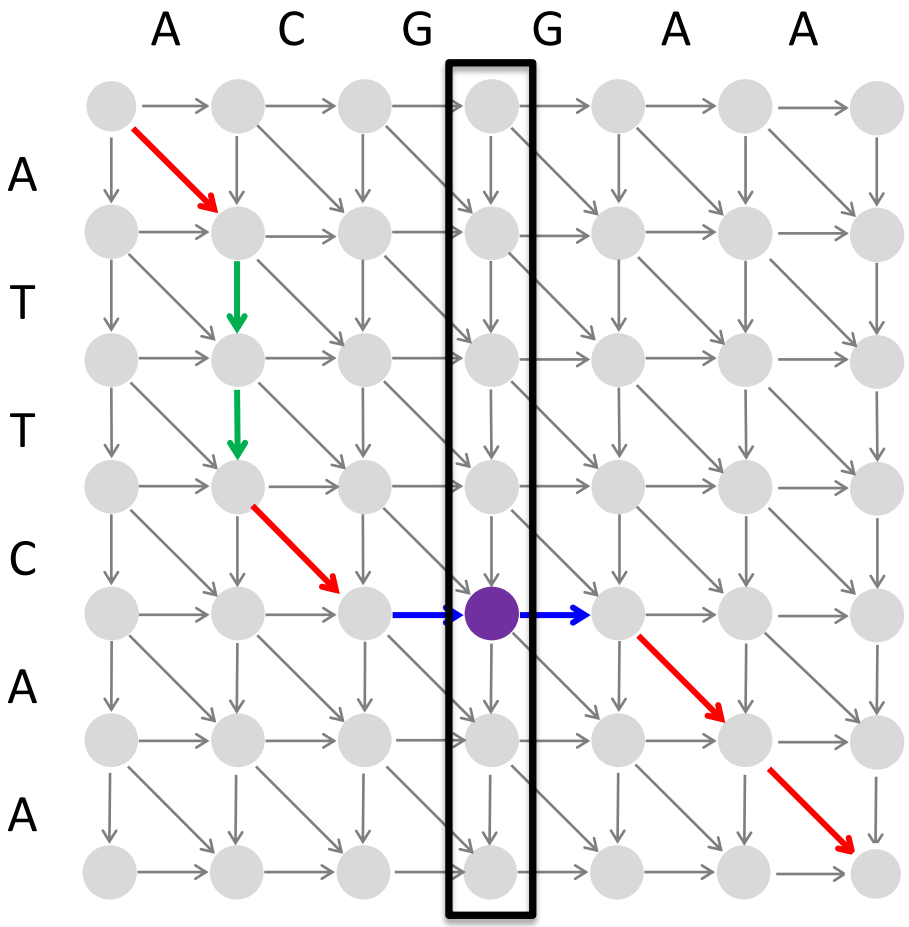
Copyright 2018 Compeau and Pevzner.

Middle Column of the Alignment



middle column
(*middle*=#columns/2)

Middle Node of the Alignment

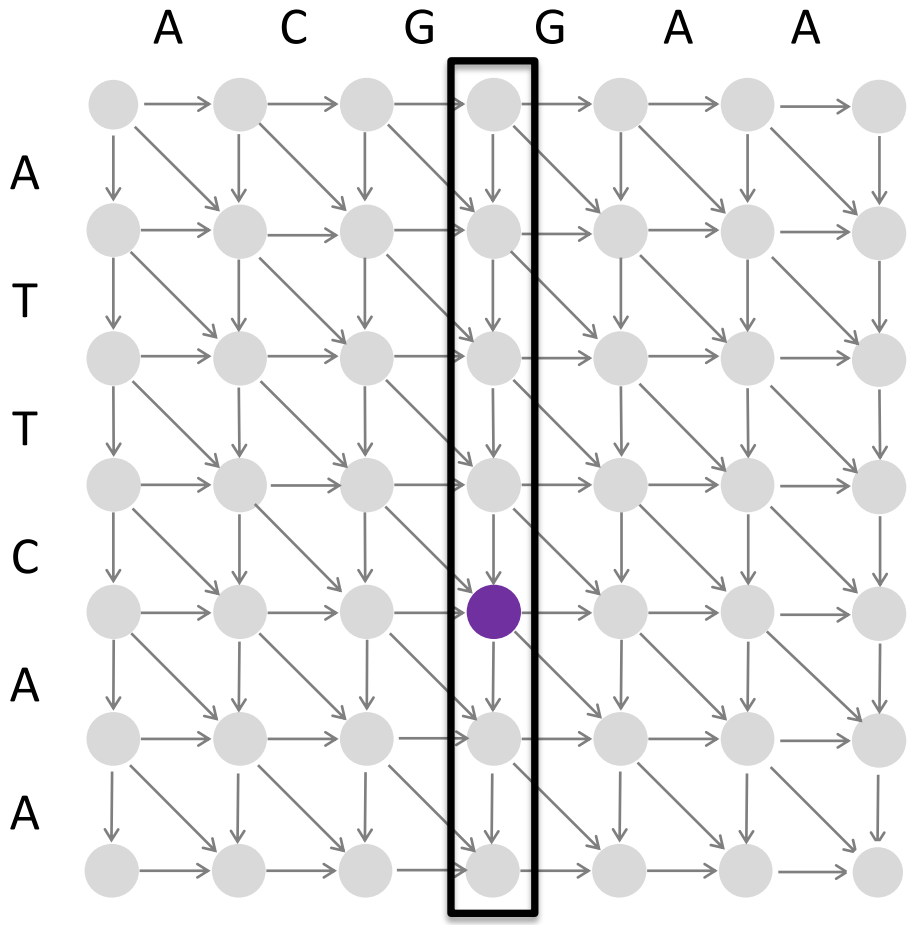


(a node where an optimal alignment path crosses the middle column)

Divide and Conquer Approach to Sequence Alignment

AlignmentPath(*source, sink*)

find *MiddleNode*



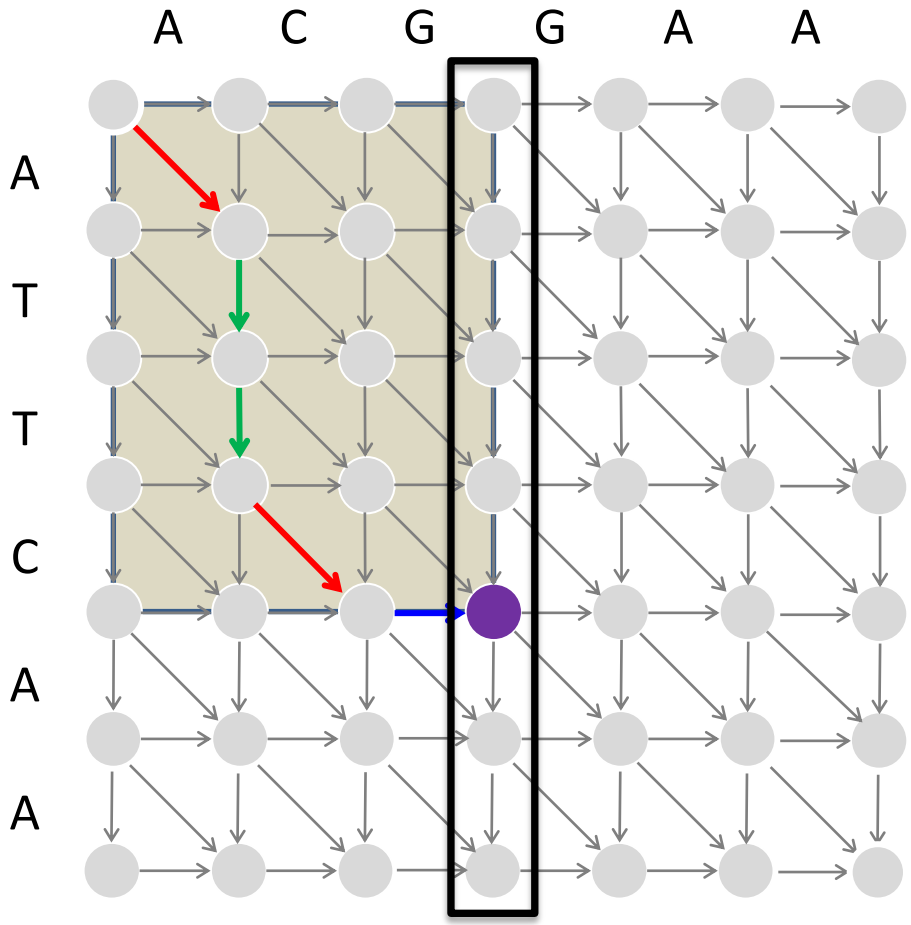
Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Divide and Conquer Approach to Sequence Alignment

AlignmentPath(*source, sink*)

find *MiddleNode*

AlignmentPath(*source, MiddleNode*)



Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

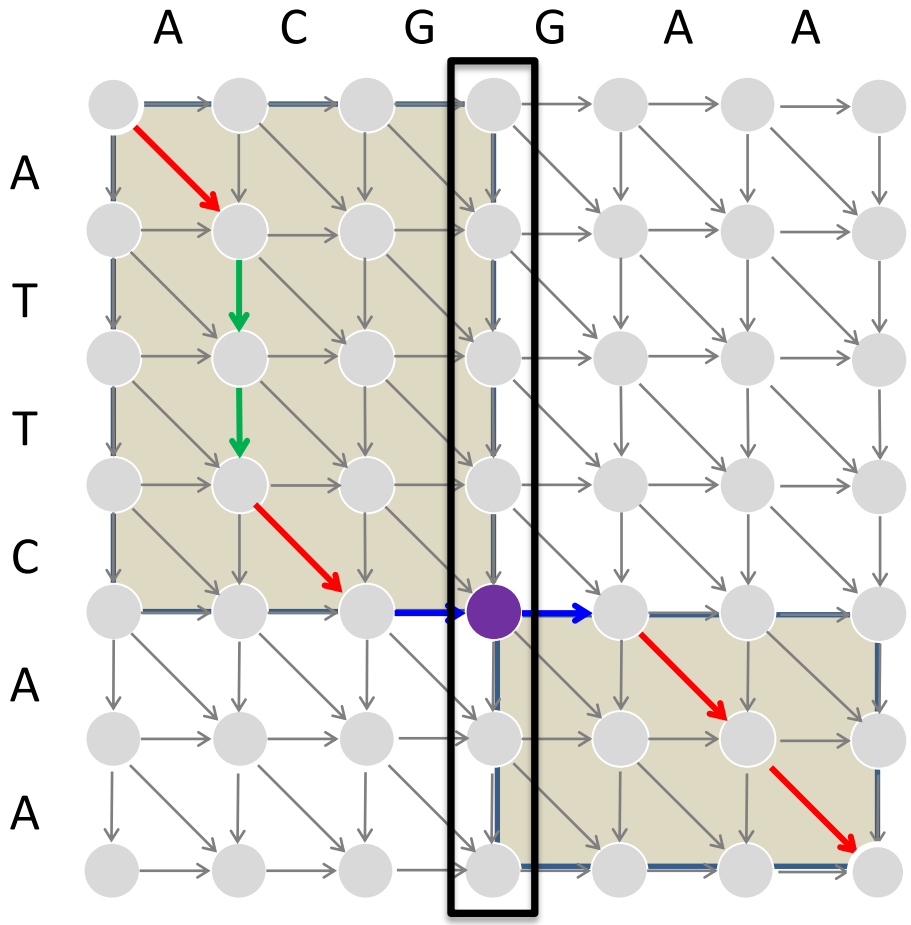
Divide and Conquer Approach to Sequence Alignment

AlignmentPath(*source, sink*)

find *MiddleNode*

AlignmentPath(*source, MiddleNode*)

AlignmentPath(*MiddleNode, sink*)



The only problem left is how to find this middle node in **linear space**!

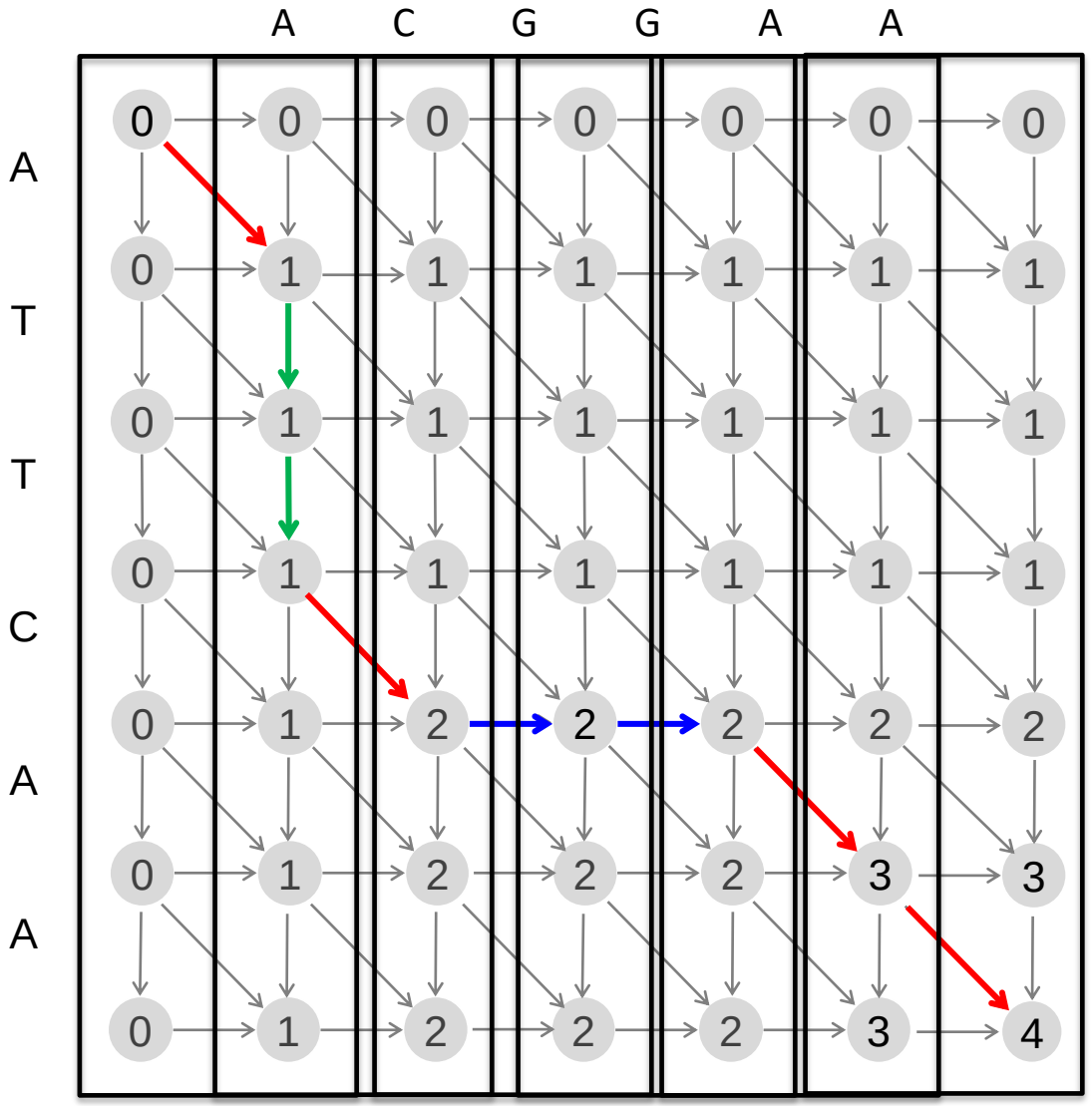
Computing Alignment Score in Linear Space

Finding the **longest path** in the alignment graph **requires** storing all backtracking pointers – O(*nm*) memory.

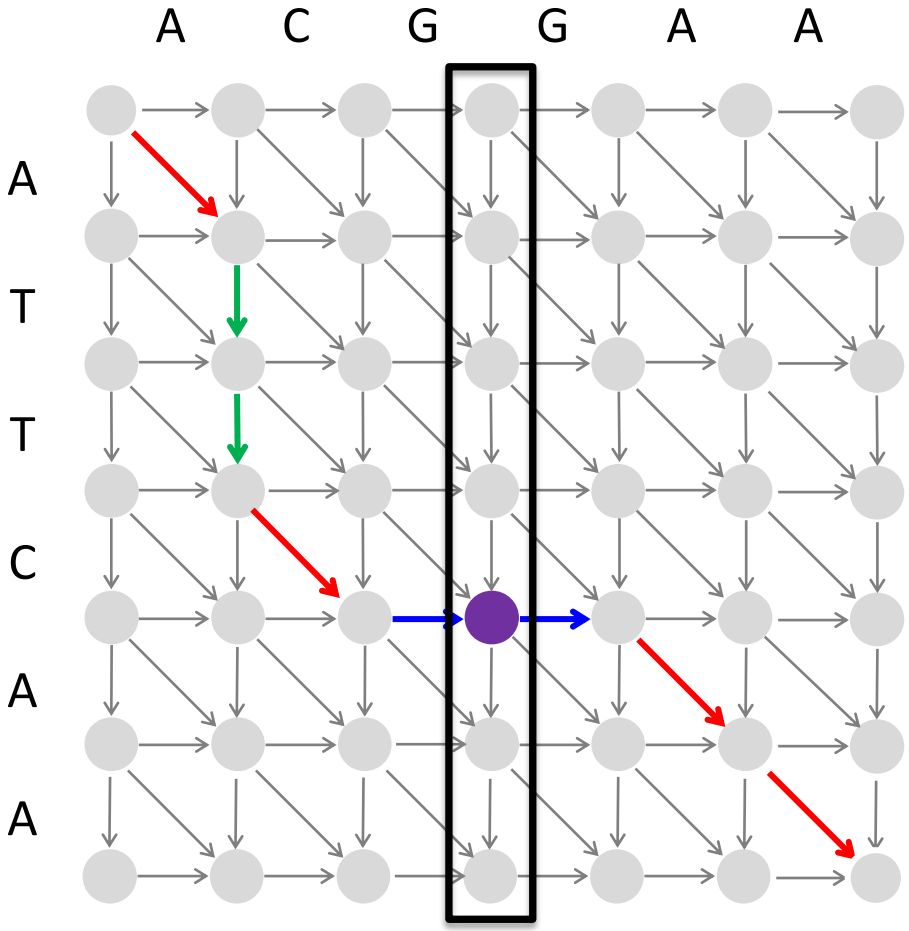
Finding the **length of the longest path** in the alignment graph **does not require** storing any backtracking pointers – O(*n*) memory.

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Recycling the Columns in the Alignment Graph



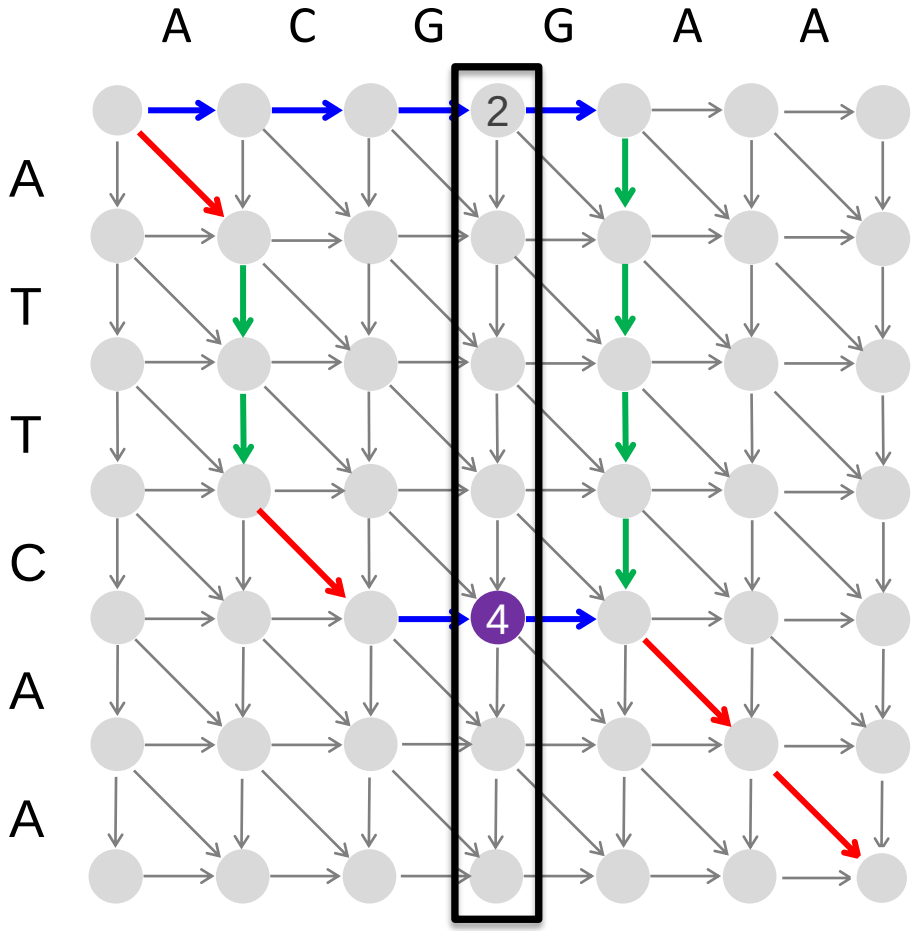
Can We Find the Middle Node without Constructing the Longest Path?



4-path that visits the node (4,middle) In the middle column

***i*-path** – a longest path among paths that visit the *i*-th node in the middle column

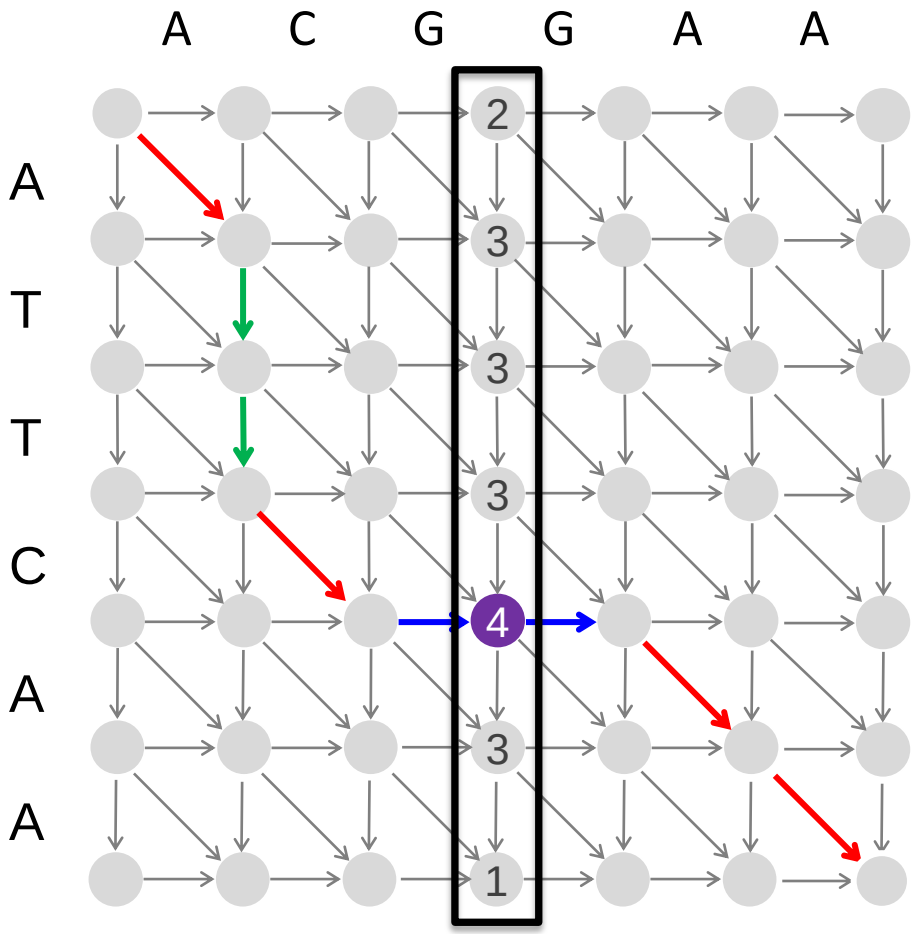
Can We Find The Lengths of All *i*-paths?



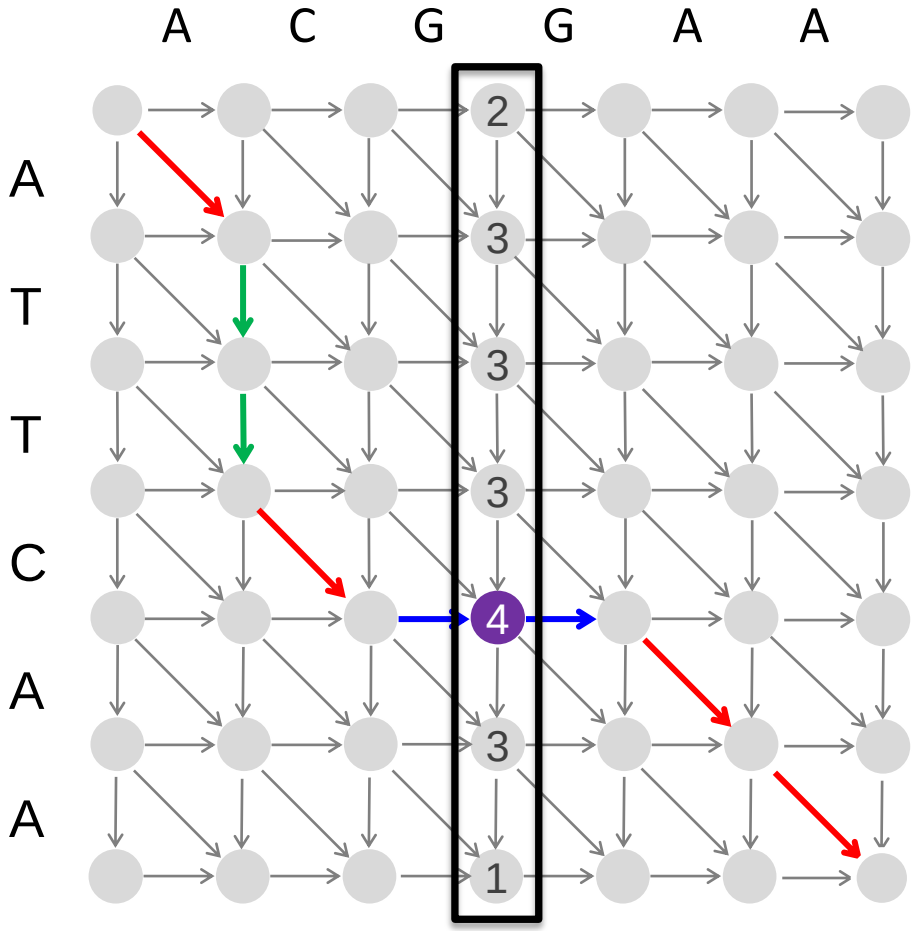
length(i):
length of an *i*-path:

length(0)=2
length(4)=4

Can We Find The Lengths of All *i*-paths?



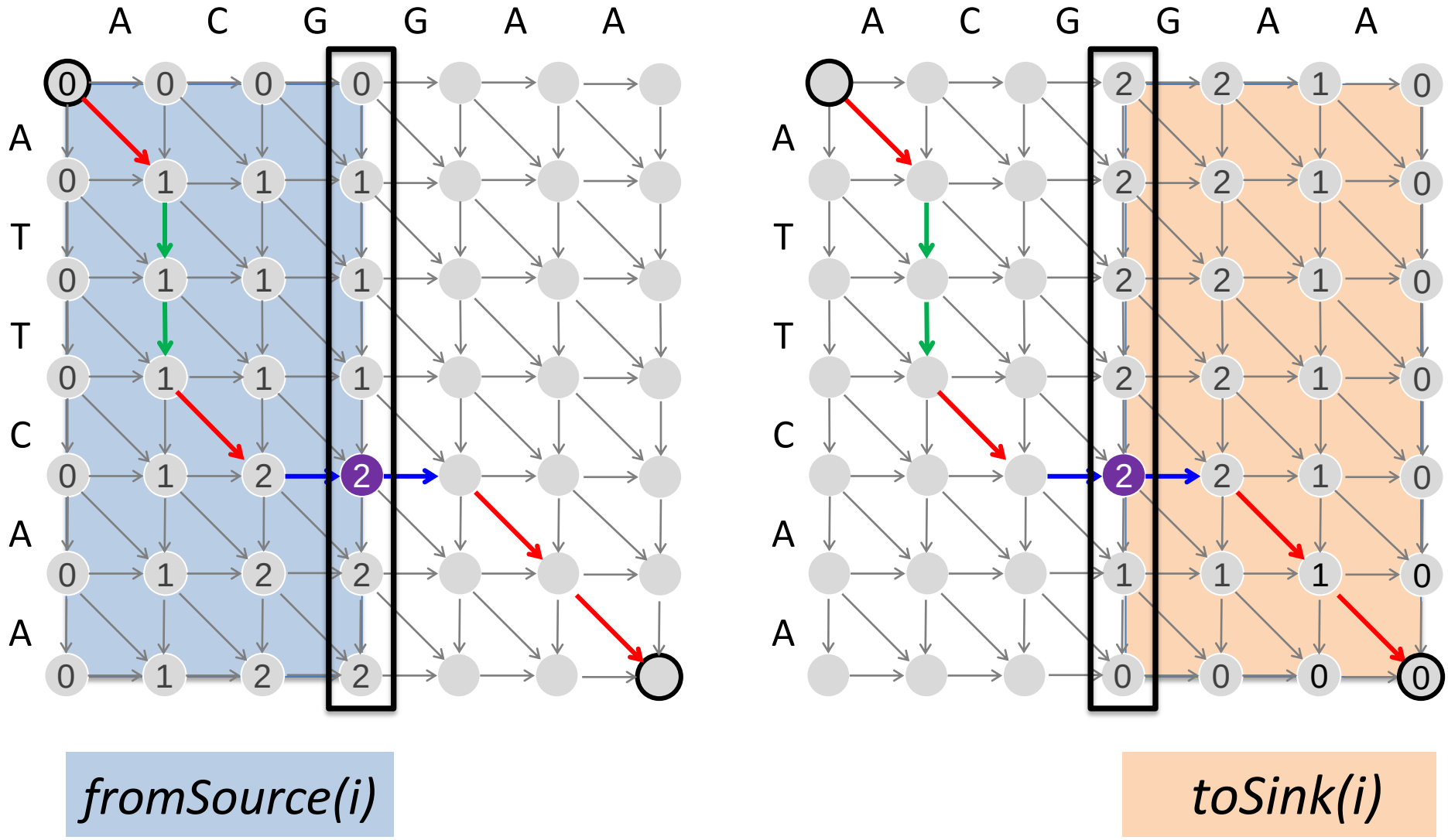
Can We Find The Lengths of *i*-paths?



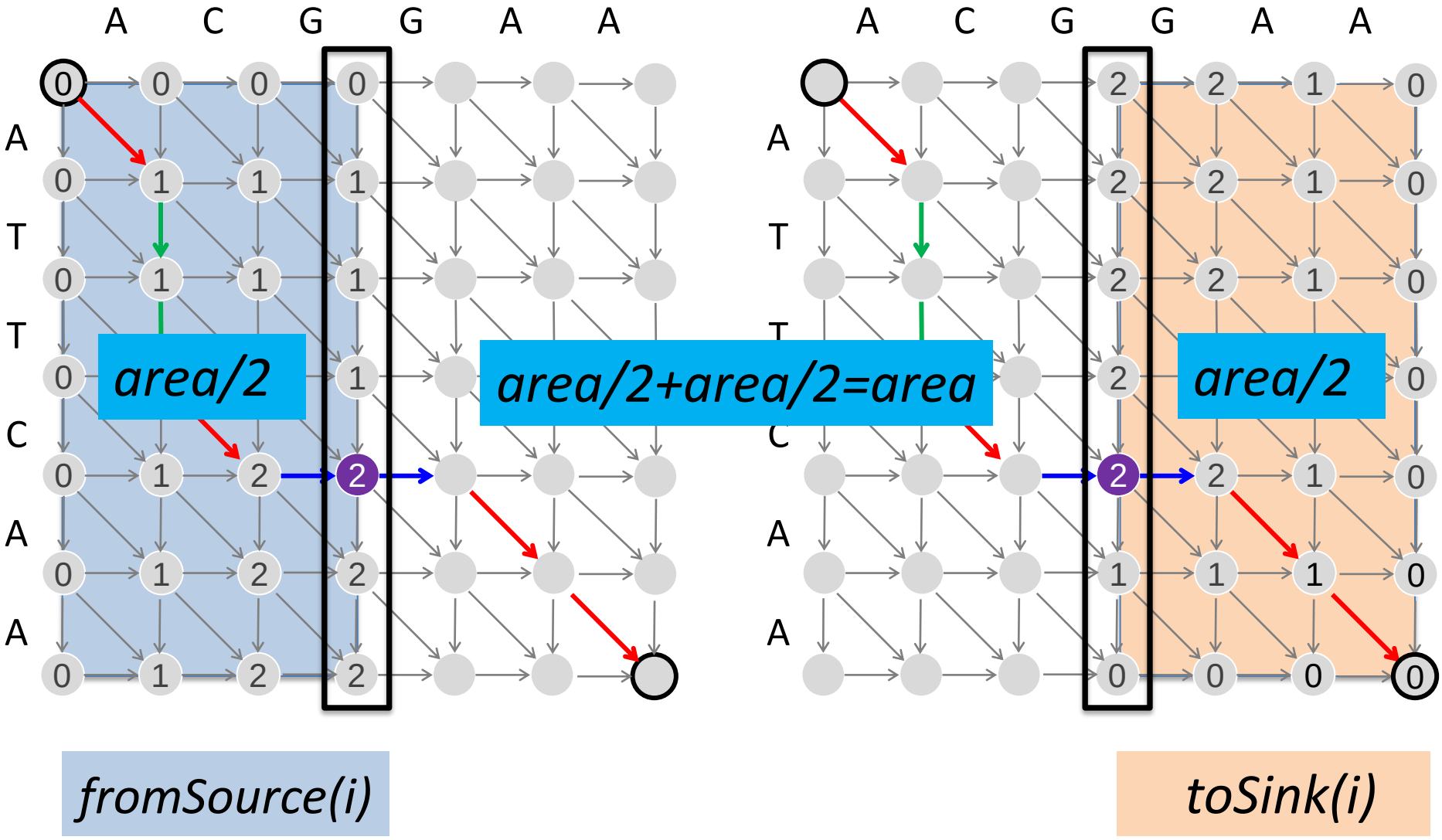
length(i):
length of an *i*-path

length(i)=fromSource(i)+toSink(i)

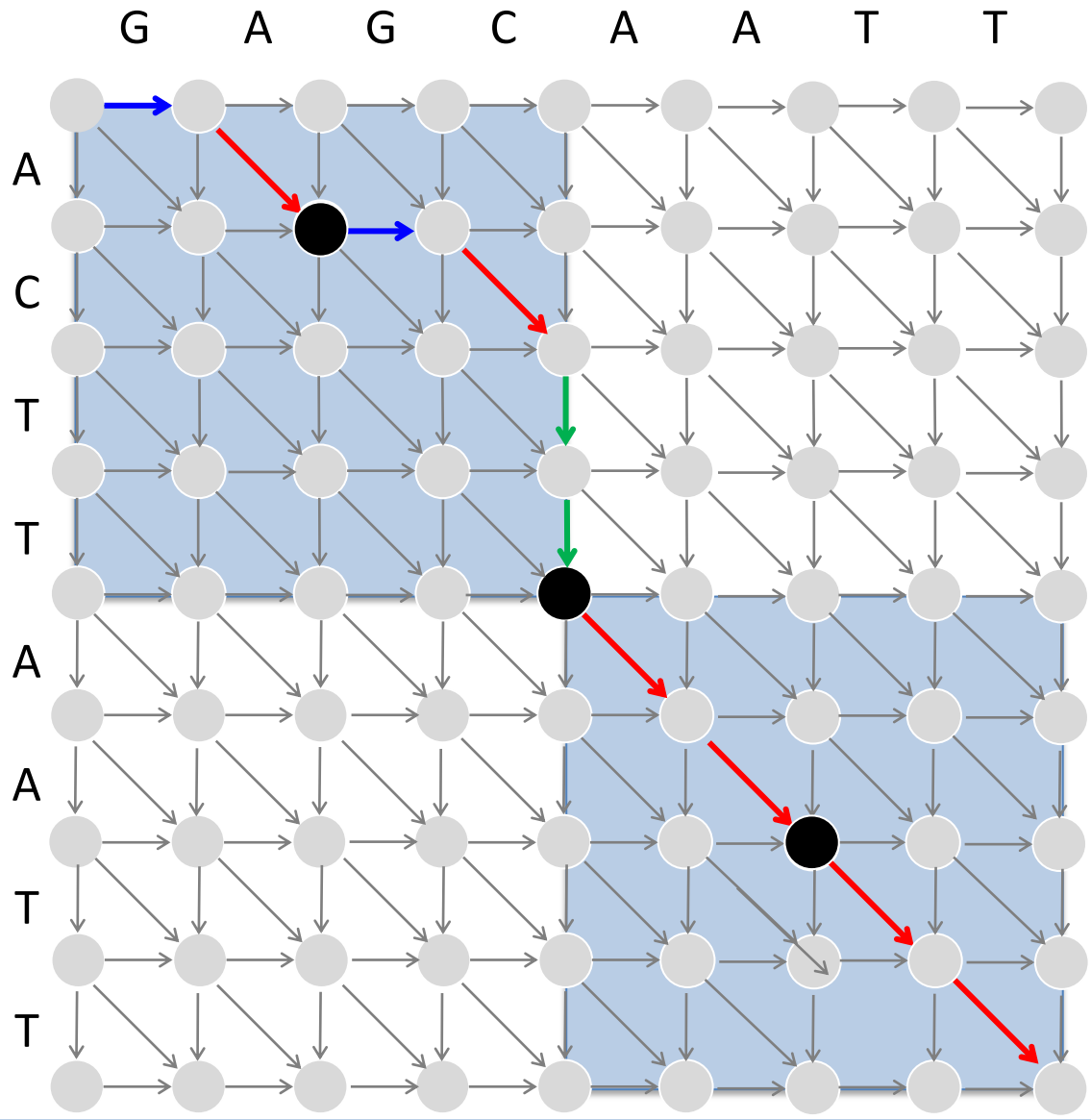
Computing *FromSource* and *toSink*



How Much Time Did It Take to Find the Middle Node ?



Laughable Progress: O(*nm*) Time to Find **ONE** Node!

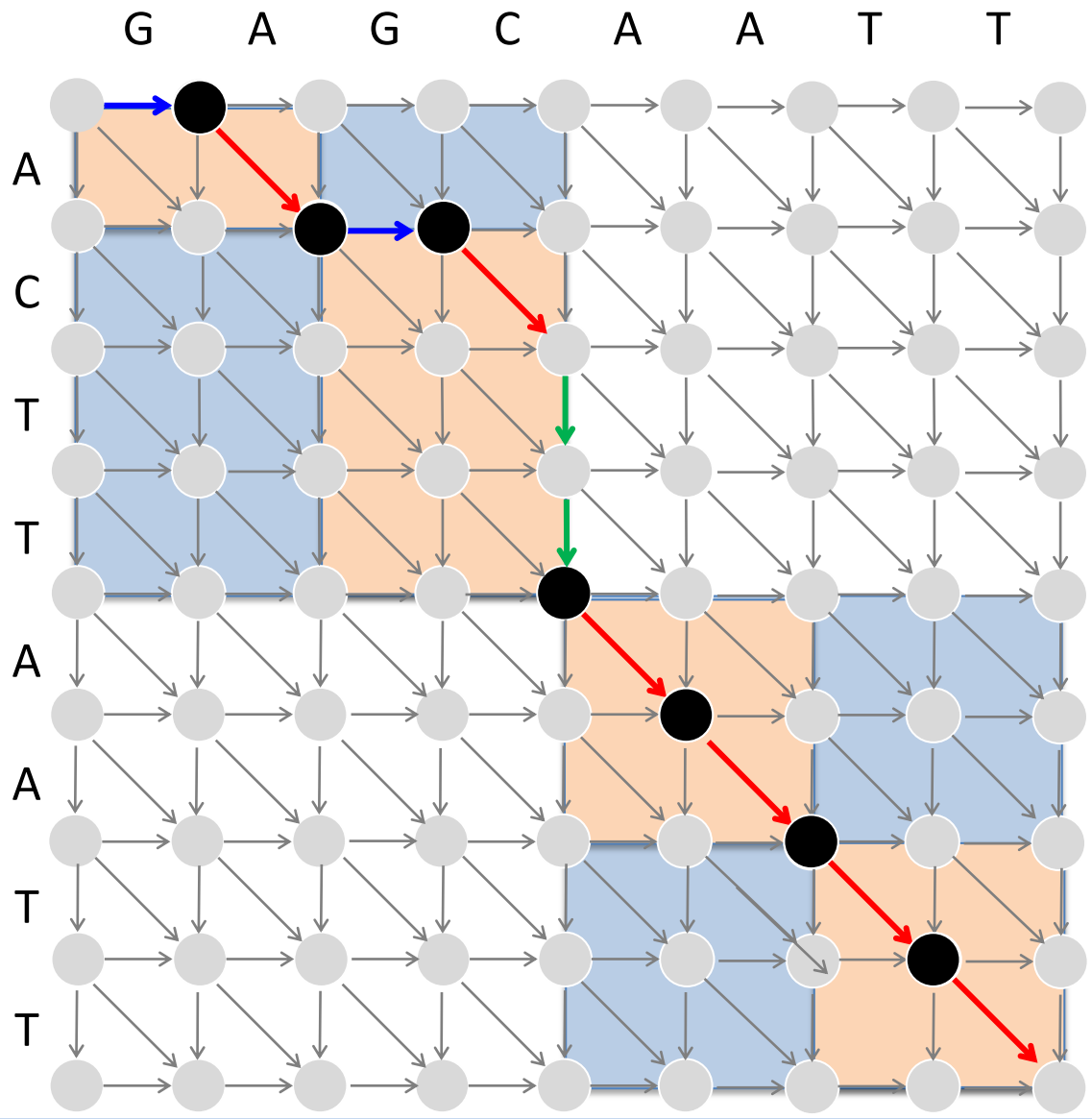


Each subproblem can be conquered in time proportional to its area:

area/4+area/4= ***area/2***

How much time would it take to conquer 2 subproblems?

Laughable Progress: O(*nm+nm/2*) Time to Find **THREE** Nodes!

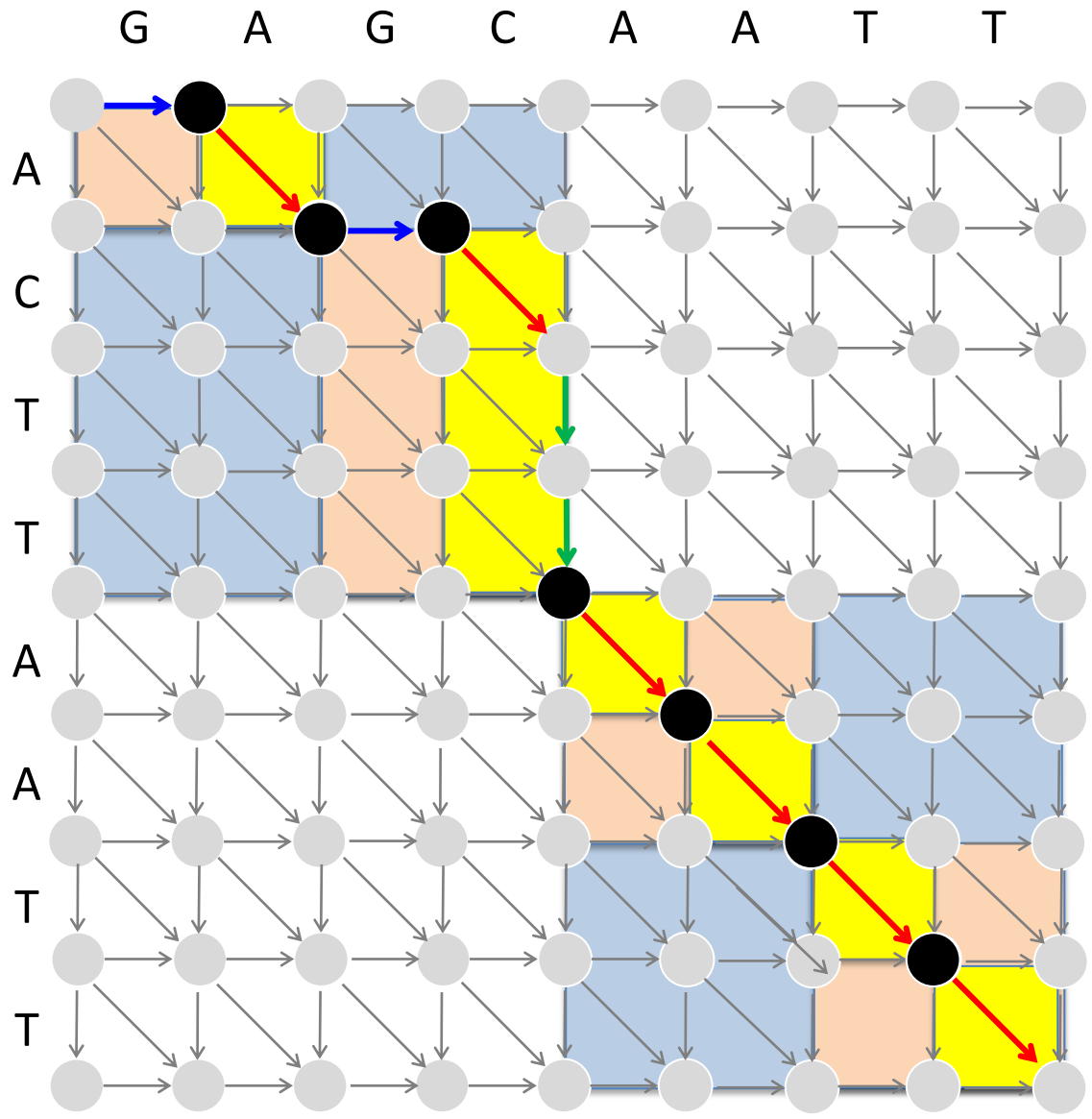


Each subproblem can be conquered in time proportional to its area:

area/8+area/8+ area/8+area/8= ***area/4***

How much time would it take to conquer 4 subproblems?

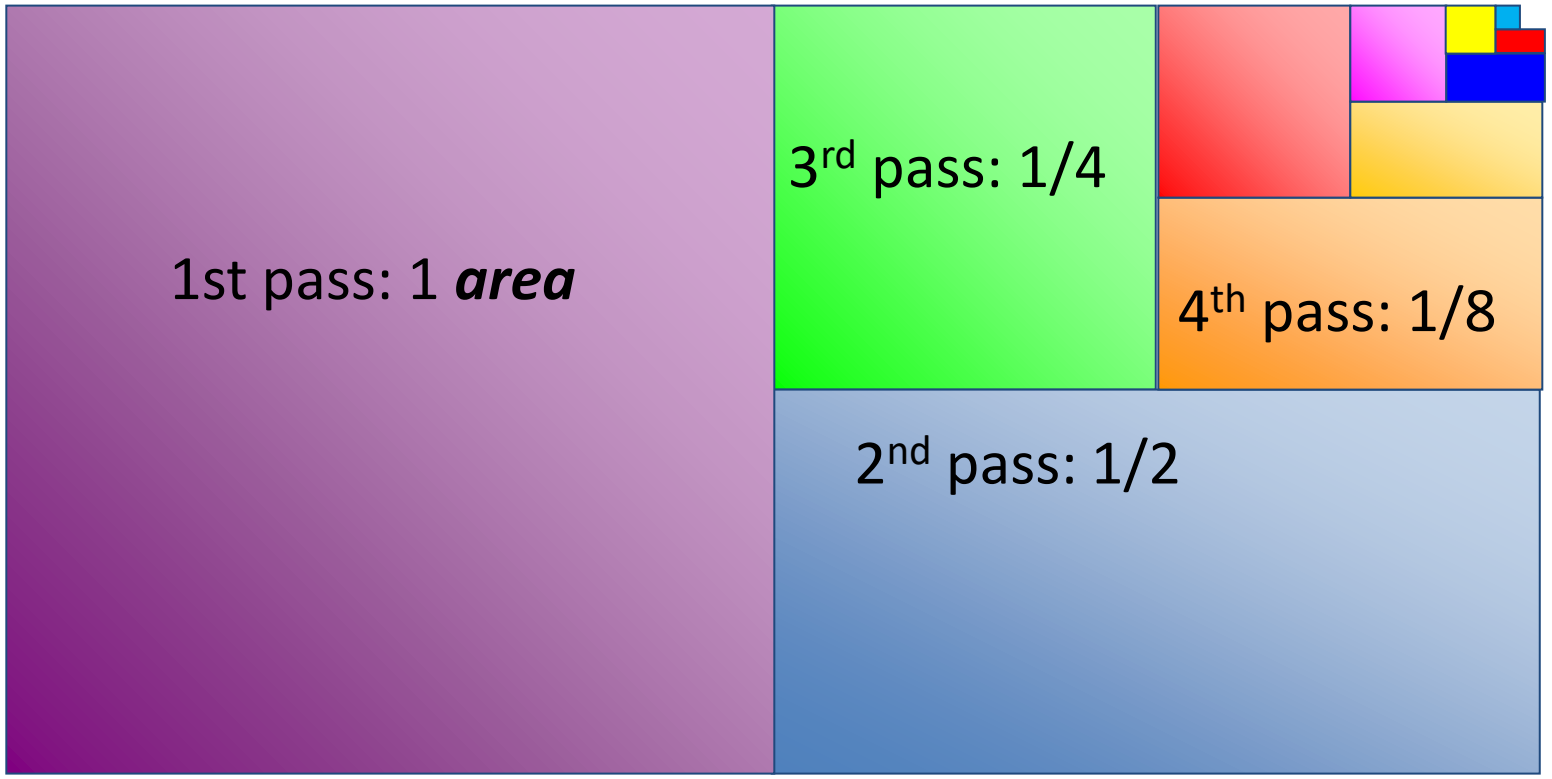
O(*nm+nm/2+nm/4*) Time to Find **NEARLY ALL** Nodes!



area+
area/2
+area/4
+area/8
+area/16
+….+
<
2·area

How much time would it take to conquer ALL subproblems?

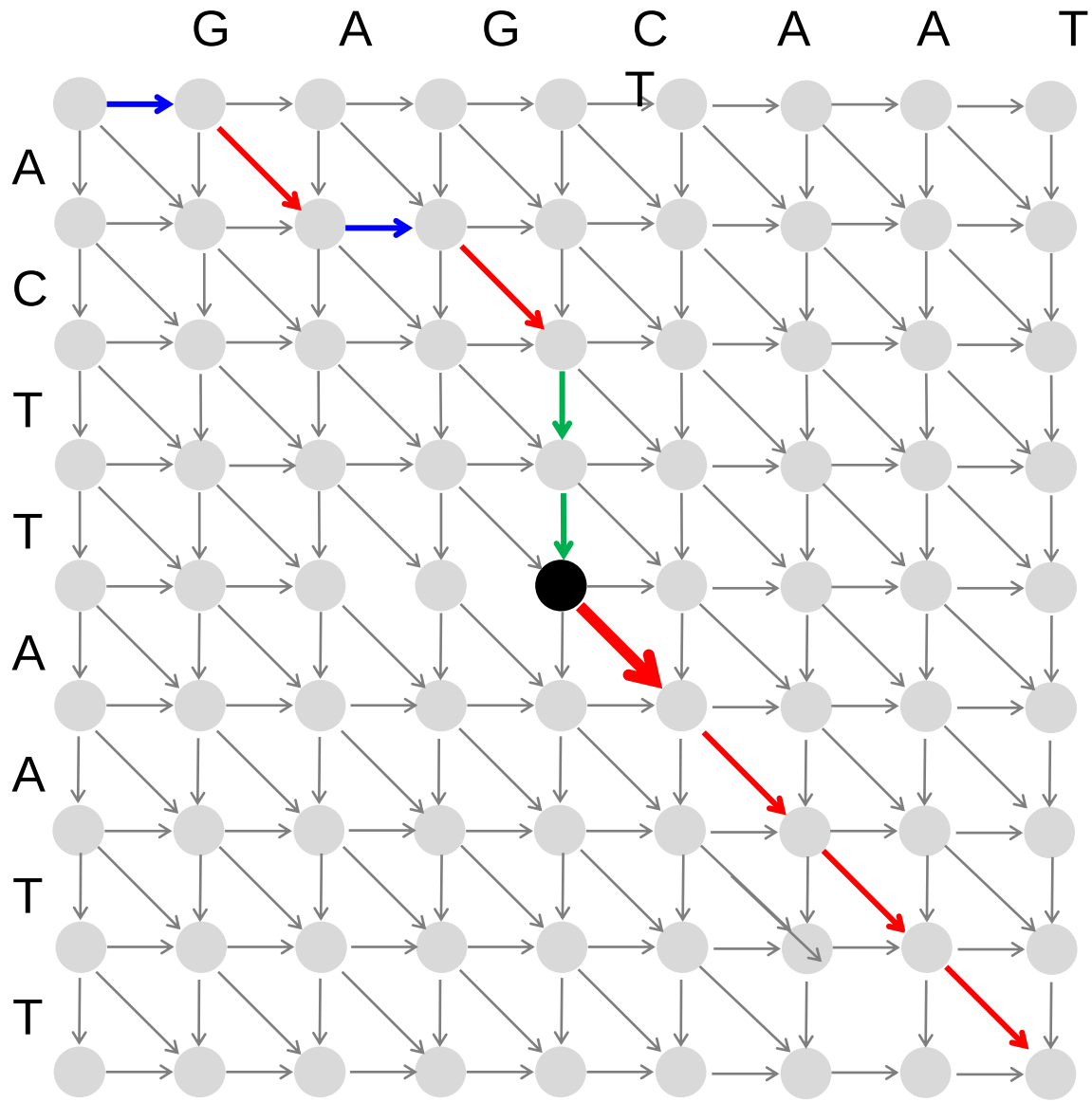
Total Time: ***area+area/2+area/4+area/8+area/16+…***



$$1 + ½ + ¼ + \ldots < 2$$

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

The Middle Edge



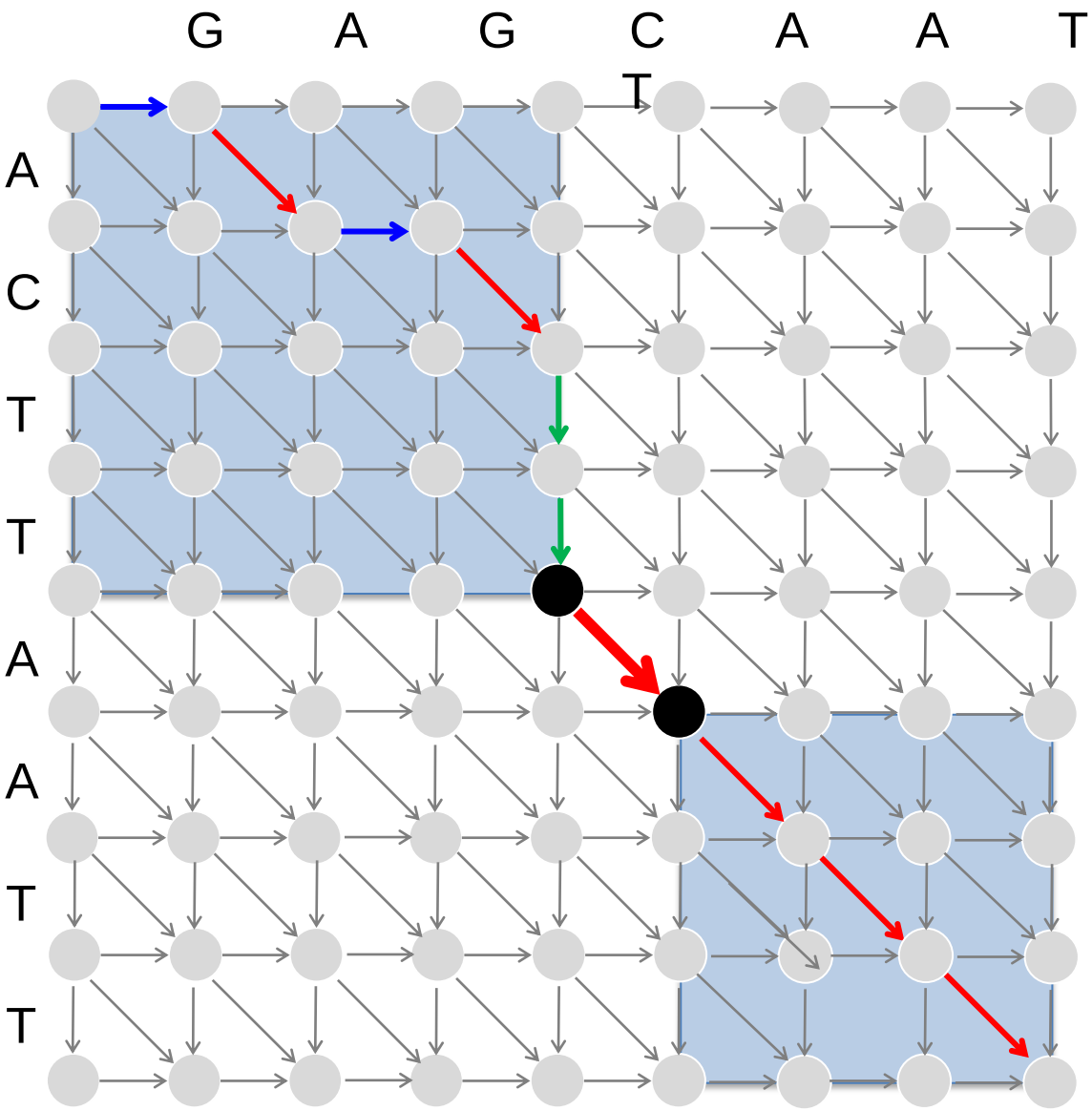
Middle Edge: an edge in an optimal alignment path starting at the middle node

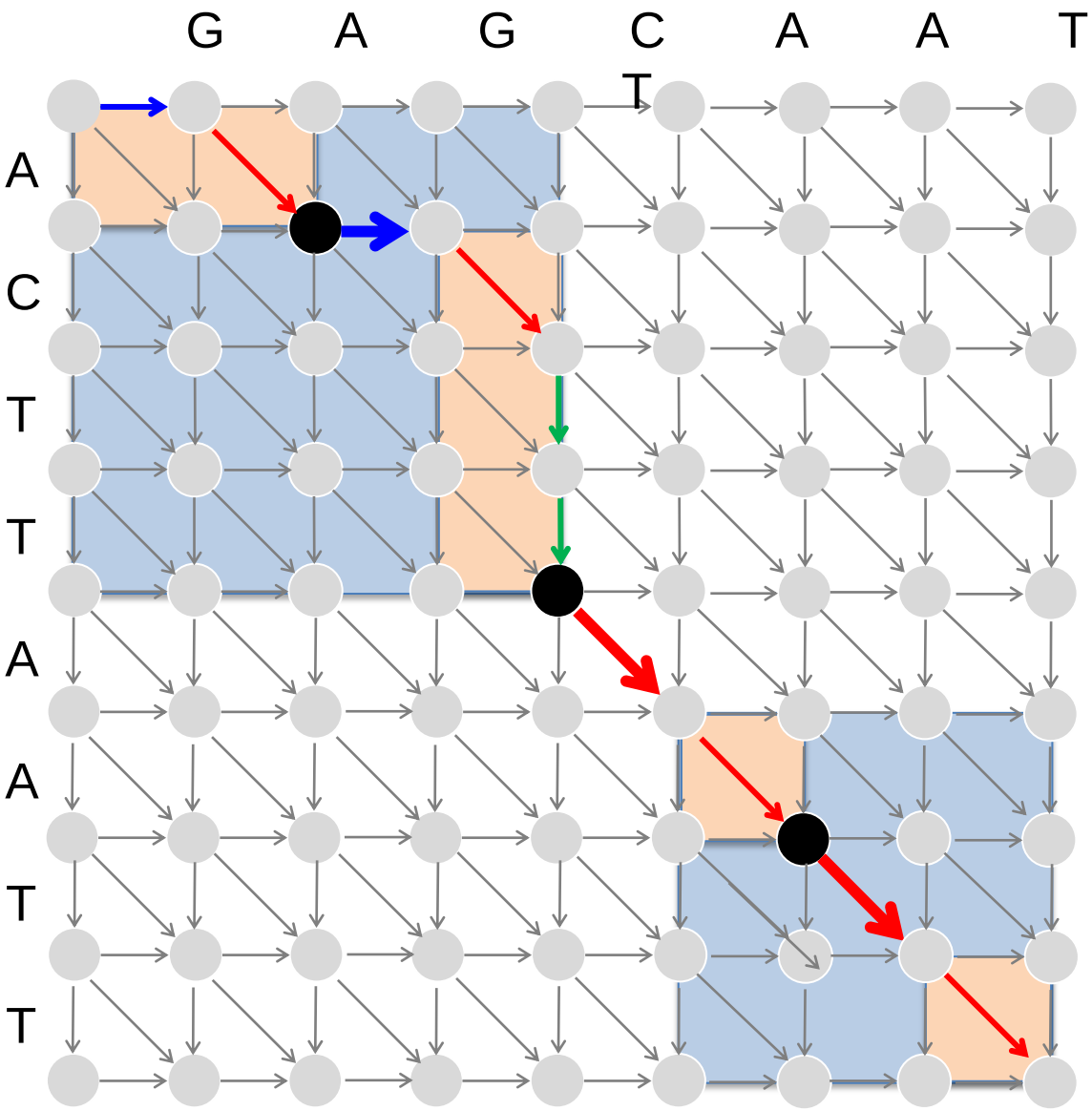
The Middle Edge Problem

Middle Edge in Linear Space Problem. Find a middle edge in the alignment graph in linear space.

- **Input:** Two strings and matrix ***score***.
- **Output:** A middle edge in the alignment graph of these strings (as defined by the matrix ***score***).

Copyright 2018 Compeau and Pevzner.


G A G C A A T
T
A
C
T
T
A
A
T
T


G A G C A A T
T
A
C
T
T
A
A
T
T

Recursive **LinearSpaceAlignment**

```
LinearSpaceAlignment(top,bottom,left,right)
  if left = right
    return alignment formed by bottom-top edges "↓"
  middle ← |(left+right)/2|
  midNode ← MiddleNode(top,bottom,left,right)
  midEdge ←  MiddleEdge(top,bottom,left,right)
  LinearSpaceAlignment(top,midNode,left,middle)
  output midEdge
  if midEdge = "→" or midEdge = "↘"
    middle  ← middle+1
  if  midEdge = "↓" or midEdge = "↘"
    midNode  ← midNode+1
  LinearSpaceAlignment(midNode,bottom,middle,right)
```

How Do We Compare Biological Sequences

- The Alignment Game and the Longest Common Subsequence
- The Manhattan Tourist Problem
- Dynamic Programming and Backtracking Pointers
- From Manhattan to an Arbitrary Directed Acyclic Graph
- From Global to Local Alignment
- Penalizing Insertions and Deletions in Sequence Alignment
- Space-Efficient Sequence Alignment
- **Multiple Sequence Alignment**

Copyright 2018 Compeau and Pevzner.

From Pairwise to Multiple Alignment

- Up until now we have only tried to align two sequences.
- A faint (and statistically insignificant) similarity between two sequences becomes significant if it is present in many other sequences.
- Multiple alignments can reveal subtle similarities that pairwise alignments do not reveal.



Alignment of Three A-domains

```
YAFDLGYTCMFPVLLGGGELHIVQKETYTAPDEIAHYIKEHGITYIKLTPSLFHTIVNTASFAFDANFESLRLIVLGGEKIIPIDVIAFRKMYGHTE-FINHYGPTEATIGA
-AFDVSAGDFARALLTGGQLIVCPNEVKMDPASLYAIIKKYDITIFEATPALVIPLMEYI-YEQKLDISQLQILIVGSDSCSMEDFKTLVSRFGSTIRIVNSYGVTEACIDS
IAFDASSWEIYAPLLNGGTVVCIDYYTTIDIKALEAVFKQHHIRGAMLPPALLKQCLVSA----PTMISSLEILFAAGDRLSSQDAILARRAVGSGV-Y-NAYGPTENTVLS
```


Copyright 2018 Compeau and Pevzner.

Alignment of Three A-domains

```
YAFDLGYTCMFPVLLGGGELHIVQKETYTAPDEIAHYIKEHGITYIKLTPSLFHTIVNTASFAFDANFESLRLIVLGGEKIIPIDVIAFRKMYGHTE-FINHYGPTEATIGA
-AFDVSAGDFARALLTGGQLIVCPNEVKMDPASLYAIIKKYDITIFEATPALVIPLMEYI-YEQKLDISQLQILIVGSDSCSMEDFKTLVSRFGSTIRIVNSYGVTEACIDS
IAFDASSWEIYAPLLNGGTVVCIDYYTTIDIKALEAVFKQHHIRGAMLPPALLKQCLVSA----PTMISSLEILFAAGDRLSSQDAILARRAVGSGV-Y-NAYGPTENTVLS
```


Copyright 2018 Compeau and Pevzner.

Alignment of Three A-domains

```
YAFDLGYTCMFPVLLGGGELHIVQKETYTAPDEIAHYIKEHGITYIKLTPSLFHTIVNTASFAFDANFESLRLIVLGGEKIIPIDVIAFRKMYGHTE-FINHYGPTEATIGA
-AFDVSAGDFARALLTGGQLIVCPNEVKMDPASLYAIIKKYDITIFEATPALVIPLMEYI-YEQKLDISQLQILIVGSDSCSMEDFKTLVSRFGSTIRIVNSYGVTEACIDS
IAFDASSWEIYAPLLNGGTVVCIDYYTTIDIKALEAVFKQHHIRGAMLPPALLKQCLVSA----PTMISSLEILFAAGDRLSSQDAILARRAVGSGV-Y-NAYGPTENTVLS
```


Copyright 2018 Compeau and Pevzner.

Generalizing Pairwise to Multiple Alignment

- Alignment of 2 sequences is a 2-row matrix.
- Alignment of 3 sequences is a 3-row matrix

```
A T - G C G -
A - C G T - A
A T C A C - A
```

- Our scoring function should score alignments with conserved columns higher.

Alignments = Paths in 3-D

- Alignment of ATGC, AATC, and ATGC

	A	--	T	G	C

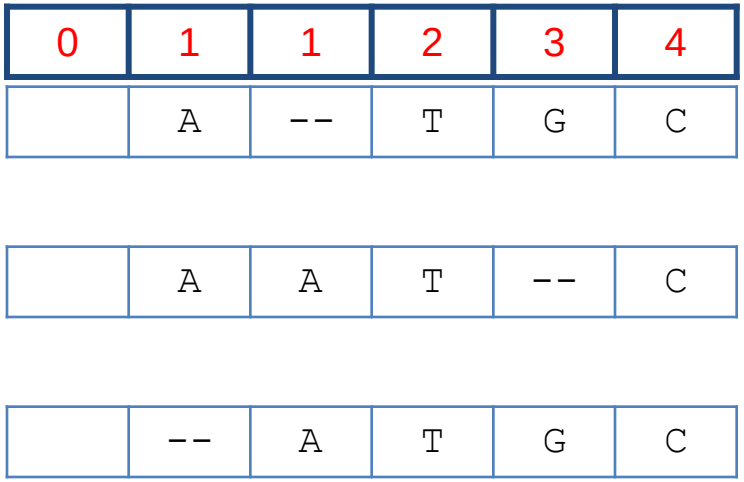
	A	A	T	--	C

	--	A	T	G	C

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Alignments = Paths in 3-D

- Alignment of ATGC, AATC, and ATGC



#symbols up to a given position

Alignments = Paths in 3-D

- Alignment of ATGC, AATC, and ATGC

0	1	1	2	3	4
	A	--	T	G	C

0	1	2	3	3	4
	A	A	T	--	C

	--	A	T	G	C

#symbols up to a given position

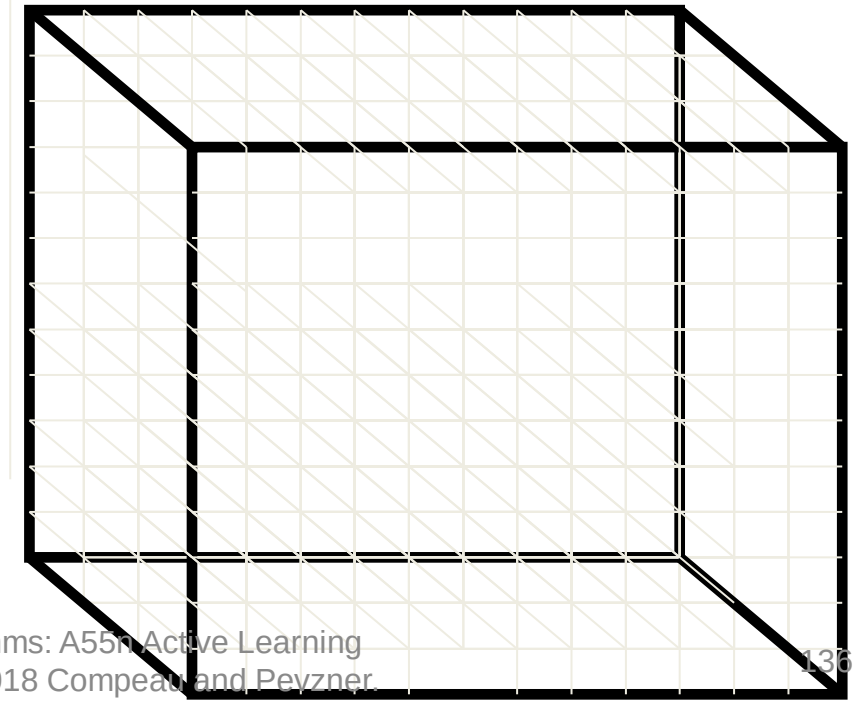
Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Alignments = Paths in 3-D

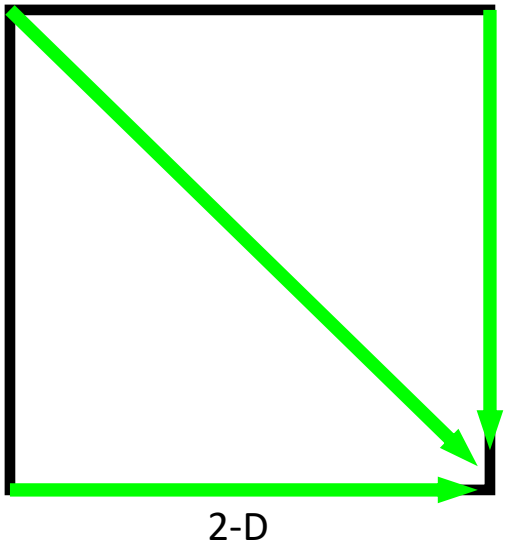
- Alignment of ATGC, AATC, and ATGC

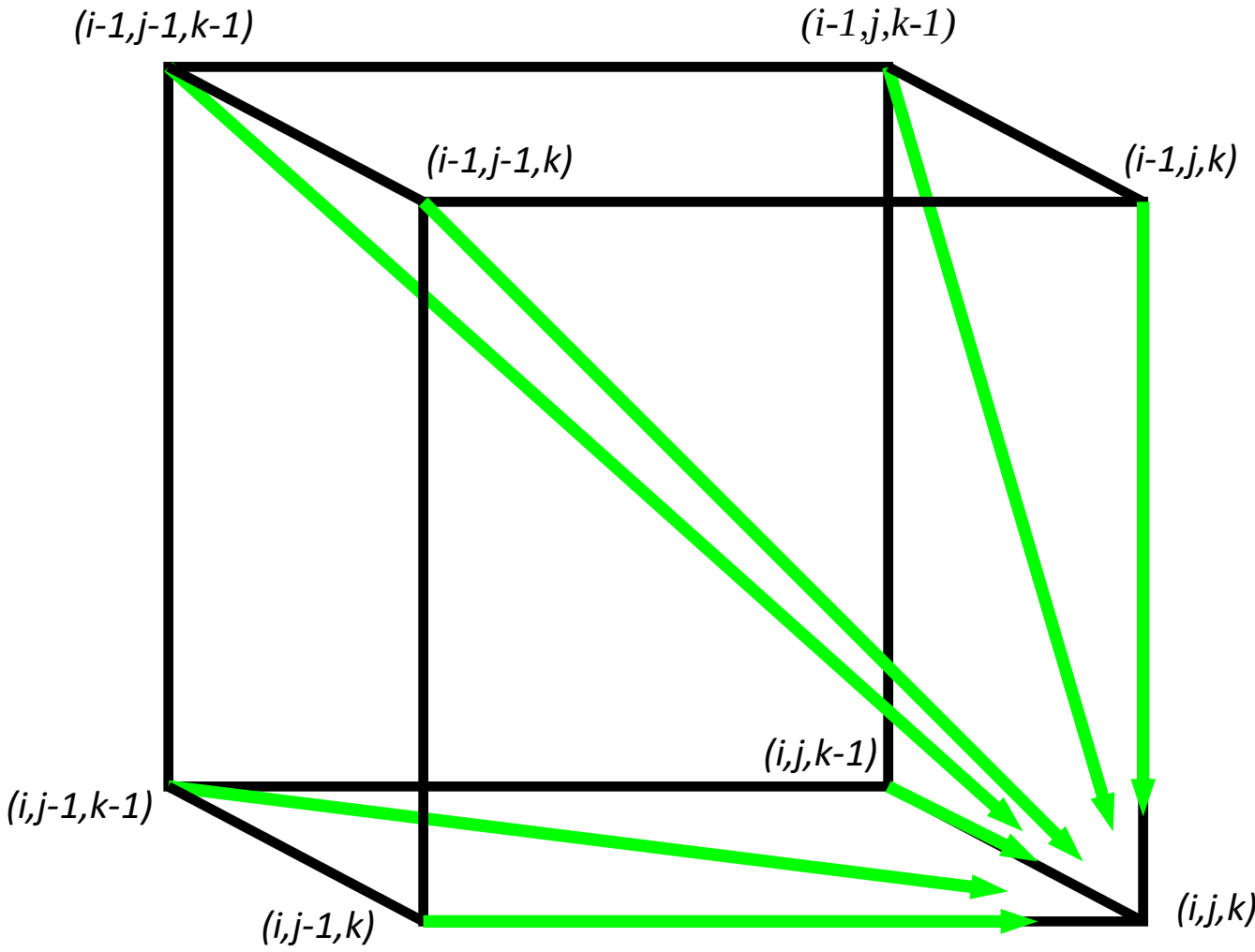
(0,0,0)→(1,1,0)→(1,2,1) →(2,3,2) →(3,3,3) →(4,4,4)

0	1	1	2	3	4
	A	--	T	G	C
0	1	2	3	3	4
	A	A	T	--	C
0	0	1	2	3	4
	--	A	T	G	C



2-D Alignment Cell versus 3-D Alignment Cell





Bioinformatics Algorithms: A55n Active Learning Approach.
Copyright 2018 Compeau and Pevzner.

Multiple Alignment: Dynamic Programming

$$s_{i,j,k} = \max \begin{cases} s_{i-1,j-1,k-1} + \delta(v_i, w_j, u_k) \\ s_{i-1,j-1,k} + \delta(v_i, w_j, -) \\ s_{i-1,j,k-1} + \delta(v_i, -, u_k) \\ s_{i,j-1,k-1} + \delta(-, w_j, u_k) \\ s_{i-1,j,k} + \delta(v_i, -, -) \\ s_{i,j-1,k} + \delta(-, w_j, -) \\ s_{i,j,k-1} + \delta(-, -, u_k) \end{cases}$$

- $\delta(x, y, z)$ is an entry in the 3-D scoring matrix.

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Multiple Alignment: Running Time

- For 3 sequences of length n, the run time is proportional to $7n^3$

- For a k-way alignment, build a k-dimensional Manhattan graph with
  - n^k nodes
  - most nodes have $2^k - 1$ incoming edges.
  - Runtime: $O(2^k n^k)$

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Multiple Alignment Induces Pairwise Alignments

Every multiple alignment induces pairwise alignments:

```
A C - G C G G - C
A C - G C - G A G
G C C G C - G A G
```



```
ACGCGG-C    AC-GCGG-C    AC-GCGAG
ACGC-GAC    GCCGC-GAG    GCCGCGAG
```

Idea: Construct Multiple from Pairwise Alignments

Given a set of **arbitrary** pairwise alignments, can we construct a multiple alignment that induces them?

```
AAAATTTT----        ----AAAATTTT        TTTTGGGG----
----TTTTGGGG        GGGGAAAA----        ----GGGGAAAA
```

Profile Representation of Multiple Alignment

	-	A	G	G	C	T	A	T	C	A	C	C	T	G
	T	A	G	–	C	T	A	C	C	A	-	-	-	G
	C	A	G	–	C	T	A	C	C	A	-	-	-	G
	C	A	G	–	C	T	A	T	C	A	C	–	G	G
	C	A	G	–	C	T	A	T	C	G	C	–	G	G
A	0	1	0	0	0	0	1	0	0	.8	0	0	0	0
C	.6	0	0	0	1	0	0	.4	1	0	.6	.2	0	0
G	0	0	1	.2	0	0	0	0	0	.2	0	0	.4	1
T	.2	0	0	0	0	1	0	.6	0	0	0	0	.2	0
-	.2	0	0	.8	0	0	0	0	0	0	.4	.8	.4	0

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Aligning Sequence Against Sequence

- In the past we were aligning a **sequence against a sequence**.

	-	A	G	G	C	T	A	T	C	A	C	C	T	G
	T	A	G	–	C	T	A	C	C	A	-	-	-	G
	C	A	G	–	C	T	A	C	C	A	-	-	-	G
	C	A	G	–	C	T	A	T	C	A	C	–	G	G
	C	A	G	–	C	T	A	T	C	G	C	–	G	G
A	0	1	0	0	0	0	1	0	0	.8	0	0	0	0
C	.6	0	0	0	1	0	0	.4	1	0	.6	.2	0	0
G	0	0	1	.2	0	0	0	0	0	.2	0	0	.4	1
T	.2	0	0	0	0	1	0	.6	0	0	0	0	.2	0
-	.2	0	0	.8	0	0	0	0	0	0	.4	.8	.4	0

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Aligning Sequence Against Profile

- In the past we were aligning a **sequence against a sequence**.
  - Can we align a **sequence against a profile?**

	-	A	G	G	C	T	A	T	C	A	C	C	T	G
	T	A	G	–	C	T	A	C	C	A	-	-	-	G
	C	A	G	–	C	T	A	C	C	A	-	-	-	G
	C	A	G	–	C	T	A	T	C	A	C	–	G	G
	C	A	G	–	C	T	A	T	C	G	C	–	G	G
A	0	1	0	0	0	0	1	0	0	.8	0	0	0	0
C	.6	0	0	0	1	0	0	.4	1	0	.6	.2	0	0
G	0	0	1	.2	0	0	0	0	0	.2	0	0	.4	1
T	.2	0	0	0	0	1	0	.6	0	0	0	0	.2	0
-	.2	0	0	.8	0	0	0	0	0	0	.4	.8	.4	0

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Aligning Profile Against Profile

- In the past we were aligning a **sequence against a sequence.**
  - Can we align a **sequence against a profile?**
  - Can we align a **profile against a profile?**

	-	A	G	G	C	T	A	T	C	A	C	C	T	G
	T	A	G	–	C	T	A	C	C	A	-	-	-	G
	C	A	G	–	C	T	A	C	C	A	-	-	-	G
	C	A	G	–	C	T	A	T	C	A	C	–	G	G
	C	A	G	–	C	T	A	T	C	G	C	–	G	G
A	0	1	0	0	0	0	1	0	0	.8	0	0	0	0
C	.6	0	0	0	1	0	0	.4	1	0	.6	.2	0	0
G	0	0	1	.2	0	0	0	0	0	.2	0	0	.4	1
T	.2	0	0	0	0	1	0	.6	0	0	0	0	.2	0
-	.2	0	0	.8	0	0	0	0	0	0	.4	.8	.4	0

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Multiple Alignment: Greedy Approach

- Choose the most similar sequences and combine them into a profile, thereby reducing alignment of *k* sequences to an alignment of of *k* – 2 sequences and 1 profile.
- Iterate

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Greedy Approach: Example

- Sequences: GATTCA, GTCTGA, GATATT, GTCAGC.

- 6 pairwise alignments (premium for match +1, penalties for indels and mismatches -1)

```
s2  GTCTGA
s4  GTCAGC   (score = 2)

s1  GAT-TCA
s2  G-TCTGA  (score = 1)

s1  GAT-TCA
s3  GATAT-T  (score = 1)

s1  GATTCA--
s4  G-T-CAGC (score = 0)

s2  G-TCTGA
s3  GATAT-T  (score = -1)

s3  GAT-ATT
s4  G-TCAGC  (score = -1)
```

Bioinformatics Algorithms: A55n Active Learning Approach. Copyright 2018 Compeau and Pevzner.

Greedy Approach: Example

- Since s_2 and s_4 are closest, we consolidate them into a profile:

$$\left.\begin{array}{ll} s2 & \text{GTCTGA} \\ s4 & \text{GTCAGC} \end{array}\right\} s_{2,4} = \text{GTCt/aGa/cA}$$

- New set of 3 sequences to align:

s_1 GATTCA
s_3 GATATT
$s_{2,4}$ GTCt/aGa/c